AKADEMIKA

WL-WIFI
Wireless Fidelity
EXPERIMENTAL MANUAL

…. A MESSAGE FROM

Today's system designers are faced with tomorrow's problems. Digital Communication is one of the important subject need to teach while learning electronics.
It is our vision to provide you with the product you need for training ensuring lasting reliability & quality.

OUR MOTTO;

- Light years ahead – refers to leadership.

As leaders in our industry in India, we are totally committed to servicing as the standard against which all are measured in the areas of

• Design • Quality • Value • Delivery • Support.

We are truly light years ahead of our competition in this area. That means that you our valued customers are guaranteed satisfaction.

As you will move through this manual you will quickly discover that We have a complete, truly innovative & superior training products we are so committed to quality that we back our products with a complete comprehensive warranty.

AKADEMIKA LAB SOLUTIONS

Sr.No.15/8/1,Unit No. 9 Kruti Industrial Estate,

Opp Karishma Society, Off Karve Road,Kothrud,

Pune – 411038,India

ContactNo.: +91-7447438443

Email: lab@akademika.in

www.akademika.in

SAFETY RULES

Carefully follow the instructions contained in this manual as they provide you with important points on safety during the installation, use and maintenance. Keep this manual always with you for easy reference.

Arrange all accessories in order after unpacking, so that its integrity is checked with respect to its checklist. Also ensure that no visible damage as such appear on any accessories.

Before connecting the power supply to the kit, be sure that the jumpers or connecting chords are connected correctly as per experiment.

These kits must be employed only for the use for which it has been conceived, i.e.as educational equipment, and must be used under the direct survey of expert personnel. Any other use is improper and so dangerous. The manufacturer cannot be considered responsible for eventual damages due to improper, wrong or unreasonable uses.

In case of any fault or malfunctioning in the trainer kit, turn off the power supply and do not tamper the kit. In case servicing is required, contact the service center for technical assistance.

The kits are liable to malfunction/under-perform if they are not operated under following conditions,

- Ambient temperature: from 0 to 45 ° C.
- Relative humidity: from 20 to 80 %.

Avoid any immediate/significant change of temperature and humidity.

WARRANTY

These kits are warranted against defects in workmanship and materials. Any failure due to defect in either workmanship or material should occur under normal use within a year from the original date of purchase, such failure will be corrected free of charge to the purchaser by repair or replacement of defective part or parts. When the failure is result of user's neglect, natural disaster or accident, we charge for repairs regardless of the warranty period. The warranty does not cover perishable items like connecting chords, Crystals, etc. and other imported items.

This warranty is subject to the following conditions and limitations. The warranty is void and inapplicable if the defective product is not brought or sent to our authorized service center or sales outlet within the warranty period. Defective product is, on Akademika Lab Solutions sole judgment. The defective product will be replaced with new one or repaired without charge or with charge.

In the warranty period if the service is needed, purchaser should get in touch with the service center or sales outlet. The purchaser should return the product to the service center or sales outlet at his or her sole expense. The loss and damage in transit will be outside the preview of this warranty. A returned product must be accompanied by a written description of the defects. Type and Model No. of kits has to be mentioned specifically. We return the product to the purchaser at our expense. In case that warranty does not cover the product on Akademika Lab Solutions judgment, we repair the product after obtaining prior permission from purchaser who receives an estimate statement about repairing charges. In such case, Akademika Lab Solutions bares the transporting expenses required to send back all the repaired products for the moment, and then repairs and transporting expenses will be charged against the purchaser by the statement of accounts.

When the authorized sales agents sell our products, they must notify the purchaser of the warranty contents, but have no rights to stretch the meaning of original warranty contents or offer additional warranty. Akademika Lab Solutions does not provide any other promise or suggestive warranty and hold no liability for the damage caused by negligence, abnormal use or natural disaster. Akademika Lab Solutions is not responsible for the damages even if it is notified of above dangers in advance as well.

For more special service or overall repairs, maintenance and up gradation of products, be sure to contact our service center or sales outlet.

INDEX

TECHNICAL SPECIFICATIONS

INTRODUCTION

WL-WIFI Trainer is designed to study the WI-FI Wireless Technology IEEE 802.11 Standard and it covers all Topologies & Protocol. Trainer carrying a powerful WIFI Transceiver ESP-32-WROOM and it works with arduino ESP32 plug-in. It transfers data using WI-FI Services to and from the Base Station and the Remote Station. Trainer provides full accessibility to the user to Communicate between Remote Device and WL-WI-FI in both directions. It is capable of Controlling WL-WI-FI Peripherals using Webpage on PC.

General Specifications -

1. Trainer should be a bench top model with an open PCB architecture.
2. Should operate using PC control.
3. Trainer should be implemented with a WI-FI enabled module.
4. Controls should be available through webpage on PC.
5. System should be capable of demonstrating WI-FI functionalities.
6. It should include on board peripherals which can be used for designing various small scale applications.

Technical Specifications - System

WL-WIFI should have following specifications.

1. The Trainer should be based on module ESP32-WROOM, which is a powerful, generic Wi-Fi+BT+BLE MCU module and Compatible with IEEE 802.11b/g/n networks, Frequency range 2.4 GHz ~ 2.5 GHz, Data rate: up to 150 Mbps.
2. System should operate from single voltage supply.
3. Webpage should be designed to initiate WI-FI functionalities.
4. Trainer should be based on following components:
   - **i. ESP32-WROOM WI-FI module:**
     Frequency range 2.4 GHz ~ 2.5 GHz
     Compatible with IEEE 802.11b/g/n networks
     Data rate: up to 150 Mbps
     20 dBm Output Power at the antenna
     Capacitive-sensing GPIOs
     Supply Voltage: 2.7~3.6 VDC
     Two 12-bit SAR ADCs, Two 8-bit DACs
   - **ii. Relay :**
     It is an actuator, 5V GoodSKY SPDT Mechanical Relay
     Operate Time: 10 mSec. Max.
     NO & NC LED indicator

iii. LED & Switch:

One supply indicator LED.
Two user interface input switches.

iv. ADC:

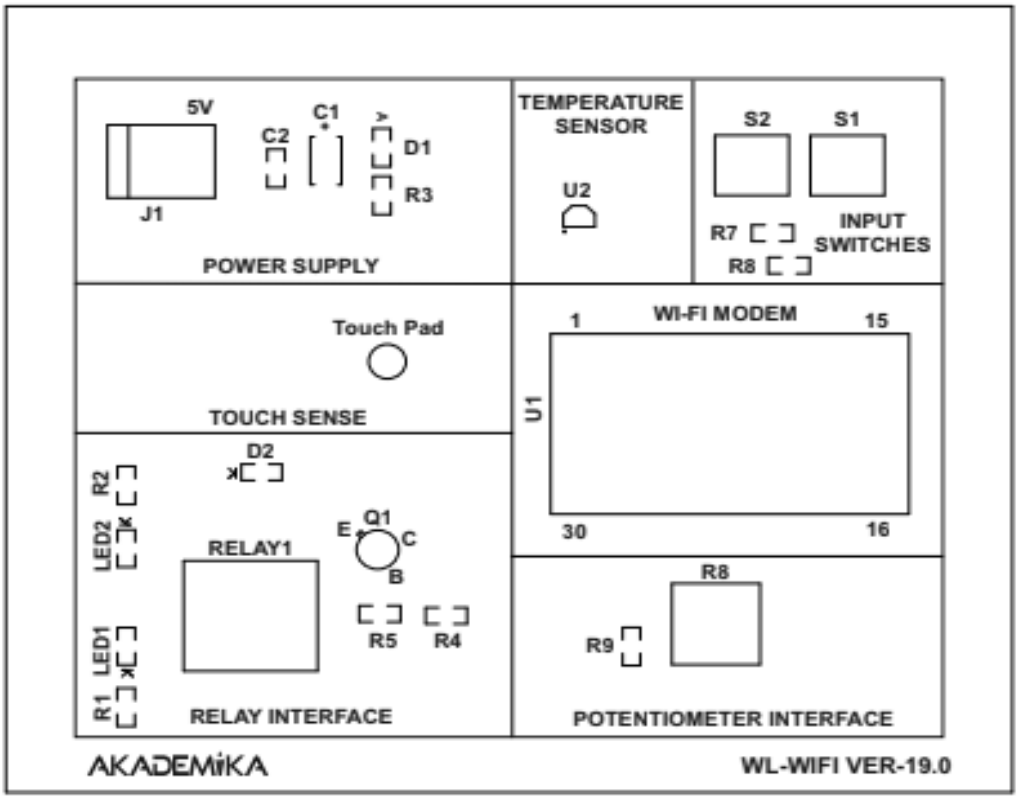
For generation of an analog signals, we should use

- potentiometer :
  Potentiometer value 1K
- Temperature sensor LM35:
  Operating Temperature range: -55°C ≤ TA ≤ +150°C
  Scale Factor: 10 mV/°C

Accessories:

	WL-WIFI	Quantity
1	WL-WIFI Trainer Kits	3
2	USB A Male to Micro-B 1mtr cable	3
3	WL-WIFI EXPT Manual	1
4	Software CD	1
5	Power Supply	3

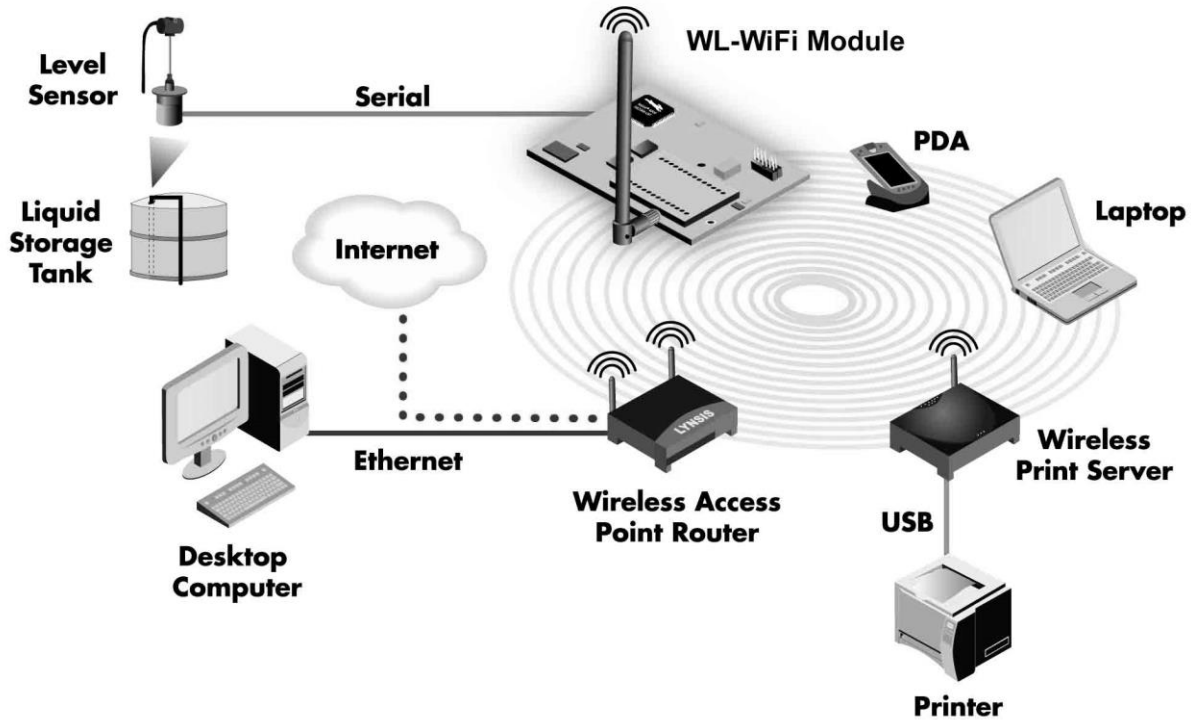
FUNCTIONAL BLOCK



INTRODUCTION

Wi-Fi, also spelled Wi-Fi or Wii, is a local area wireless technology that allows an electronic device to exchange data or connect to the internet using 2.4 GHz UHF and 5 GHz SHF radio waves. A common misconception is that the term Wi-Fi is short for "*wireless fidelity*," however this is not the case. Wi-Fi is simply a trademarked phrase that means *IEEE 802.11x*.

Wi-Fi works with no physical wired connection between sender and receiver by using radio frequency (RF) technology, a frequency within the electromagnetic spectrum associated with radio wave propagation. When an RF current is supplied to an antenna, an electromagnetic field is created that then is able to propagate through space.

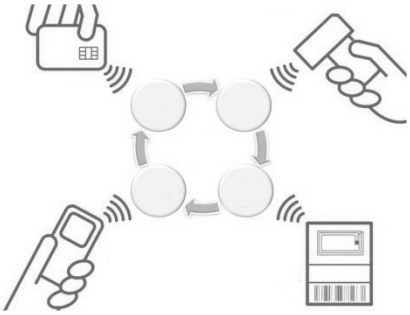


The cornerstone of any wireless network is an access point (AP). The primary job of an access point is to broadcast a wireless signal that computers can detect and "tune" into. In order to connect to an access point and join a wireless network, computers and devices must be equipped with wireless network adapters.

Wi-Fi is supported by many applications and devices including video game consoles, home networks, PDAs, mobile phones, major operating systems, and other types of consumer electronics. The radio frequency band used in Wi Fi technology are 2.5GHz for 802.11b, 802.11g, or 802.11n, and 5GHz for 802.11a

WIRELESS COMMUNICATION SYSTEM

What is a Wireless System?



A wireless system is any collection of elements that operate interdependently and use unguided electromagnetic-wave propagation to perform some specified function.

Some examples of systems that fit this definition are

- Systems that convey information between two or more locations, such as personal communication systems, police and fire department radio systems, commercial broadcast systems, satellite broadcast systems, telemetry and remote monitoring systems.
- Systems that sense the environment or objects in the environment, including radar systems that may be used for detecting the presence of objects in some region or volume of the environment and measuring their relative motion and position, systems for sensing or measuring atmospheric conditions and systems for mapping the surface of the Earth or planets
- Systems that aid in navigation or determine the location of an object on the Earth or in space

Each of these systems contains at least one transmitting antenna and at least one receiving antenna. In the abstract, an antenna may be thought of as any device that converts a guided signal, such as a signal in an electrical circuit or transmission line, into an unguided signal propagating in space, or vice versa. For example, the Wi-Fi local area network computer interface uses a single antenna that is switched between transmitter and receiver.

As the examples show, some systems may be used to convey information, whereas others may be used to extract information about the environment based on how the transmitted signal is travels the path between transmitting and receiving antennas. In either case, the physical and electromagnetic environment in the neighborhood of the path may significantly modify the signal. We define a channel as the physical and electromagnetic environment surrounding and connecting the endpoints of the transmission path that is, surrounding and connecting the system's transmitter and receiver. *(A channel may consist of wires, waveguide and coaxial cable, fiber, the Earth's atmosphere and surface, free space, and so on.)* When a wireless system is used to convey information between endpoints, the environment often corrupts the signal in an unpredictable way and impairs the system's ability to extract the transmitted information accurately at a receiving end.

WIRELESS COMMUNICATIONS

It is a type of data communication that is performed and delivered wirelessly. This is a broad term that incorporates all procedures and forms of connecting and communicating between two or more devices using a wireless signal through wireless communication technologies and devices. Wireless communication generally works through electromagnetic signals that are broadcast by an enabled device within the air,

physical environment or atmosphere. The sending device can be a sender or an intermediate device with the ability to propagate wireless signals. The communication between two devices occurs when the destination or receiving intermediate device captures these signals, creating a wireless communication bridge between the sender and receiver device. Wireless communication has various forms, technology and delivery methods.

The first wireless transmitters went on the air in the early 20th century using radiotelegraphy. Later, as modulation made it possible to transmit voices and music via wireless, the medium came to be called "RADIO". With the advent of television, fax, data communication, and the effective use of a larger portion of the spectrum, the term "wireless" has been resurrected.

Common examples of wireless equipment in use today include:

Cellular phones and pagers -- provide connectivity for portable and mobile applications, both personal and business.
Global Positioning System (GPS) -- allows drivers of cars and trucks, captains of boats and ships, and pilots of aircraft to ascertain their location anywhere on earth.
Cordless computer peripherals -- the cordless mouse is a common example; keyboards and printers can also be linked to a computer via wireless.
Cordless telephone sets -- these are limited-range devices, not to be confused with cell phones.
Home-entertainment-system control boxes -- the VCR control and the TV channel control are the most common examples; some hi-fi sound systems and FM broadcast receivers also use this technology.
Two-way radios -- this includes Amateur and Citizens Radio Service, as well as business, marine, and military communications.
Baby monitors -- these devices are simplified radio transmitter/receiver units with limited range.
Satellite television -- allows viewers in almost any location to select from hundreds of channels.
Wireless LAN or Local Area Networks -- provide flexibility and reliability for business computer users.

Wireless technology is rapidly evolving, and is playing an increasing role in the lives of people throughout the world. In addition, ever-larger numbers of people are relying on the technology directly or indirectly. More specialized and exotic examples of wireless communications and control include:

Global System for Mobile Communication (GSM) -- a digital mobile telephone system used in Europe and other parts of the world.
General Packet Radio Service (GPRS) -- a packet-based wireless communication service that provides continuous connection to the Internet for mobile phone and computer users.
Enhanced Data GSM Environment (EDGE) -- a faster version of the Global System for Mobile (GSM) wireless service.

Universal Mobile Telecommunications System (UMTS) -- a broadband, packet-based system offering a consistent set of services to mobile computer and phone users no matter where they are located in the world.
Wireless Application Protocol (WAP) -- a set of communication protocols to standardize the way that wireless devices, such as cellular telephones and radio transceivers, can be used for Internet access.
i-Mode -- the world's first "smart phone" for Web browsing, first introduced in Japan; provides color and video over telephone sets.

Wireless can be divided into:

FIXED WIRELESS -- the operation of wireless devices or systems in homes and offices, and in particular, equipment connected to the Internet via specialized modems.

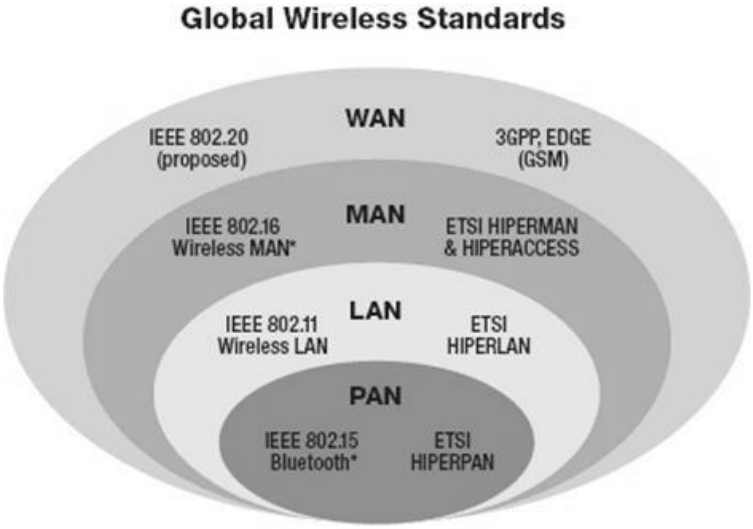
MOBILE WIRELESS -- the use of wireless devices or systems aboard motorized, moving vehicles; examples include the automotive cell phone and PCS (personal communications services).

PORTABLE WIRELESS -- the operation of autonomous, battery-powered wireless devices or systems outside the office, home, or vehicle; examples include handheld cell phones and PCS units.

IR WIRELESS -- the use of devices that convey data via IR (infrared) radiation; employed in certain limited-range communications and control systems.

WIRELESS STANDARDS

Different methods and standards of wireless communication have developed across the world, based on various commercially driven requirements. These technologies can roughly be classified into four individual categories, based on their specific application and transmission range. These categories are summarized in the figure below



PERSONAL AREA NETWORK (PAN)

A Personal Area Network (PAN) is a computer network used for communication among computer devices (including telephones and personal digital assistants) close to one person. The reach of a PAN is typically a few meters. PAN's can be used for communication among the personal devices themselves (intrapersonal communication), or for connecting to a higher-level network and the Internet.

Personal area networks may be wired with computer buses such as USB and FireWire. However, a Wireless Personal Area Network (WPAN) is made possible with network technologies such as Infrared (IrDA) and Bluetooth.

Bluetooth: Bluetooth is an industrial specification for wireless personal area networks (PANs), also known as IEEE 802.15.1. Bluetooth provides a way to connect and exchange information between devices such as personal digital assistants (PDAs), mobile phones, laptops, PCs, printers, digital cameras and video game consoles via a secure, globally unlicensed short-range radio frequency. Bluetooth is a radio standard and communications protocol primarily designed for low power consumption, with a short range (power class dependent: 1 meter, 10 meters, 100 meters) based around low-cost transceiver microchips in each device.

Infrared (IrDA): The Infrared Data Association (IrDA) defines physical specifications communications protocol standards for the short-range exchange of data over infrared light, for typical use in Personal Area Networks.

LOCAL AREA NETWORK (LAN)

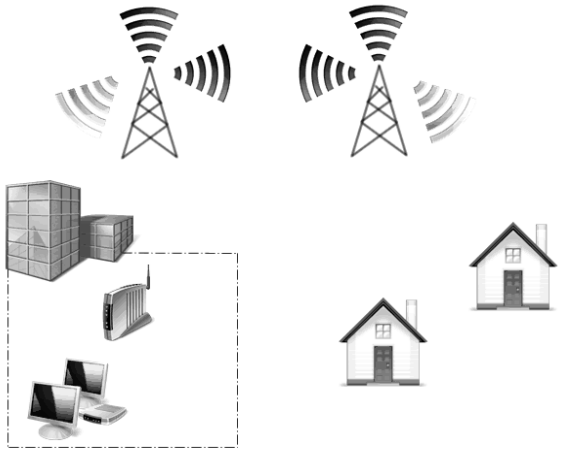
A wireless LAN or WLAN is a wireless Local Area Network, which is the linking of two or more computers without using wires. It uses radio communication to accomplish the same functionality that a wired LAN has.

WLAN utilizes spread-spectrum technology based on radio waves to enable communication between devices in a limited area, also known as the basic service set. This gives users the mobility to move around within a broad coverage area and still be connected to the network.

IEEE 802.11: IEEE 802.11, the Wi-Fi standard, denotes a set of Wireless LAN/WLAN standards developed by working group 11 of the IEEE LAN/MAN Standards Committee (IEEE 802). The 802.11 family currently includes six over-the-air modulation techniques that all use the same protocol. The most popular (and prolific) techniques are those defined by the b, a, and g amendments to the original standard.

METROPOLITAN AREA NETWORK (MAN)

Wireless Metropolitan Area Network (MAN) is the name trademarked by the IEEE 802.16 Working Group on Broadband Wireless Access Standards for its wireless metropolitan area network standard (commercially known as WiMAX), which defines broadband Internet access from fixed or mobile devices via antennas. Subscriber stations communicate with base-stations that are connected to a core network. This is a good alternative to fixed line networks and it is simple to build and relatively inexpensive



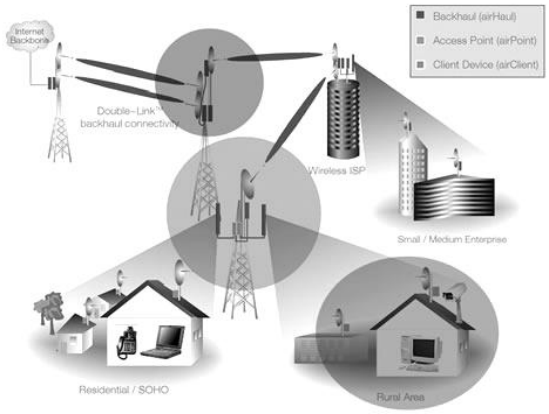
WiMAX: it is defined as Worldwide Interoperability for Microwave Access by the WiMAX Forum, formed in June 2001 to promote conformance and interoperability of the IEEE 802.16 standard, officially known as Wireless MAN. The Forum describes WiMAX as "a standards-based technology enabling the delivery of last mile wireless broadband access as an alternative to cable and DSL".

The range of WiMAX probably generates more confusion than any other single aspect of WiMAX. It is common to see statements in the media describing WiMAX multipoint coverage extending 30 miles. In a strict technical sense (in some spectrum ranges) this is correct, with even greater ranges being possible in point to point links. In practice in the real world (and especially in the license-free bands) this is wildly overstated especially where non line of sight (NLOS) reception is concerned.

WIDE AREA NETWORK (WAN)

A Wide Area Network or WAN is a computer network covering a broad geographical area. Contrast with personal area networks (PAN's), local area networks (LAN's) or metropolitan area networks (MAN's) that are usually limited to a room, building or campus. The largest and most well-known example of a WAN is the Internet.

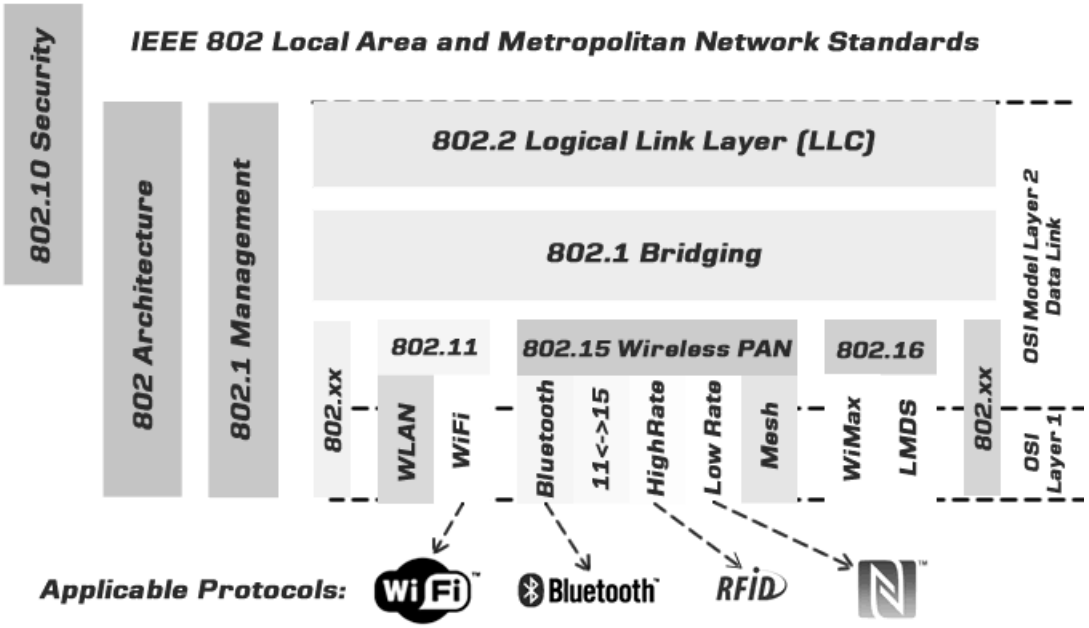
WAN's are used to connect local area networks (LAN's) together, so that users and computers in one location can communicate with users and computers in other locations. Many WAN's are built for one particular organization and are private. Others, built by Internet service providers, provide connections from an organization's LAN to the Internet.



In addition, WAN's also referred to Mobile Data Communications, such as GSM, GPRS and 3G. Please refer to our Mobility section for further details.

TECHNOLOGY SUMMARY

These wireless communication technologies evolved over time to enable the transmission of larger amounts of data at greater speeds across a global network. The following figure summarizes the technical details of each cluster of technologies.

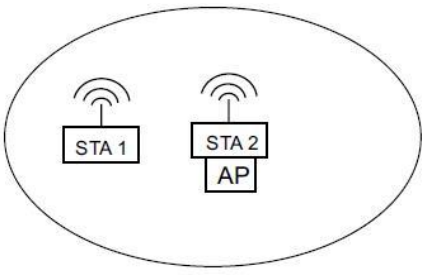


WIFI TECHNOLOGY ARCHITECTURE & TOPOLOGY

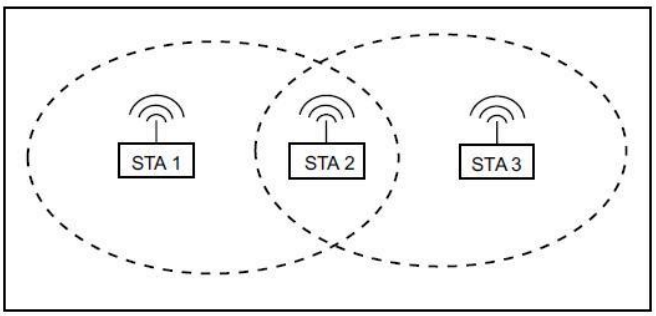
The architectural components defined by the 802.11 standard are describes the structure and organization of the network. This knowledge informs various tasks, such things as selecting the right operating mode to suit your application or completing an effective site survey for the physical location of the network.

Basic Components

All wireless devices that join a Wi-Fi network, whether mobile, portable or fixed, are called wireless stations (STAs). A wireless station might be a PC, a laptop, a PDA, a phone, or a Rabbit core module. When two or more STAs are wirelessly connected, they form a basic service set (BSS). This is the basic building block of a Wi-Fi network.



BSS



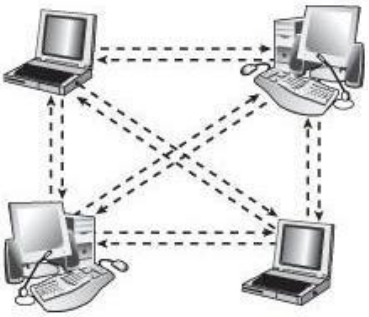
A BSS is a set of STAs controlled by a single coordination function (CF). The CF is a logical function that determines when a STA transmits and when it receives. The BSS has shown an example of the simplest

Wi-Fi network possible: two wireless stations. The oval shape around them roughly represents the coverage area. While a circle may represent the idealized coverage area of a single radio, it is not very accurate in real world situations. Environmental factors cause dramatic variations to the coverage area. For example, a STA with an omnidirectional antenna placed in the corner of a building may have most of its coverage area outside the building and in the adjacent parking lot. Not all STAs in a BSS can necessarily communicate directly. Consider second figure STA 1 and 3 are mutually out of range, thus require use of STA 2 to relay messages.

Different Operating Modes

This section discusses the two operating modes specified in the IEEE 802.11 standard: infrastructure mode and ad-hoc mode. Each one makes use of the BSS, but they yield different network topologies.

Ad-Hoc Mode

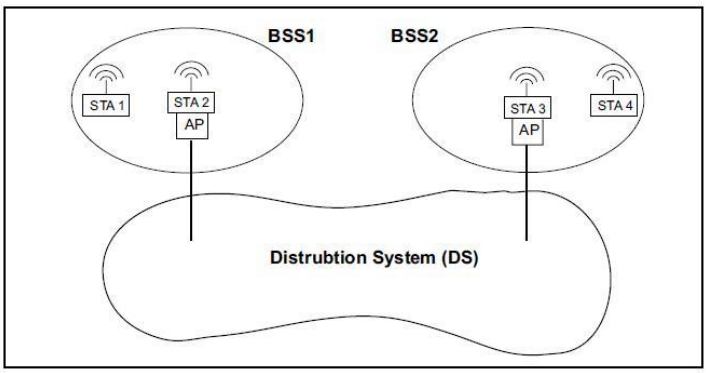


The independent BSS (IBSS) is the simplest type of 802.11 networks. Wireless stations communicate directly with one another using the ad-hoc operating mode. Such a network follows a peer-to-peer model. A BSS operating in ad-hoc mode is isolated. There is no connection to other Wi-Fi networks or to any wired LANs. Even so, the ad-hoc mode can be very useful in a number of situations. Because an ad-hoc network can spring up anywhere, it is especially useful in situations that demand a quick setup in areas that do not have any infrastructure, such as emergency sites and combat zones.

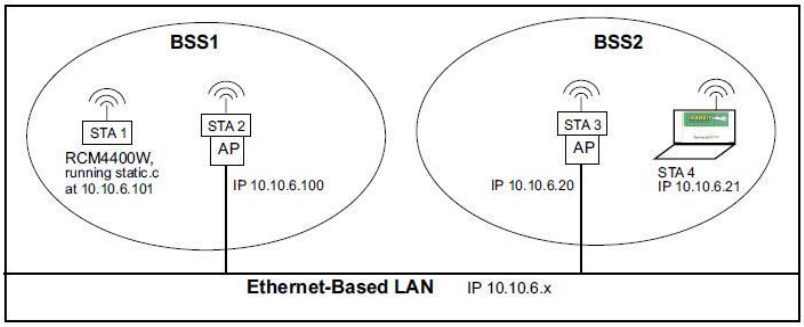
As another example of the usefulness of ad-hoc Wi-Fi, consider that it is now common for people to have their laptops with them in business meetings, in airports waiting for flights, or even at their local coffee hang-out. Operating in ad-hoc mode, people can easily and quickly form a network to do things like share large files or anything else they may want to do cooperatively.

Infrastructure Mode

The infrastructure operating mode requires that the BSS contain one wireless access point (AP). An AP is a STA with additional functionality. A major role for an AP is to extend access to wired networks for the clients of a wireless network. All wireless devices trying to join the BSS must associate with the AP. An AP provides access to its associated STAs to what is called the distribution system (DS). The DS is an architectural component that allows communication among APThe IEEE 802.11 specification does not define any physical characteristics or physical implementations for the DS. Instead, it defines services that the DS must provide. The DS medium comprises the physical components of a specific DS implementation, e.g., coaxial cabling or fiber optic cabling.



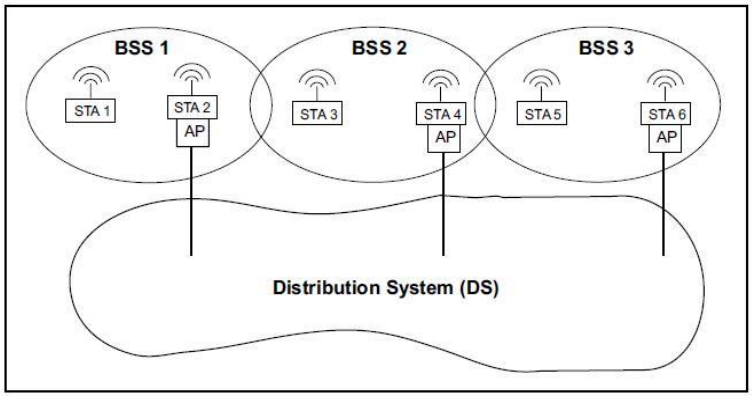
The DS medium is logically different from the wireless medium (WM). Thus, the addresses used by the AP on the wireless medium and the addresses used on the DS medium do not have to be the same.



All wireless communication to or from an associated STA goes through the AP when the network is configured to use infrastructure mode. This setup is similar to the host/hub model (or "star topology") used frequently in wired networks. In the 802.11 specification, both AP ("hub") and "host" are called wireless stations or STAs.
To make this concept clearer we will look at a concrete example using these architectural components. The serves have static web page that is then viewable from the laptop that has joined a separate Wi-Fi network, BSS2, on the same subnet. A wireless station that created or joined a BSS on that subnet would be able to communicate with any other wireless station on that subnet. The AP acts as a wireless hub or switch.

Another architectural component described by 802.11, called a portal, is also needed for the access to another system.
The portal and DS access functionality can be implemented in the same device and typically are, but there is no requirement in the IEEE 802.11 standard that this must be the case.

Extended Service Set



A common distribution system (DS) and two or more BSSs create what is called an extended service set (ESS). An ESS is a Wi-Fi network of arbitrary size and complexity. Figure representation of an ESS comprised of BSS 1, 2 and 3. The distribution system is not part of the ESS. The distribution system enables mobility in a Wi-Fi network by a method of tracking the physical location of STAs, thus ensuring that frames1 are delivered to the AP associated with the destination STA. Mobility means a wireless client can move anywhere within the coverage area of the ESS and keep an uninterrupted connection. However, please note that since the 802.11 specification does not specify how the DS works, APs from different vendors may not work well with each other to provide an uninterrupted connection. The network name, or SSID, must be the same for all APs participating in the same ESS.

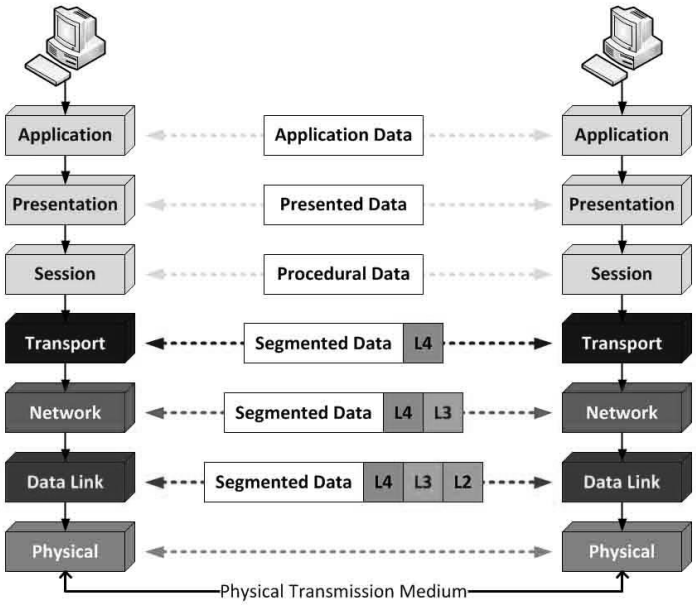
OSI & TCP/IP LAYER CONCEPT

In wired or wireless, most data communications today happen by way of packets of information travelling over one or more networks. But before these networks can work together, though, they must use a common protocol, or a set of rules for transmitting and receiving these packets of data. Many protocols have been developed. One of the most widely used is the Transmission Control Protocol/Internet Protocol (TCP/IP). Also, a generic protocol model used in describing network communications known as the Open System Interconnection (OSI) model is useful for comparing and contrasting

OSI Model Details

Designated ISO/IEC 7498-1, the OSI model is a standard of the International Organization for Standardization (ISO). It is a general-purpose paradigm for discussing or describing how computers communicate with one another over a network. Its seven-layered approach to data transmission divides the many operations up into specific related groups of actions at each layer.

The transmitting computer software gives the data to be transmitted to the applications layer, where it is processed and passed from layer to layer down the stack with each layer performing its designated functions. The data is then transmitted over the physical layer of the network until the destination computer or another device receives it. At this point the data is passed up through the layers again, each layer performing its assigned operations until the data is used by the receiving computer's software. During transmission, each layer adds a header to the data that directs and indentifies the packet. This process is called encapsulation. The header and data together form the data packet for the next layer that, in turn, adds its header and so on. The combined encapsulated packet is then transmitted and received. The receiving computer reverses the process, de-encapsulating the data at each layer with the header information directing the operations. Then, the application finally uses the data. The process is continued until all data is transmitted and received.



All of the necessary and desirable operations required are grouped together in a logical sequence at each of the layers. Each layer is responsible for specific functions:

Layer 7 – application: This layer works with the application software to provide communications functions as required. It verifies the availability of a communications partner and the resources to support any data transfer. It also works with end applications such as domain name service (DNS), file transfer protocol (FTP), hypertext transfer protocol (HTTP), Internet message access protocol (IMAP), post office protocol (POP), simple mail transfer protocol (SMTP), Telnet, and terminal emulation.

Layer 6 – presentation: This layer checks the data to ensure that it is compatible with the communications resources. It ensures compatibility between the data formats at the applications level and the lower levels. It also handles any needed data formatting or code conversion, as well as data compression and encryption.

Layer 5 – session: Layer 5 software handles authentication and authorization functions. It also manages the connection between the two communicating devices, establishing a connection, maintaining the connection, and ultimately terminating it. This layer verifies that the data is delivered as well.

Layer 4 – transport: This layer provides quality of service (QoS) functions and ensures the complete delivery of the data. The integrity of the data is guaranteed at this layer via error correction and similar functions.

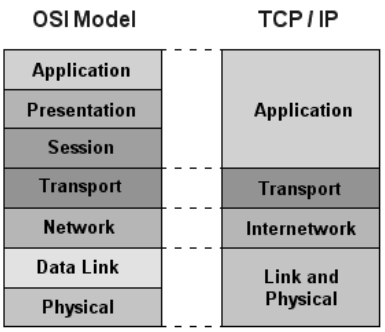
Layer 3 – network: The network layer handles packet routing via logical addressing and switching functions.
Layer 2 – data link: Layer 2 operations package and unpack the data in frames.

Layer 1 – physical: This layer defines the logic levels, data rate, physical media, and data conversion functions that make up the bit stream of packets from one device to another.

There are two key points to make about the OSI model.

First, the OSI model is just that, a model. Its use is not mandated for networking, yet most protocols and systems adhere to it quite closely. It is mainly useful for discussing, describing, and understanding individual network functions.
Second, not all layers are used in some simpler applications. While layers 1, 2, and 3 are mandatory for any data transmission, the application may use some unique interface layer to the application instead of the usual upper layers of the model.

TCP/IP



TCP/IP was developed during the 1960s as part of the Department of Defense's (DoD) Advanced Research Projects Agency (ARPA) effort to build a nationwide packet data network. It was first used in UNIX-based computers in universities and government installations. Today, it is the main protocol used in all Internet operations. TCP/IP also is a layered protocol but does not use all of the OSI layers, though the layers are equivalent in operation and function. The network access layer is equivalent to OSI layers 1 and 2. The Internet Protocol layer is comparable to layer 3 in the OSI model. The host-to-host layer is equivalent to OSI layer 4. These are the TCP and UDP (user datagram protocol) functions. Finally, the application layer is similar to OSI layers 5, 6, and 7 combined.

The TCP layer packages the data into packets. A header that's added to the data includes source and destination addresses, a sequence number, an acknowledgment number, a check sum for error detection and correction, and some other information.

The header is 20 octets (octet = 8 bits) grouped in 32-bit increments. These bits are transmitted from left to right and top to bottom.

At the receiving end of the link, TCP reassembles the packets in the correct order and routes them up the stack to the application. TCP can retransmit a packet if an error occurs. In any case, TCP's main job is just to pack and unpack the data and provide some assurance of the reliable transmission of error-free data. The IP layer actually transmits the TCP packet.

The IP layer transmits the data over the physical-layer connection. IP adds its own header to the packet. The header comprises 32 octets again grouped in 32-bit words. Note the 32-bit source and destination addresses. These are the well-known IP addresses that we see in dotted decimal format (e.g., 197.45.204.36) where each 8-bit octet is expressed in its decimal value. This is the address assigned to the device by the Internet Assigned Numbers Authority (IANA).

The header in IP version 4 (IPv4), since the IANA has run out of 32-bit addresses a newer version is rapidly being adopted. IPv6 uses 128-bit addresses. With 2^{128} addresses, there should be enough for all of the planet's computers, tablets, and smart phones as well as all of the devices that may be connected to form the so-called Internet of Things (IoT).

Once the IP header is added to the data, it is transferred to the Network Access layer. This layer repackages the data again into Ethernet packets or some other protocol for final physical transmission. The Ethernet packets are then reconfigured again for transmission over a DSL or cable TV connection or over a wide-area network using Sonet or optical transport network (OTN).

Application

Equivalent to Application, Presentation and Session in OSI
HTTP, FTP, SMTP, SSH, DNS, DHCP etc.
The layer provides a way for applications to use network services.
Real world example: google.com

Transport (Host-to-Host)

It is responsible for ensuring logical connections between hosts and transfer of data. In other words, it acts as the delivery service used by the application layer above.
Data can be delivered with or without reliability.
TCP keeps track of packets sent and received, hence reliable.

If A is sending a stream of data to B, TCP will split it into smaller blocks called segments. Each segment is checked for errors, and retransmitted if any are found. A knows whether a segmented contains errors depending upon the acknowledgement it receives from B.

Note that multiple segments can be sent in-between acknowledgements to minimize waiting time. If an error is detected, the faulty data will be retransmitted.

UDP is connectionless, meaning that A doesn't initiate a transport connection to B before it starts sending data. If B detects an error, it simply ignores it, leaving any

potential fix entirely up to A. UDP (User Datagram Protocol) does not keep track of packets received, hence “unreliable” (but faster). It is Useful for streaming video, VIOP and other services where lost packets are not of great concern.

Whether TCP or UDP is used depends on what the application is set up to do and in the case of a browser and HTTP, TCP is the default method used.

Internet

Equivalent to Network layer in OSI.
Ensures routing of data between different networks and subnets.
IP and ARP are the main protocols used here.

ARP is used to convert IPs to physical addresses. In this case, a physical address means the MAC address.

Network

Equivalent to Data Link and Physical layer in OSI.
This explains why TCP/IP omits “physical” as a layer in its model.
It does things such as place data in frames and passes it between hosts.
To learn more about frames, read: Introduction to Ethernet

WIFI IEEE 802.11 STANDARDS

Today there are a lot of standards used for wireless networking, this section giving a brief overview of the 802.11 standards defined by IEEE, but the most popular standard which is IEEE 802.11b. The IEEE 802.11 specifications are wireless standards that specify an "over-the-air" interface between a wireless client and a base station or access point, as well as among wireless client. IEEE 802.11 standard primarily addresses two separate layers of the ISO networking model:

Physical network layer (PHY) - lowest ISO layer that defines the physical transmission characteristics of the signal - in this case, radio signal such as the frequency, power levels, and type of modulation.

The Media Access Control layer (MAC) - is mostly made up of software- based protocols that enable devices to talk to each other.

These IEEE 802.11 standards are specifying 9 standards, IEEE 802.11 a-i.

IEEE 802.11a (called as wifi5)

It is a Physical Layer (PHY) standard that works in unlicensed 5-GHz radio band using Orthogonal Frequency Division Multiplexing (OFDM). It supports data rates from 6 Mbps up to 54 Mbps. It will use the same MAC layer as 802.11. Products started appearing in 3Q 2001 and 1Q 2002. The 802.11a standard includes features like priority for certain types of traffic; also, it offers much less potential for radio frequency (RF) interference than other PHYs (e.g. 802.11b and 802.11g). With high data rates and relatively little interference, 802.11a does a great job of supporting multimedia applications and densely populated user environments.

IEEE 802.11b (also called as Wi-Fi)

This standard that supports speeds up to 5.5Mbps and 11 Mbps, in addition to the 1Mbps and 2Mbps data rates. It's deployed in 2.4 GHZ radio band. IEEE 802.11b uses CCK (Complementary Code Keying) to provide the high data rates. IEEE 802.11 finalized this standard (IEEE Std. 802.11b-1999) in late
1999. Several vendors offer products conforming to this standard.

IEEE 802.11c

802.11c provides required information to ensure proper bridge operations. This project is completed and product developers utilize this standard when developing access points. There is really not much in this standard relevant to wireless LAN installers.

IEEE 802.11d

When 802.11 first became available, only a handful of regulatory domains (e.g. U.S., Europe and Japan) had rules in place for the operation of 802.11 WLANs. In order to support a widespread adoption, the 802.11d task group has to define PHY requirements that satisfy global harmonization. This is especially important for operation in the 5-GHz bands because the use of these frequencies differ widely from one country to another.

IEEE 802.11e

This standard works on QoS (quality of service) issue in LANs to optimize the transmission of voice and video. There's currently no effective mechanism to prioritize traffic within 802.11. As a result, the 802.11e task group is refining the 802.11MAC to improve QoS for better support of audio and video. The 802.11e group should finalize the standard by the end of 2002, with products probably available by mid-2003. Because 802.11e falls within the MAC layer, it will be common to all 802.11 PHYs and be backward compatible with existing 802.11 WLANs. As a result, the lack of 802.11e being in place today doesn't impact your decision on which PHY to use. Because 802.11e falls within the MAC layer, it will be common to all 802.11 PHYs and be backward compatible with existing 802.11 WLANs. As a result, the lack of 802.11e being in place today doesn't impact your decision on which PHY to use.

The standard, which is still under development, will use a method called Hybrid Coordination Function. A final standard is expected in the first half of (2003) and will support legacy devices via firmware and device driver updates.

IEEE 802.11f

Today, a user roaming between access points may lose some packets during the handoff between different vendors' devices. The existing 802.11 working group purposely didn't define this element in order to provide flexibility in working with different distribution systems. The problem, however, is that access points from different vendors may not interoperate when supporting roaming. 802.11f standard ensures multi-vendor access-point interoperability through the *Inter-Access Point Protocol*. The final standard, expected by year's end (may be, more likely in early 2003), will arrive as a flash update to legacy access points. The inclusion of 802.11f in access point design will eventually open up your and add some interoperability assurance when selecting access point vendors.

IEEE 802.11g

802.11g is an extension to 802.11b. It will broaden 802.11b's data rates to 54Mbps within the 2.4 GHz using OFDM (Orthogonal Frequency Division Multiplexing) technology. Because of the backward compatibility, an 802.11b radio card will interface directly with an 802.11g access point (and vice versa) at 11Mbps or lower depending on range. A big issue with 802.11g, which also applies to 802.11b, is considerable RF interference from other 2.4GHz devices, such as the cordless phones. It's possible to manage the problem by limiting sources of RF sources of RF interference; however, you can't always eliminate the problem. Initial approval in August 2002 - Final specification expected during the first half of 2003.

IEEE 802.11h

The 802.11a standard faces interference problems in Europe, where it shares the 5-GHz frequency band with radar and satellite communications. The 802.11h provides Dynamic Frequency Selection (DFS)and Transmit Power Control (TPC) to deal with this problem, this will make 802.11h the successor to 802.11a.
Dynamic Frequency Selection, or DFS, allows devices to detect such transmissions and switch to an alternative channel. The transmit power control protocol will allow users close to an access point to reduce transmission power in order to reduce interference with other users. A final standard, expected the end of 2003, will require client device driver and access-point firmware updates.

IEEE 802.11i -- Security (Expected to be ratified in September 2003)

802.11i is actively defining enhancements to the MAC Layer for Enhanced Security to counter the issues related to Wired Equivalent Privacy (WEP). The existing 802.1 standards specifies the use of relatively weak, static encryption keys without any form of key distribution management. This makes it possible for hackers to access and decipher WEP-encrypted data on your WLAN.

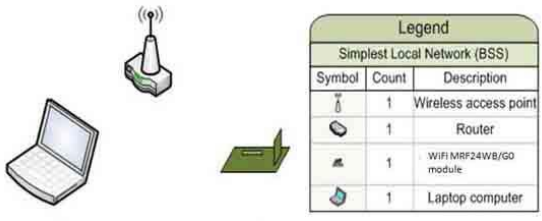
Temporal Key Integrity Protocol (TKIP) is an interim method. It will support legacy clients and access points through software updates, but cryptographers say TKIP will be broken eventually. The other scheme, based on the Advanced Encryption Standard will offer the best security but will likely require new hardware. While the specification won't be final until sometime next year, vendors are collaborating to produce an interoperable version of TKIP that should be available this year.

WI-FI NETWORK TOPOLOGIES

Infrastructure Basic Service Set (BSS)

In infrastructure Basic Service Set, a laptop computer and the WL-Wi-Fi communicating with each other through a wireless Access Point and Router. This network can gain access to the internet if the router is connected to a WAN.

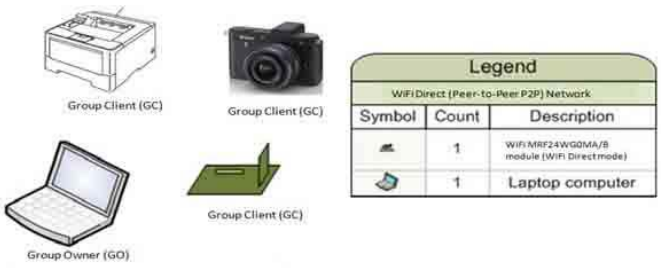
A common example of a local network operating in Infrastructure mode is shown in



Wi-Fi Direct (Peer-to-Peer (P2P)) Network

Wi-Fi Direct allows you to configure a secured wireless network between several devices, such as smart devices, laptops or computers with wireless network adapters, without using an AP. Wi-Fi Direct supports Wi-Fi Protected Setup (WPS) connection method, which is known as the WSC (Wi-Fi Simple Configuration) Config Methods in the Wi-Fi Peer-to-Peer Technical Specifications, in particular WPS Push Button method with WPA2. From the negotiation process, each device will determine which devices become group owner (GO) or group client (GC). The “GroupOwnerIntent” field in the P2P information element (IE) will indicate the level of desire to become the GO. The higher the value, the higher the desire to be the GO. Within each Wi-Fi Direct network, there can be only one GO, similar to only single AP in the infrastructure network. Wi-Fi Direct does not support 802.11b.

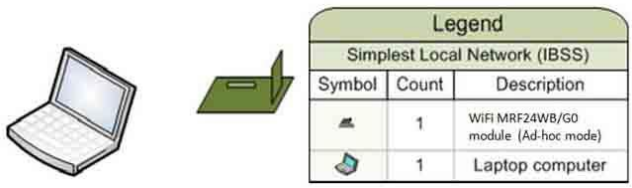
Figure show a typical example of local network, Wi-Fi Direct (peer-to-peer) network.



Ad hoc Network or Independent BSS

The Ad Hoc Network first station to broadcast when creating the network. In this case, join the laptop to the ad hoc network after the development board has gone through the steps of setting up the ad hoc network. The security mode supported is Open mode and WEP security. According to specifications, ad hoc network only supports 802.11b rates of 1 Mbps, 2 Mbps, 5.5 Mbps and 11 Mbps. Most Android devices do not support ad hoc network.

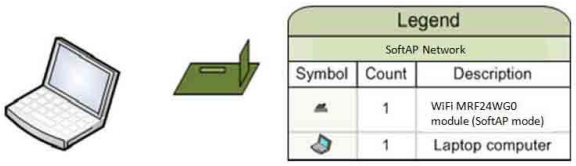
Figure show a typical example of local network Ad hoc Network or Independent BSS network.



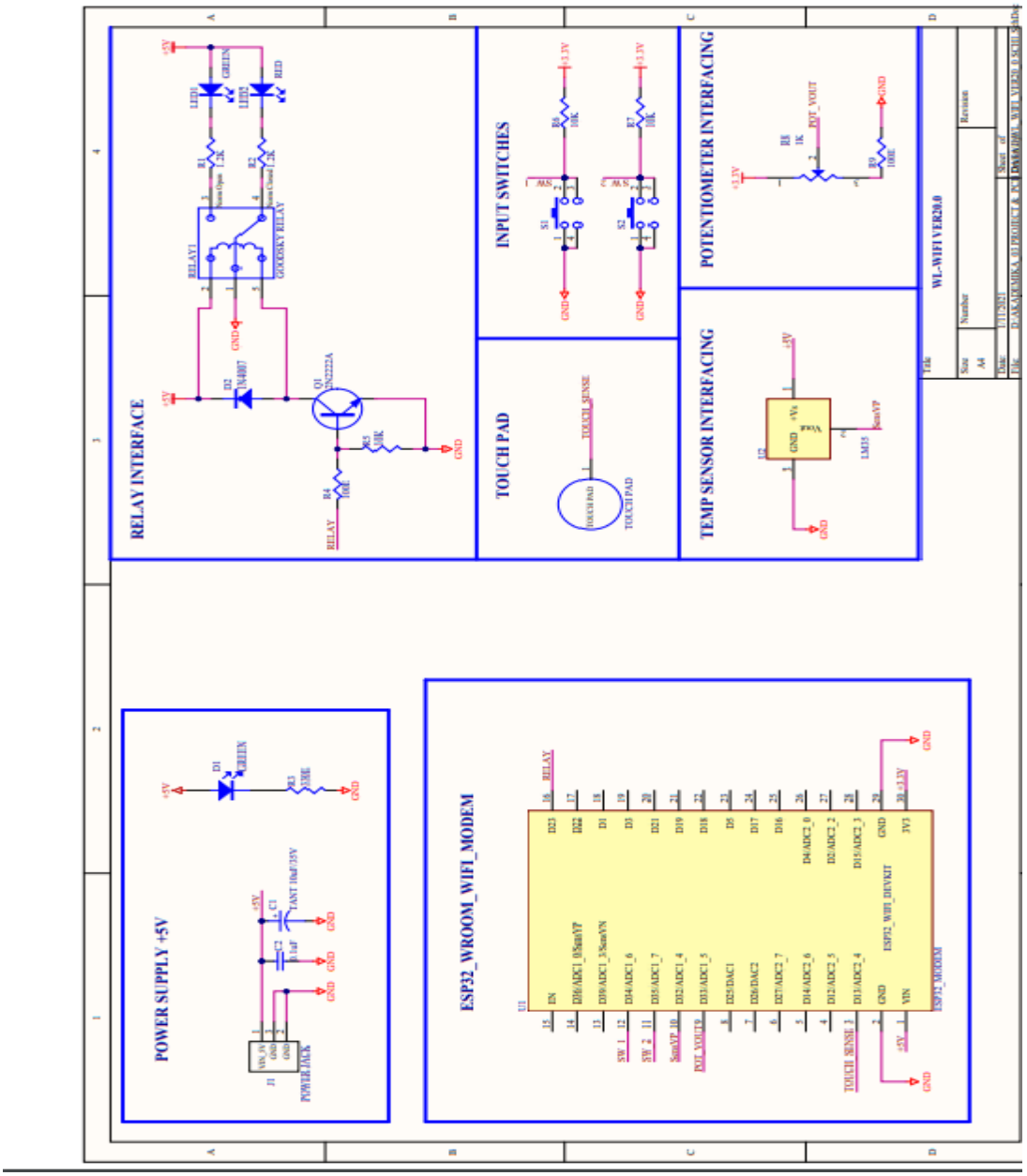
SoftAP Network

Software enabled AP (SoftAP) network. Current RF module firmware version only has programmed to support this network type. SoftAP functions can be used to extend wireless coverage and share internet.

A typical example of common local network.



CIRCUIT DIAGRAM



WL-WI-FI TRAINER INSTALLATION GUIDE LINE

1. Connect WIFI Module to the Evaluation Board

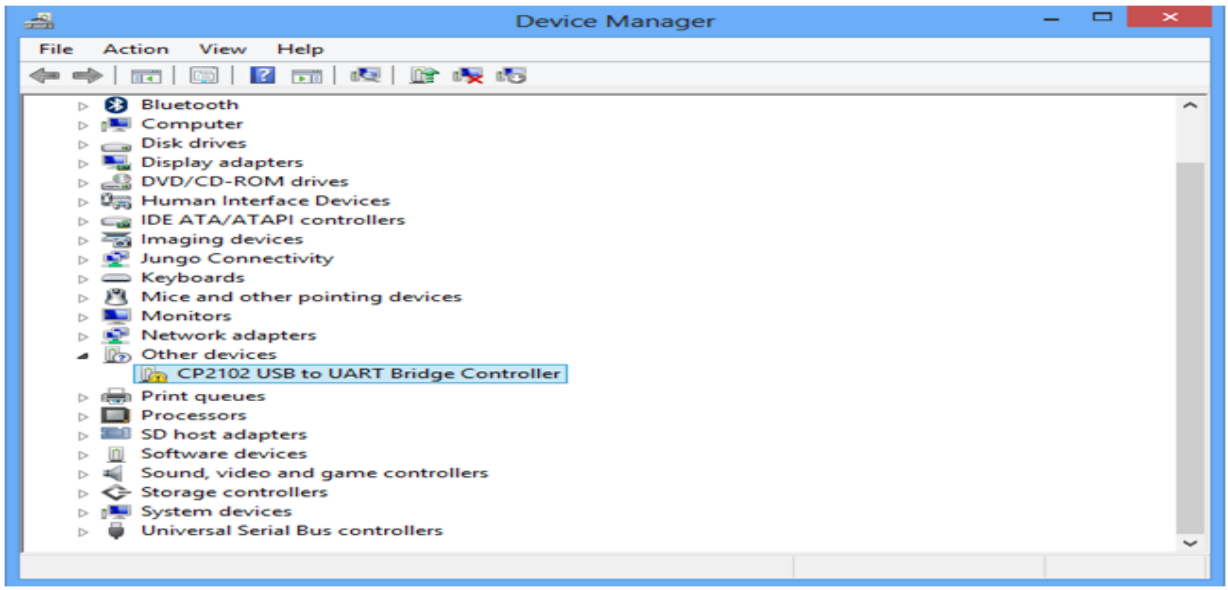
2. Check the polarity of the WIFI Module to avoid damage
3. Check the adaptor output voltage, it should be +5V DC (not recommended other voltage levels)
4. Connect the +5V Adaptor to the Trainer
5. Connect the USBA – micro cable to PC and WIFI Module on trainer board.
6. Turn ON the Power Supply (LED D1 will glow on)
7. Now PC is asking for Driver (driver installation steps are shown below)

IMPORTANT WARNING

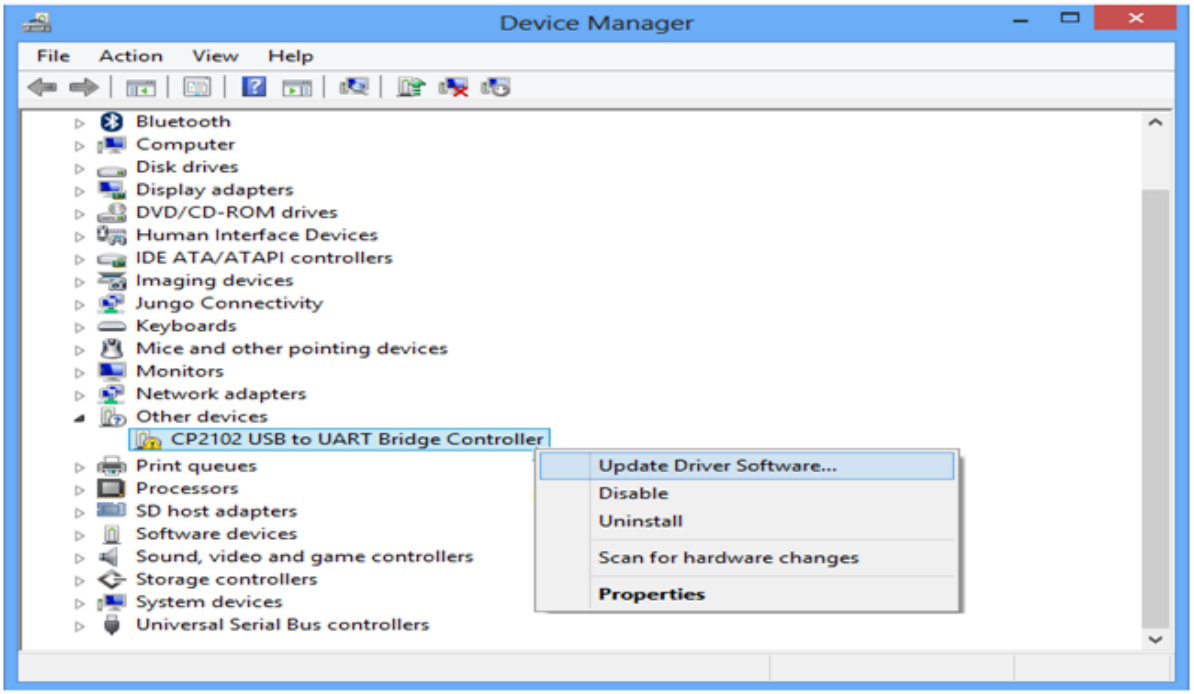
- ***ALWAYS connect the provided 5V power supply to J1 (5VDC IN) on the* Evaluation ……Board *for* Wi-Fi Module**
- ***Then connect USBA – micro cable to PC and WIFI Module on trainer board.***

INSTALL THE DRIVER

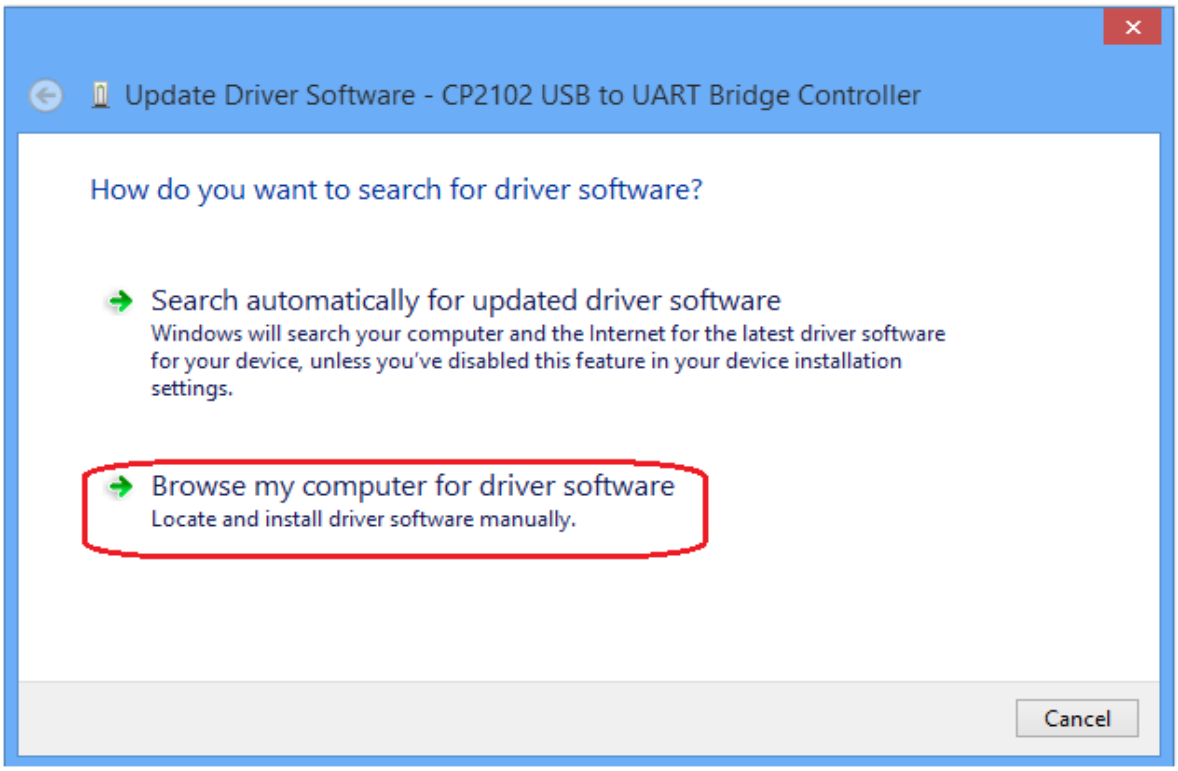
1. We need to install USB to UART Bridge Drivers as shown below.
2. Open the device manager, now the device will be listed in other devices as proper drivers are not installed.



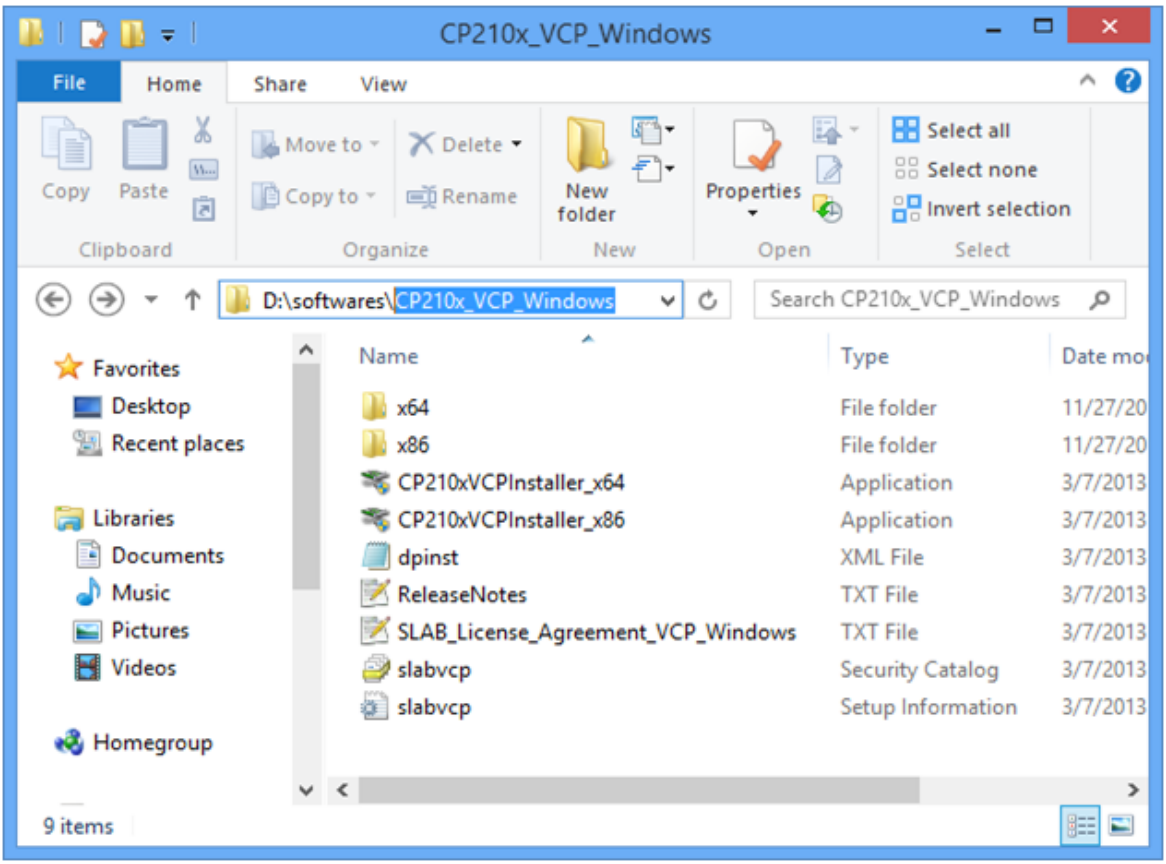
3. Now right click on the Cp2102 USB to UART Bridge Controller and select Update Driver Software option.



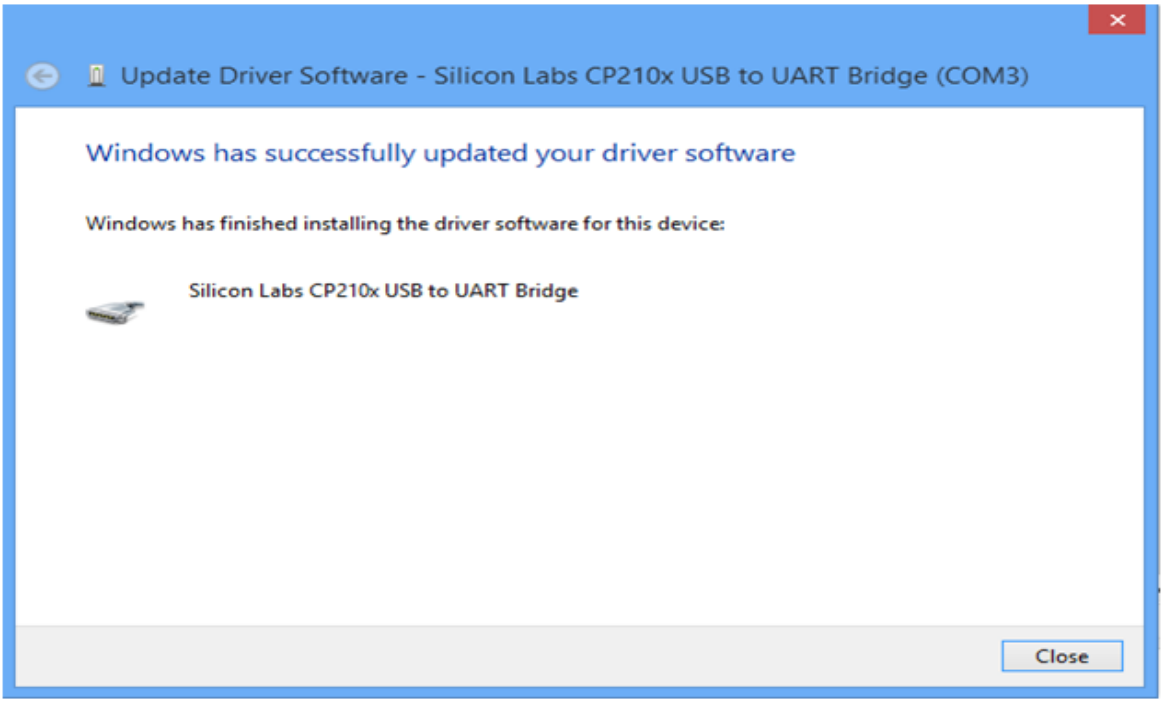
4. Browse and select the folder where the drivers are copied.



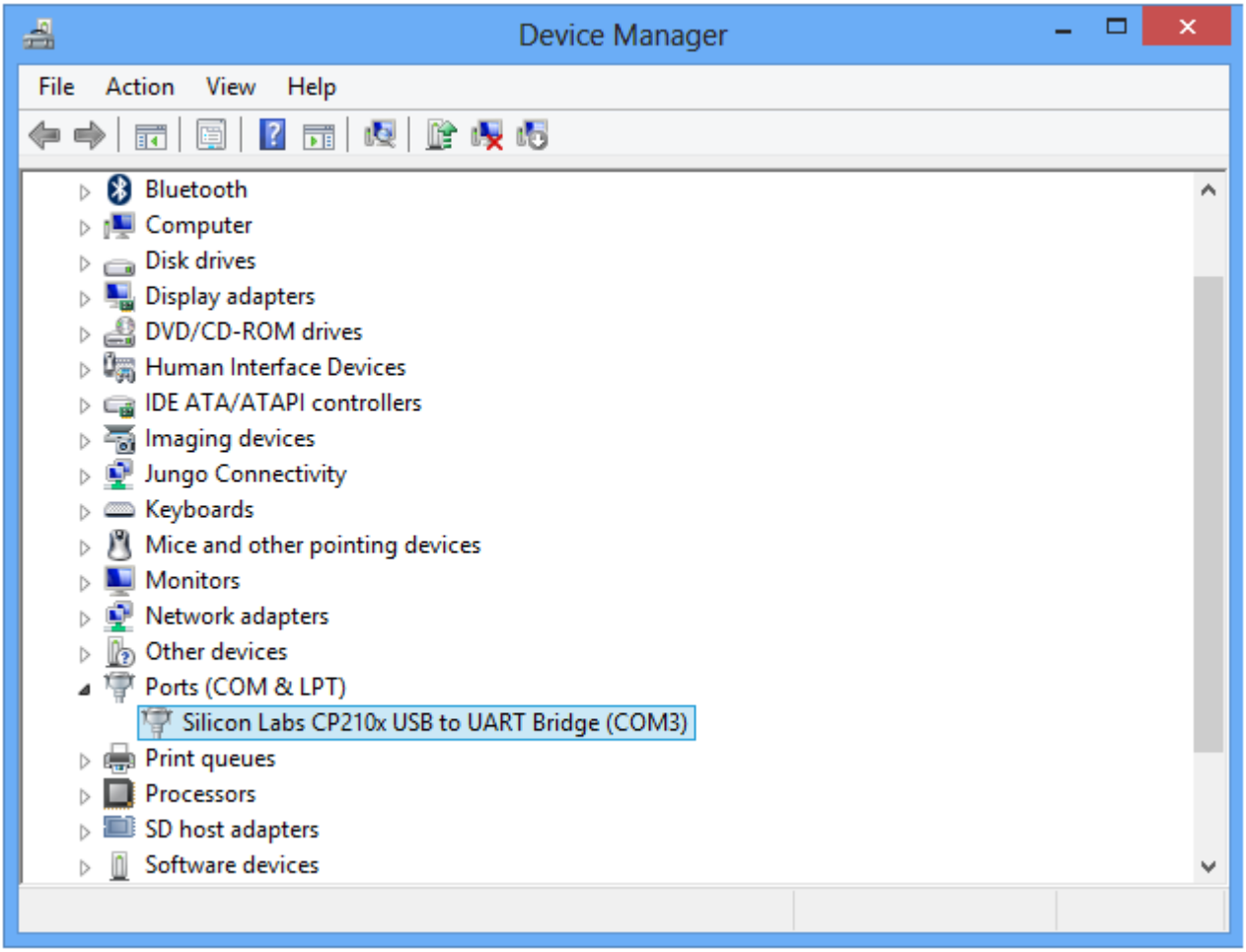
5. Select this folder for installing the drivers.



6. The below screen shot shows the successful installation of drivers.



7. Now the device will be listed under PORTS with unique serial port assigned.



Installing the ESP32 Board in Arduino IDE (Windows instructions)

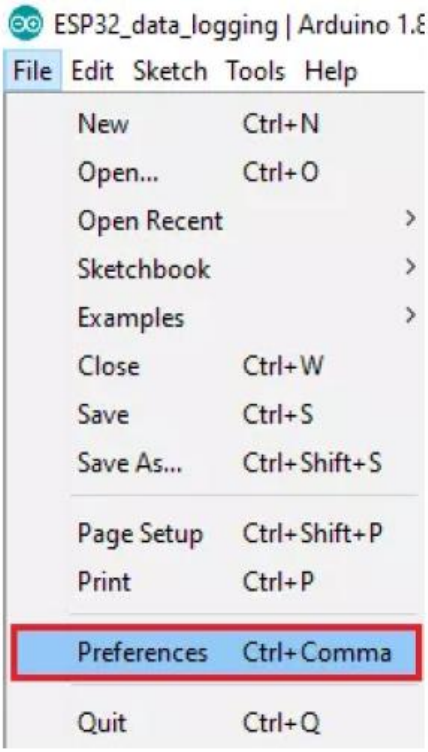
Important: before starting this installation procedure, make sure you have the latest Version of the Arduino IDE installed in your computer. If you don't, uninstall it and Install it again form provided Arduino setup file. Otherwise, it may not work.

There's an add-on for the Arduino IDE that allows you to program the ESP32 using the Arduino IDE and its programming language. In this manual we'll show you how to install the ESP32 board in the Arduino IDE for Windows.

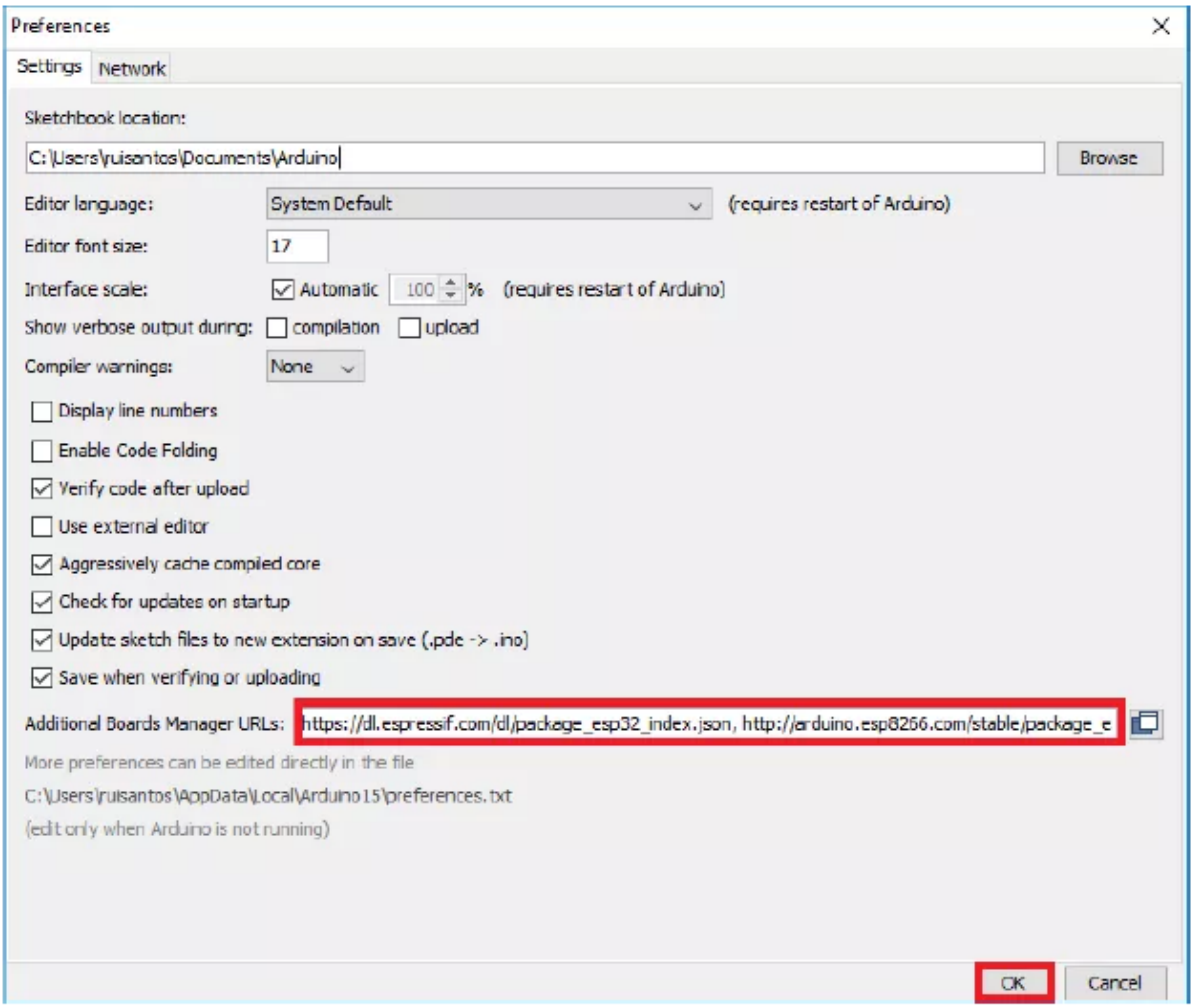
Installing the ESP32 Board

To install the ESP32 board in your Arduino IDE, follow these next instructions:

1. Open the preferences window from the Arduino IDE. Go to **File> Preferences**



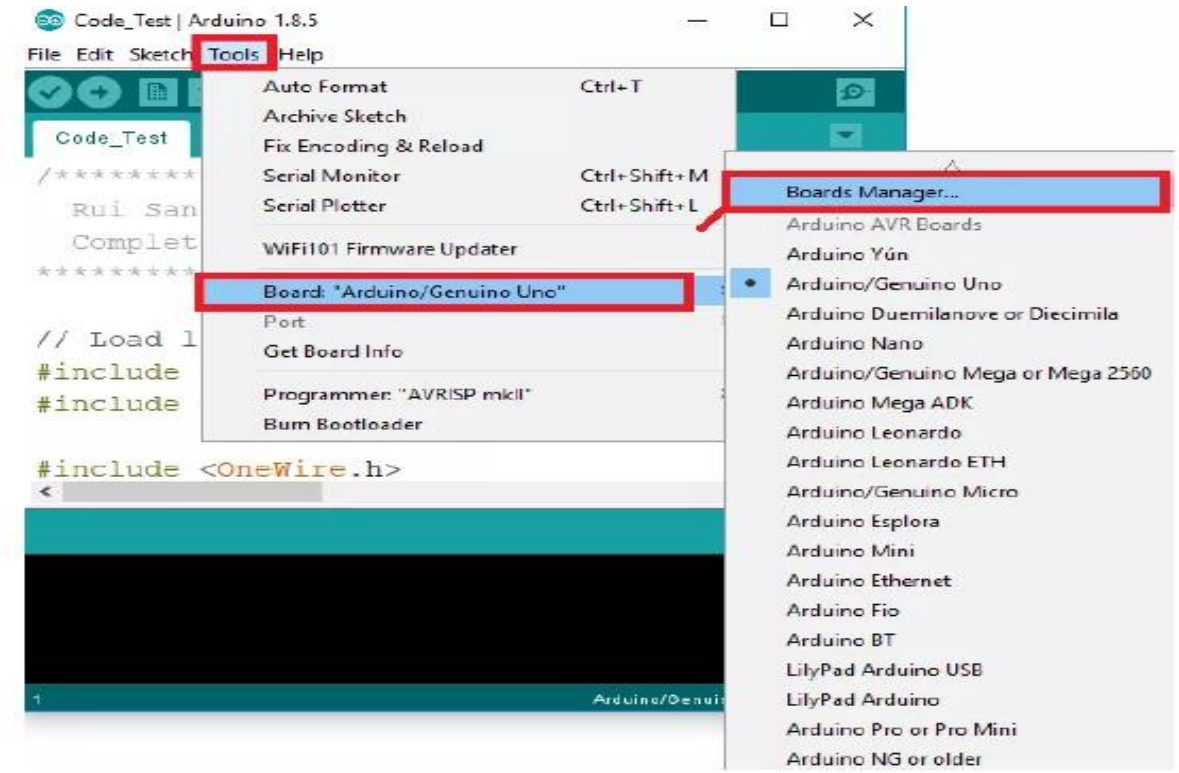
2. Enter **https://dl.espressif.com/dl/package_esp32_index.json** into the “Additional Board Manager URLs” field as shown in the figure below. Then, click the “OK” button:



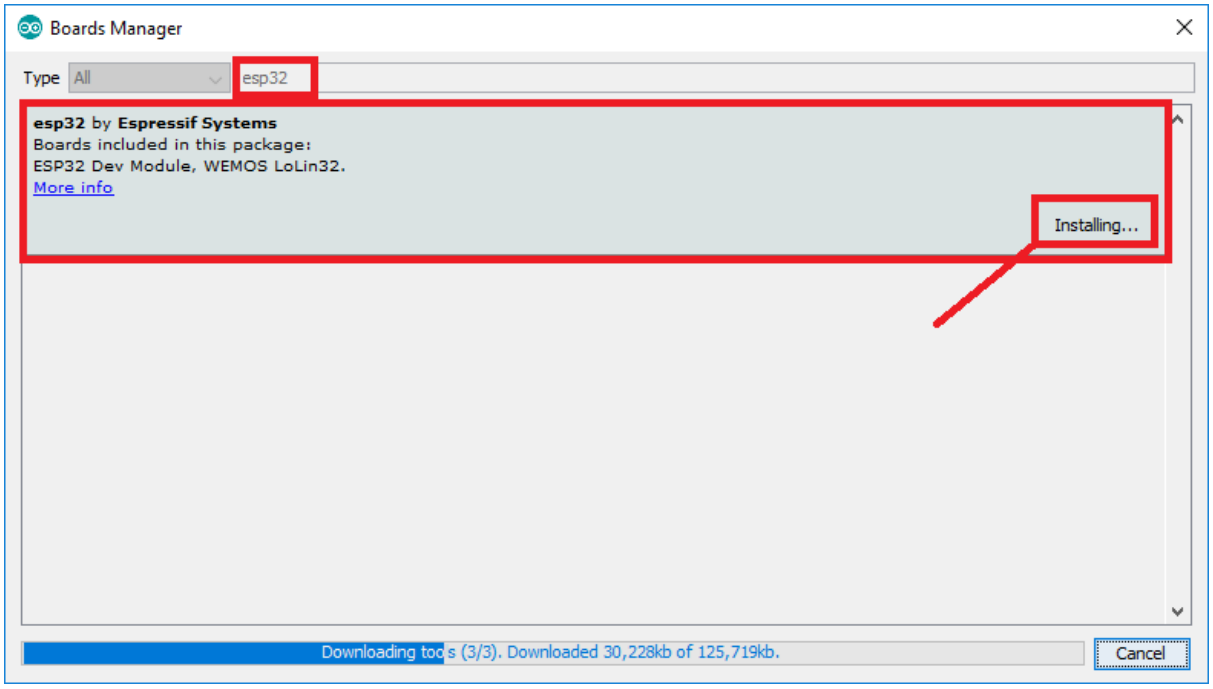
Note: if you already have the ESP8266 boards URL, you can separate the URLs with a comma as follows:

https://dl.espressif.com/dl/package_esp32_index.json,http://arduino.esp8266.com/stable/package_esp8266com_index.json.

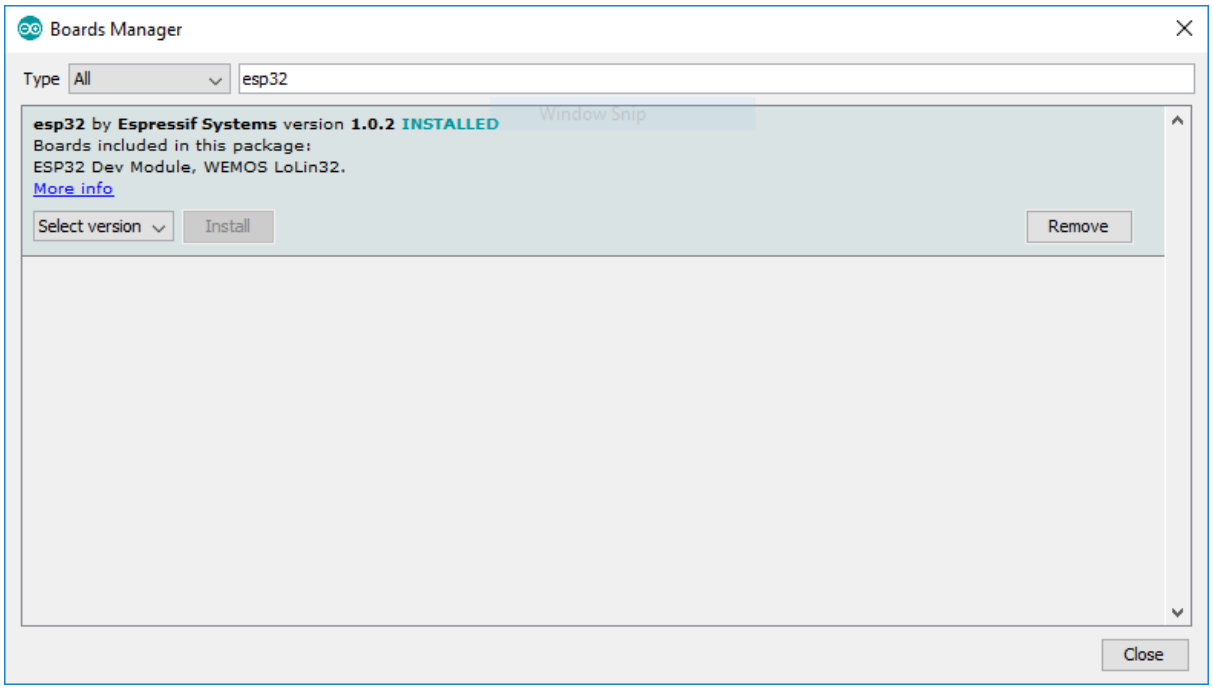
3. Open boards manager. Go to **Tools > Board > Boards Manager…**



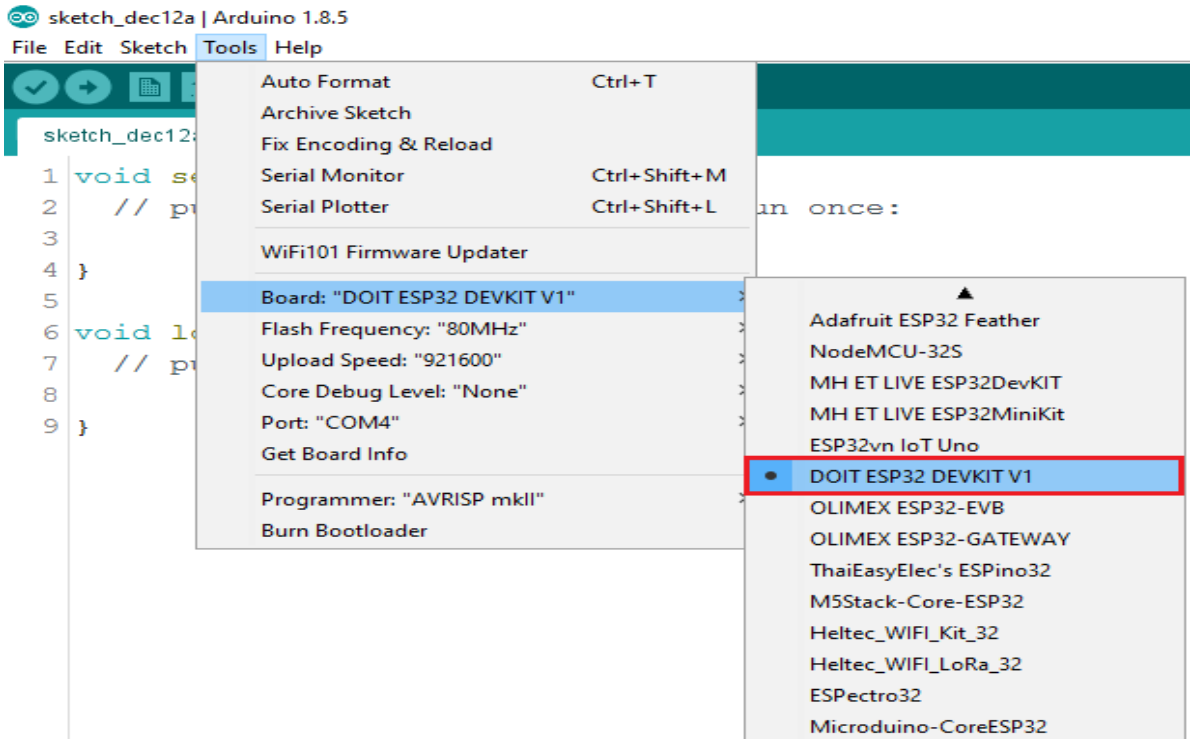
4. Search for ESP32 and press install button for the "**ESP32 by Espressif Systems** ":



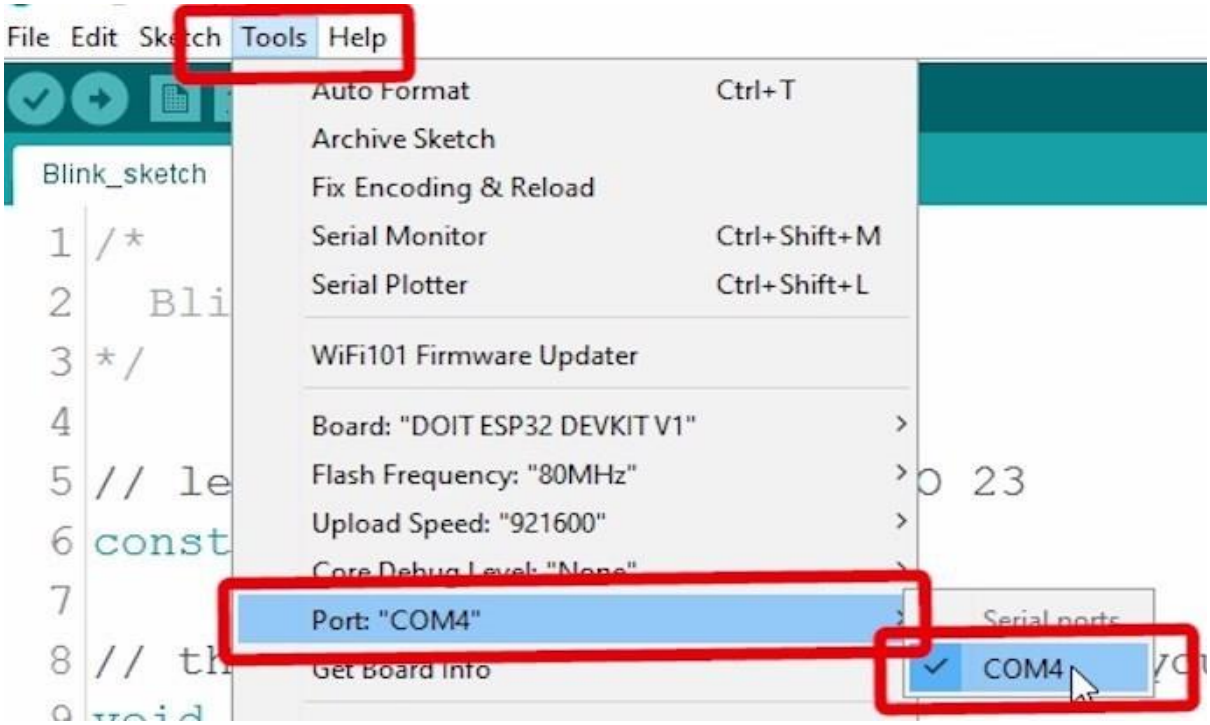
5. That's it. It should be installed after a few seconds.



6. Now, Select your Board in **Tools > Board** menu (it's the **DOIT ESP32 DEVKIT V1**)



7. Select the Port.



Now, Arduino **BOARD** selection & **PORT** selection procedure is done.

EXPERIMENT NO. 1

EXPERIMENT NO: 1

NAME:-

To test the Temperature Sensor on Wi-Fi Trainer board.

OBJECTIVE:-

To test on board peripheral temp sensor on WIFI board.

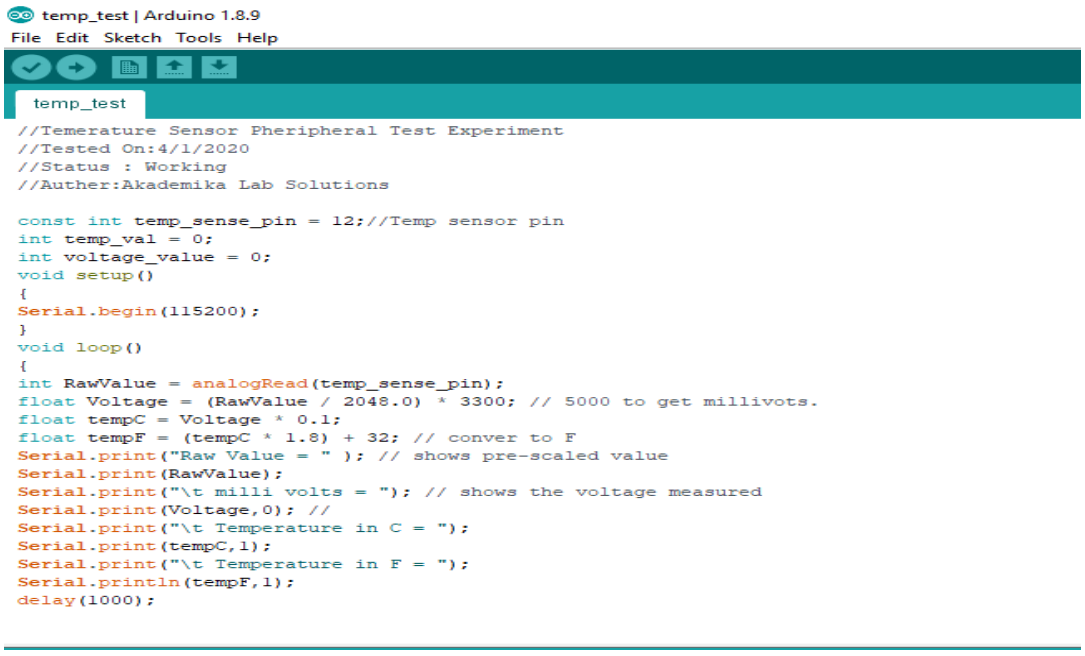
EQUIPMENTS:-

- WL-WIFI Trainer 01 no
- USB Cable 01 no
- Power supply 01 no
- PC or Laptop with installed Arduino software.

PROCEDURE:-

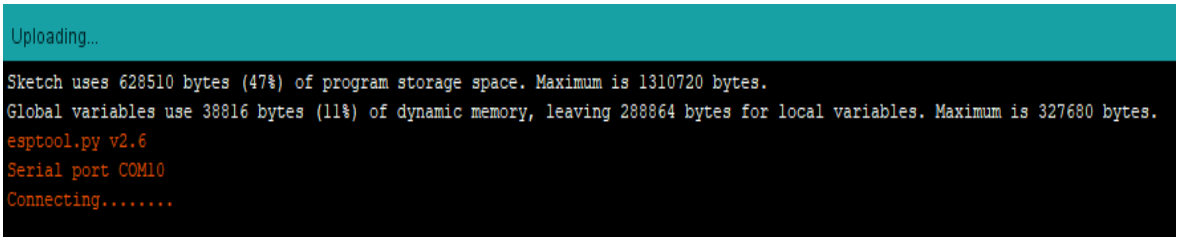
Below steps shows the method:

1. Connect USB cable between PC & WIFI trainer kit.
2. Connect 5V adaptor to the WIFI trainer kit.
3. Open the given Arduino program file temp_test in Arduino software from the given folder path 'Peripheral & Sensors related Experiment Programs\1_**Temp_Sens \temp_test**.
   Arduino program will be open, as shown below.

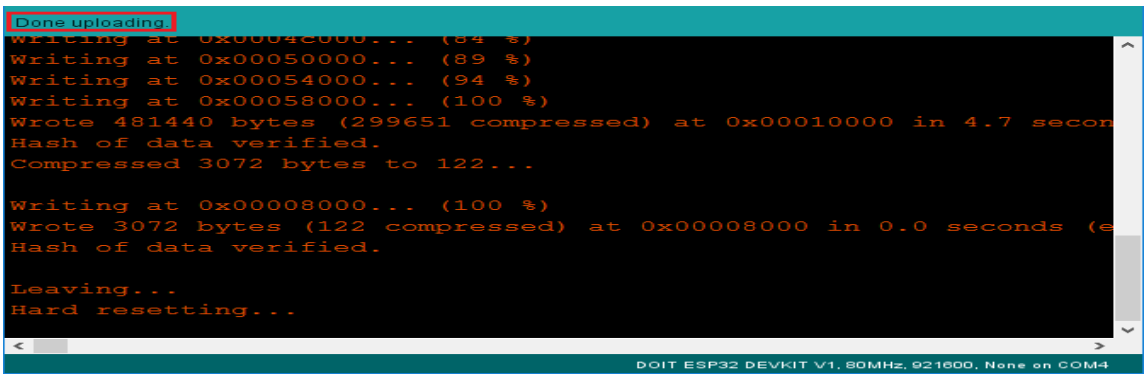



```
//Temerature Sensor Pheripheral Test Experiment
//Tested On:4/1/2020
//Status : Working
//Auther:Akademika Lab Solutions

const int temp_sense_pin = 12;//Temp sensor pin
int temp_val = 0;
int voltage_value = 0;
void setup()
{
Serial.begin(115200);
}
void loop()
{
int RawValue = analogRead(temp_sense_pin);
float Voltage = (RawValue / 2048.0) * 3300; // 5000 to get millivots.
float tempC = Voltage * 0.1;
float tempF = (tempC * 1.8) + 32; // conver to F
Serial.print("Raw Value = " ); // shows pre-scaled value
Serial.print(RawValue);
Serial.print("\t milli volts = "); // shows the voltage measured
Serial.print(Voltage,0); //
Serial.print("\t Temperature in C = ");
Serial.print(tempC,1);
Serial.print("\t Temperature in F = ");
Serial.println(tempF,1);
delay(1000);
```

4. Press the **Upload**  button in the Arduino IDE. After getting following message,



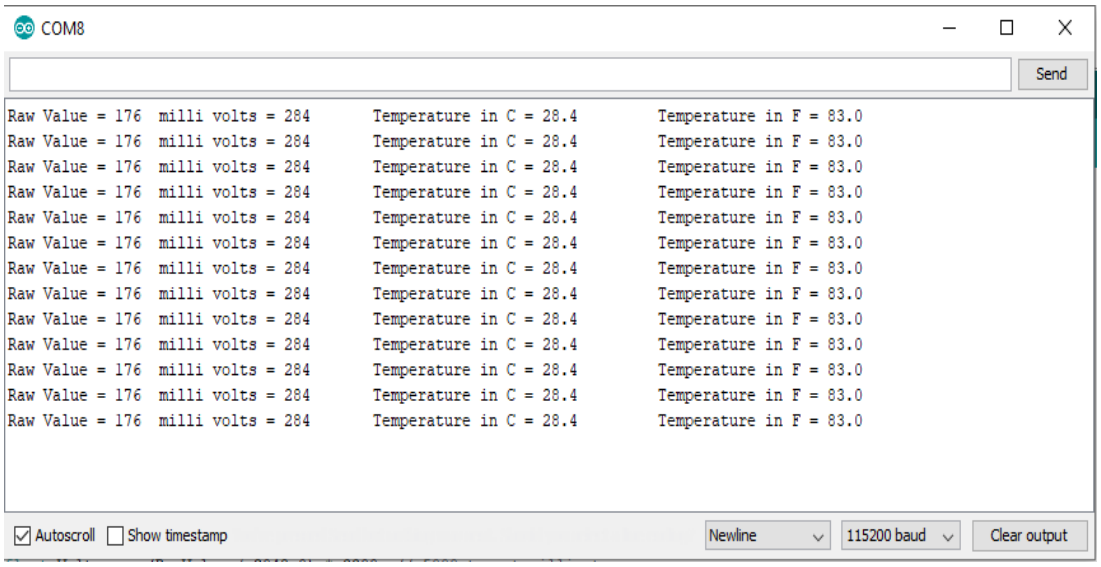
5. You need to Hold-down the "**BOOT**" button in your ESP32 board (Hardware), Wait a few seconds while the code compiles and uploads to your board.
6. If everything went as expected, you should see a "**Done uploading.**" message.



7. Open the Arduino **IDE Serial Monitor** at a baud rate of 115200.

OBSERVATIONS:-

8. Press the ESP32 on-board **Enable** button and Serial Monitor shows the current temperature in Celsius and Ferhenite unit as shown below.



EXPERIMENT NO. 2

EXPERIMENT NO: 2

NAME:-

To test the Potentiometer on Wi-Fi Trainer board.

OBJECTIVE:-

To test on board peripheral potentiometer on WIFI board.

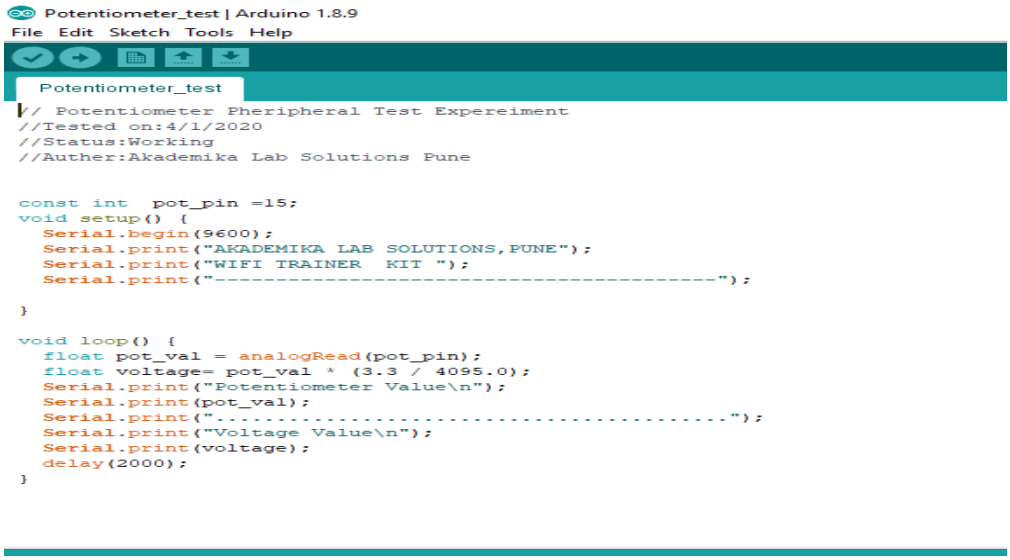
EQUIPMENTS:-

- WL-WIFI Trainer 01 no
- USB Cable 01 no
- Power supply 01 no
- PC or Laptop with installed Arduino software.

PROCEDURE:-

Below steps shows the method:

1. Connect USB cable between PC & WIFI trainer kit.
2. Coonect 5V adaptor to the WIFI trainer kit.
3. Open the given Arduino program file **Potentiometer_test** in Arduino software from the given folder path **'**Peripheral & Sensors related Experiment programs\2_**Pot_Read\Potentiometer_test**.'
   Arduino program will be open, as shown below.



4. Press the **Upload**  button in the Arduino IDE. After getting following message.

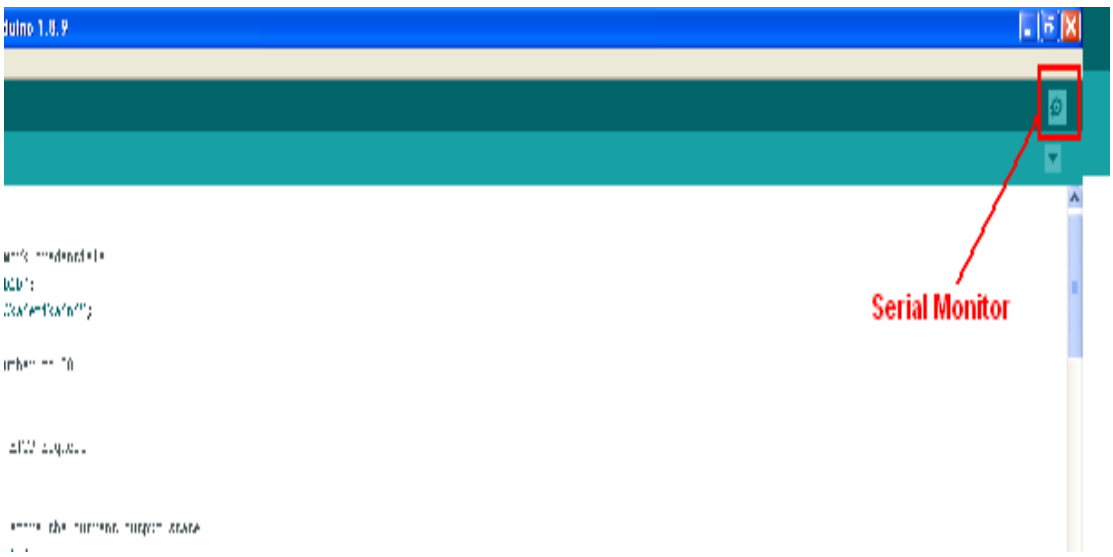
```
Uploading...

Sketch uses 628510 bytes (47%) of program storage space. Maximum is 1310720 bytes.
Global variables use 38816 bytes (11%) of dynamic memory, leaving 288864 bytes for local variables. Maximum is 327680 bytes.
esptool.py v2.6
Serial port COM10
Connecting.....
```

5. You need to Hold-down the “**BOOT**” button in your ESP32 board, Wait a few seconds while the code compiles and uploads to your board.
6. If everything went as expected, you should see a “**Done uploading.**” message.

```
Done uploading.
Writing at 0x00042000... (84 %)
Writing at 0x00050000... (89 %)
Writing at 0x00054000... (94 %)
Writing at 0x00058000... (100 %)
Wrote 481440 bytes (299651 compressed) at 0x00010000 in 4.7 secon
Hash of data verified.
Compressed 3072 bytes to 122...

Writing at 0x00008000... (100 %)
Wrote 3072 bytes (122 compressed) at 0x00008000 in 0.0 seconds (e
Hash of data verified.

Leaving...
Hard resetting...
```

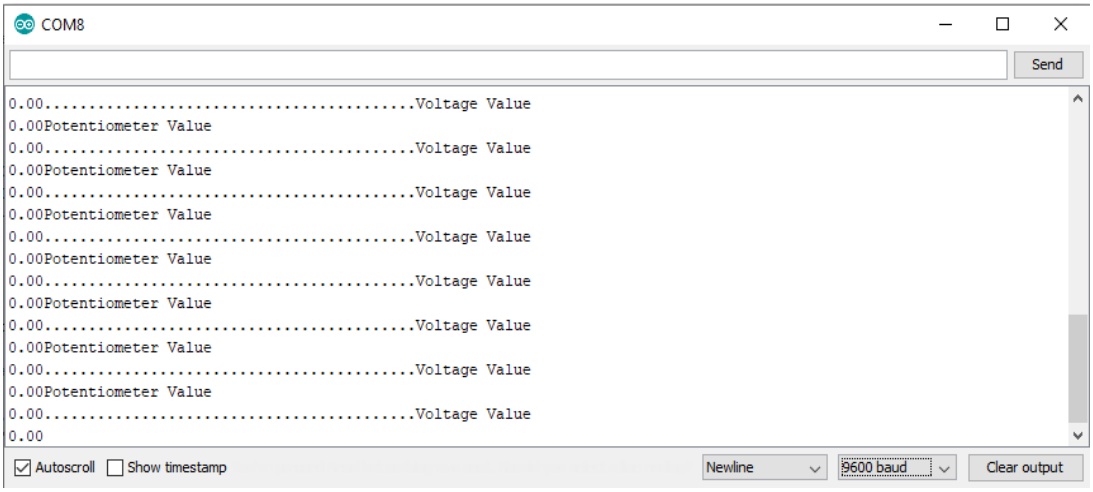
DOIT ESP32 DEVKIT V1, 80MHz, 921600, None on COM4

7. Open the Arduino **IDE Serial Monitor** at a baud rate of 9600.



OBSERVATIONS:-

8. Press the ESP32 on-board **Enable** button and Serial Monitor shows the voltage across the potentiometer as shown below.



EXPERIMENT NO. 3

EXPERIMENT NO: 3

NAME:-

To test the Touch pad sense on Wi-Fi Trainer board.

OBJECTIVE:-

To test on board peripheral touch pad on WIFI board.

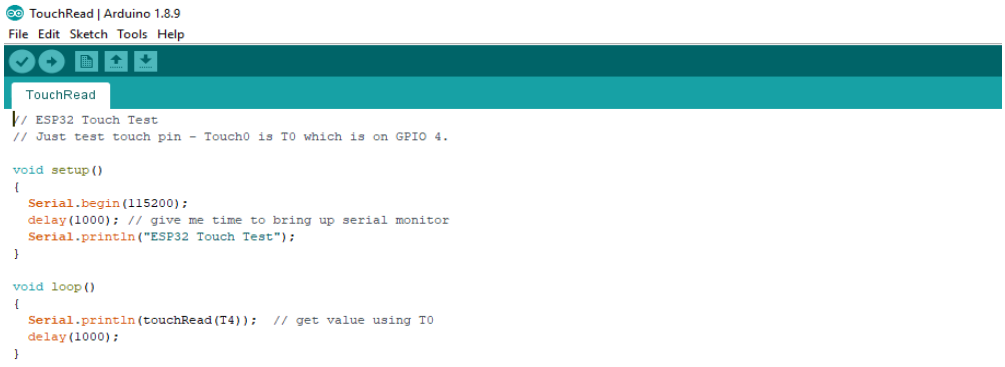
EQUIPMENTS:-

- WL-WIFI Trainer 01 no
- USB Cable 01 no
- Power supply 01 no
- PC or Laptop with installed Arduino software.

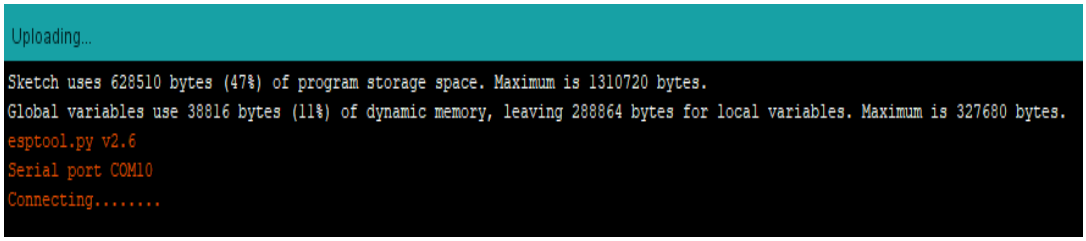
PROCEDURE:-

Below steps shows the method

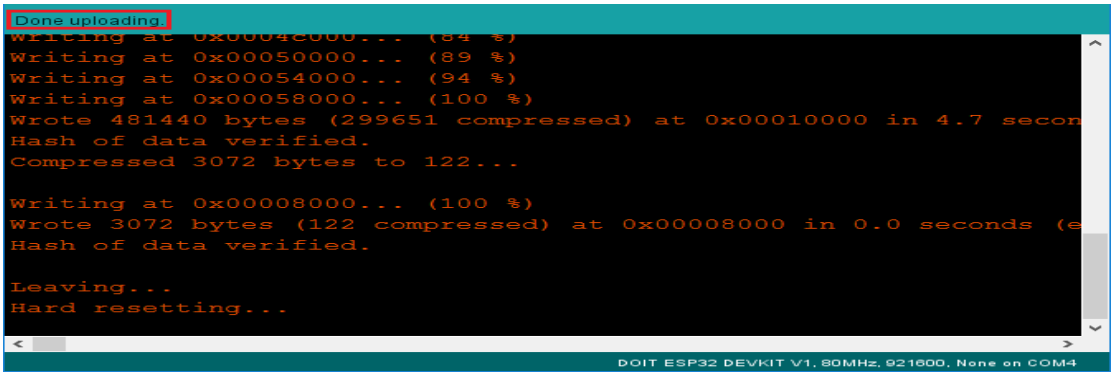
1. Connect USB cable between PC & WIFI trainer Kit.
2. Connect 5V adaptor to the WIFI trainer kit.
3. Open the given Arduino program file **Touch Read** in Arduino software from the given folder path **'**Peripheral & Sensors related Experiment Programs\3_**Touch_Pad_Read\Touch Read**.'
   Arduino program will be open, as shown below.



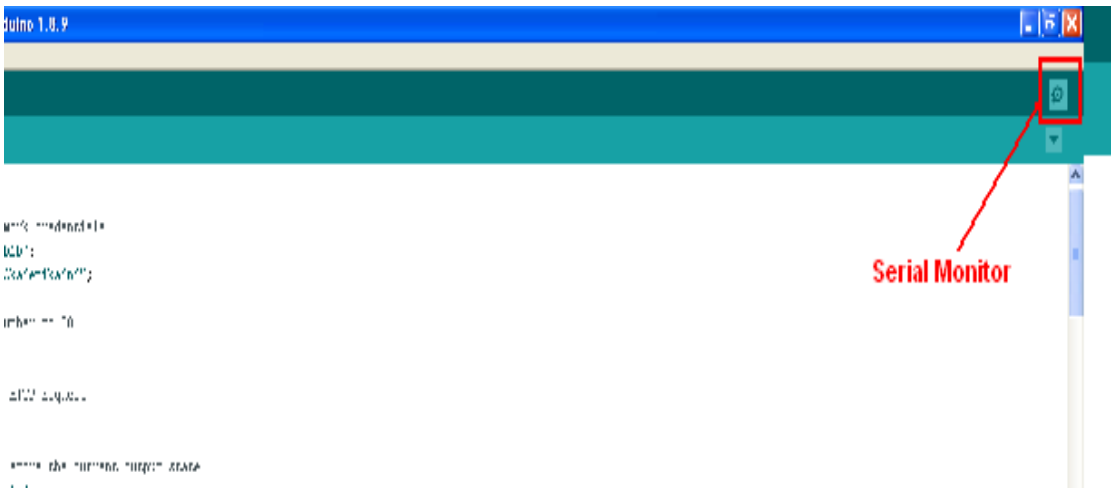
4. Press the **Upload**  button in the Arduino IDE. After getting following message,



5. You need to Hold-down the "**BOOT**" button in your ESP32 board, Wait a few seconds while the code compiles and uploads to your board.
6. If everything went as expected, you should see a "**Done uploading.**" message.

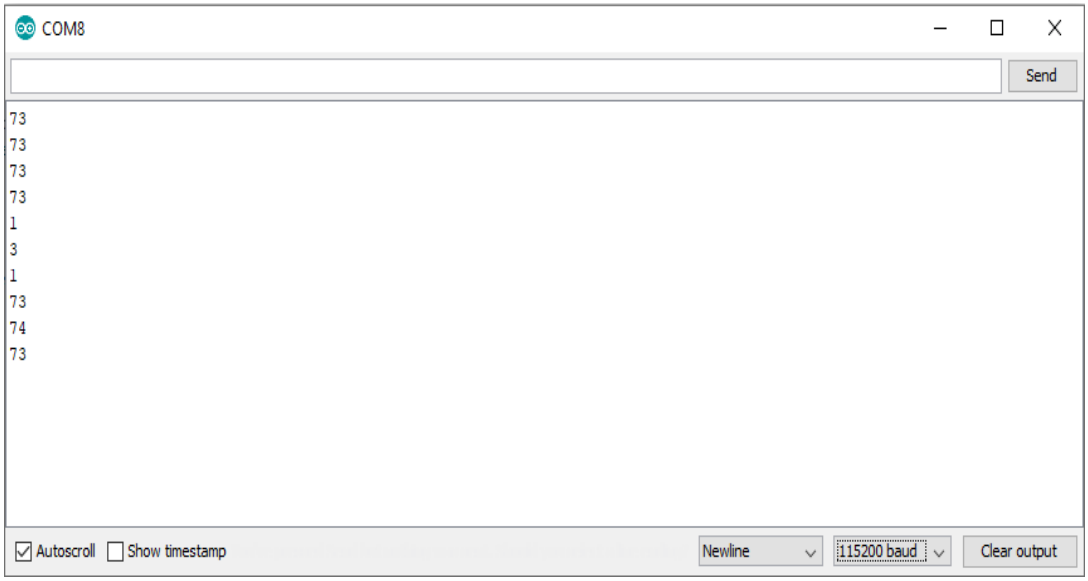


7. Open the Arduino **IDE Serial Monitor** at a baud rate of 115200.



OBSERVATIONS:-

8. Press the ESP32 on-board **Enable** button. Now touch your finger on touch pad and check Serial Monitor, It will show the touch pad is tuched or not as shown below.



EXPERIMENT NO. 4

EXPERIMENT NO: 4

NAME:-

To test the GLOWING LED on Wi-Fi Trainer board.

OBJECTIVE:-

To test on board peripheral glowing led on WIFI board.

EQUIPMENTS:-

- WL-WIFI Trainer 01 no
- USB Cable 01 no
- Power supply 01 no
- PC or Laptop with installed Arduino software.

PROCEDURE:-

Below steps shows the method:

1. Connect USB cable between PC & WIFI trainer.
2. Connect 5V adaptor to the WIFI trainer.
3. Open the given Arduino program file **LED** in Arduino software from the given folder path 'Peripheral & Sensors related Experiment Programs\4_**LED_Glow\ee\LED**.' Arduino program will be open, as shown below.


```
// the setup function runs once when you press reset or power the board
void setup() {
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
}

// the loop function runs over and over again forever
void loop() {
  digitalWrite(LED_BUILTIN, HIGH);   // turn the LED on (HIGH is the voltage level)
  delay(1000);                       // wait for a second
  digitalWrite(LED_BUILTIN, LOW);    // turn the LED off by making the voltage LOW
  delay(1000);                       // wait for a second
}
```

4. Press the **Upload** button in the Arduino IDE. After getting following message.

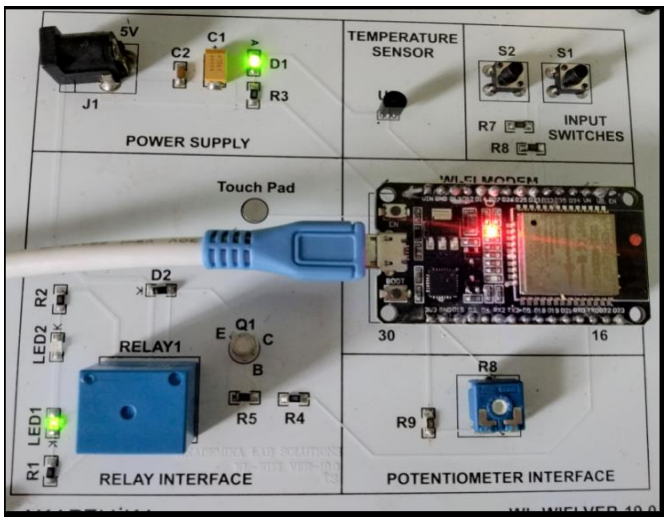
```
Uploading...

Sketch uses 628510 bytes (47%) of program storage space. Maximum is 1310720 bytes.
Global variables use 38816 bytes (11%) of dynamic memory, leaving 288864 bytes for local variables. Maximum is 327680 bytes.
esptool.py v2.6
Serial port COM10
Connecting.....
```

5. You need to Hold-down the "**BOOT**" button in your ESP32 board, Wait a few seconds while the code compiles and uploads to your board.
6. If everything went as expected, you should see a "**Done uploading.**" message.

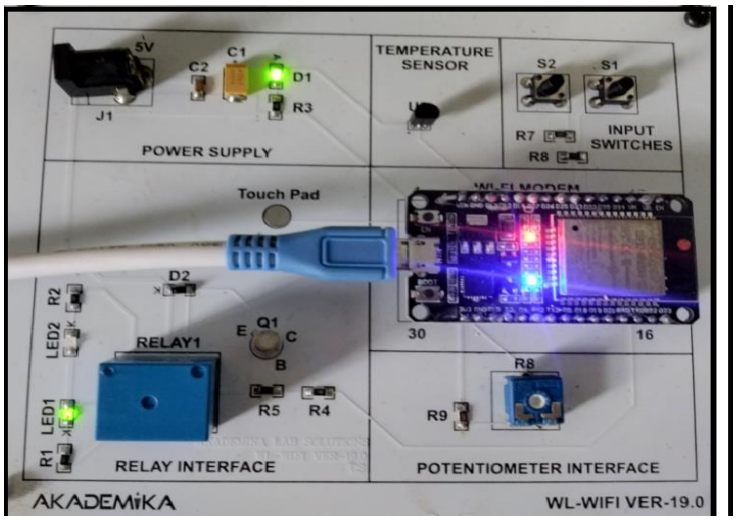
```
Done uploading.
Writing at 0x00048000... (84 %)
Writing at 0x00050000... (89 %)
Writing at 0x00054000... (94 %)
Writing at 0x00058000... (100 %)
Wrote 481440 bytes (299651 compressed) at 0x00010000 in 4.7 secon
Hash of data verified.
Compressed 3072 bytes to 122...

Writing at 0x00008000... (100 %)
Wrote 3072 bytes (122 compressed) at 0x00008000 in 0.0 seconds (e
Hash of data verified.

Leaving...
Hard resetting...

DOIT ESP32 DEVKIT V1, 80MHz, 921600, None on COM4
```

OBSERVATIONS:-

7. Press the ESP32 on-board **Enable** button and on-board LED will start to glow on and off as shown below.



Off state of LED



On state of LED

EXPERIMENT NO. 5

EXPERIMENT NO: 5

NAME:-

To test the SWITCHING RELAY on Wi-Fi Trainer board.

OBJECTIVE:-

To test on board peripheral switching relay on WIFI board.

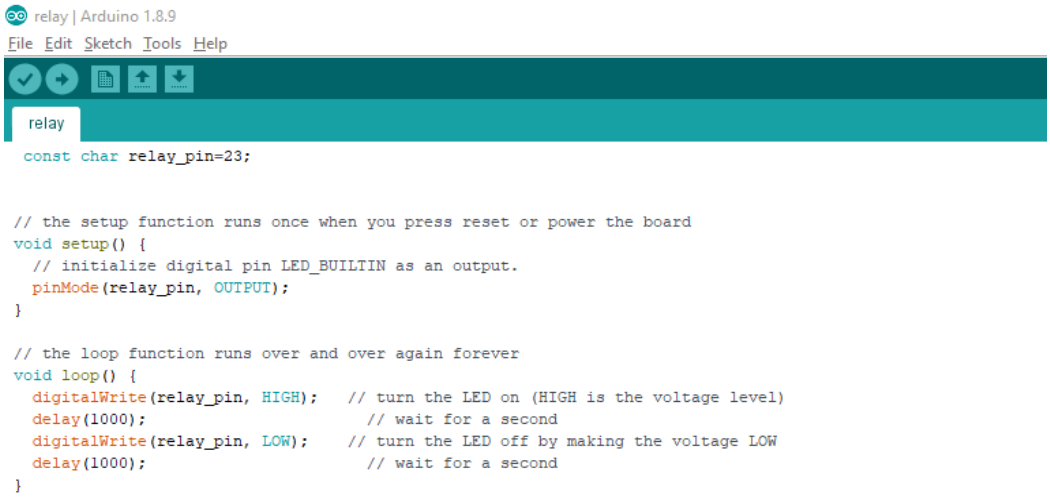
EQUIPMENTS:-

- WL-WIFI Trainer 01 no
- USB Cable 01 no
- Power supply 01 no
- PC or Laptop with installed Arduino software.

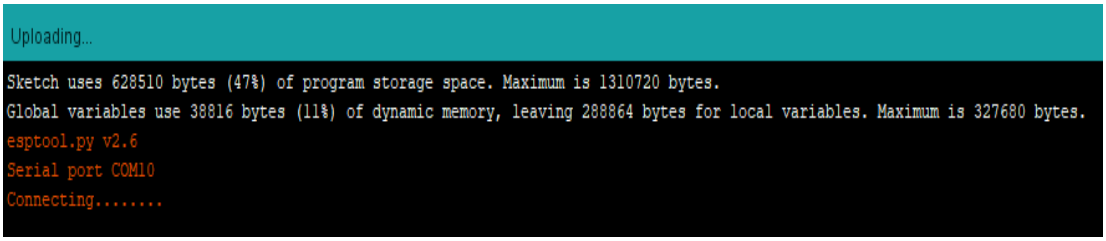
PROCEDURE:

Below steps shows the method:

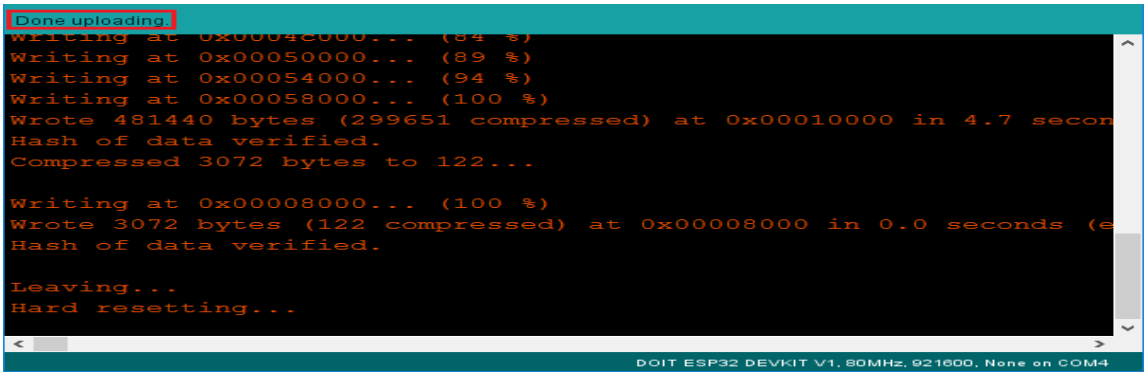
1. Connect USB Cable between PC & WIFI Trainer.
2. Connect 5V adaptor to the WIFI trainer.
3. Open the given Arduino program file relay in Arduino software from the given folder path 'Peripheral & Sensors related Experiment Programs\6_**Relay_Operation\relay.**' Arduino program will be open, as shown below.



4. Press the **Upload** button in the Arduino IDE. After getting following message.

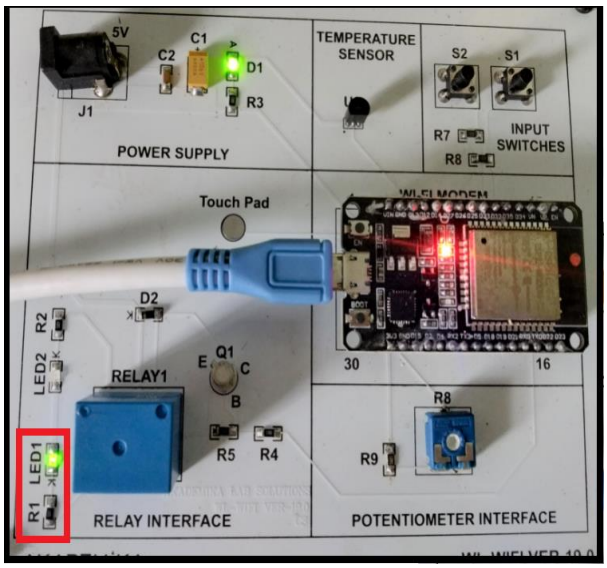


5. You need to Hold-down the "**BOOT**" button in your ESP32 board, Wait a few seconds while the code compiles and uploads to your board.
6. If everything went as expected, you should see a "**Done uploading.**" message.

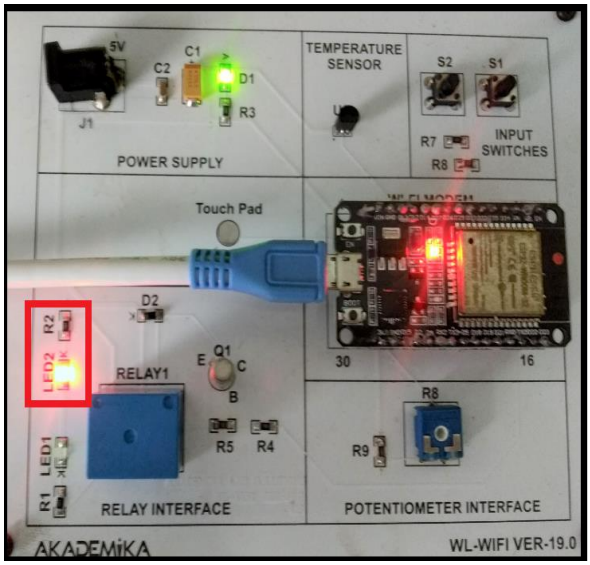


OBSERVATIONS:-

7. Press the ESP32 on-board **Enable** button and relay on board will start to switch between on and off states as shown below.



Relay off state



Relay on state

EXPERIMENT NO. 6

EXPERIMENT NO: 6

NAME:-

To test the USER SWITCHES on Wi-Fi Trainer board.

OBJECTIVE:-

To test on board peripheral user switches on WIFI board.

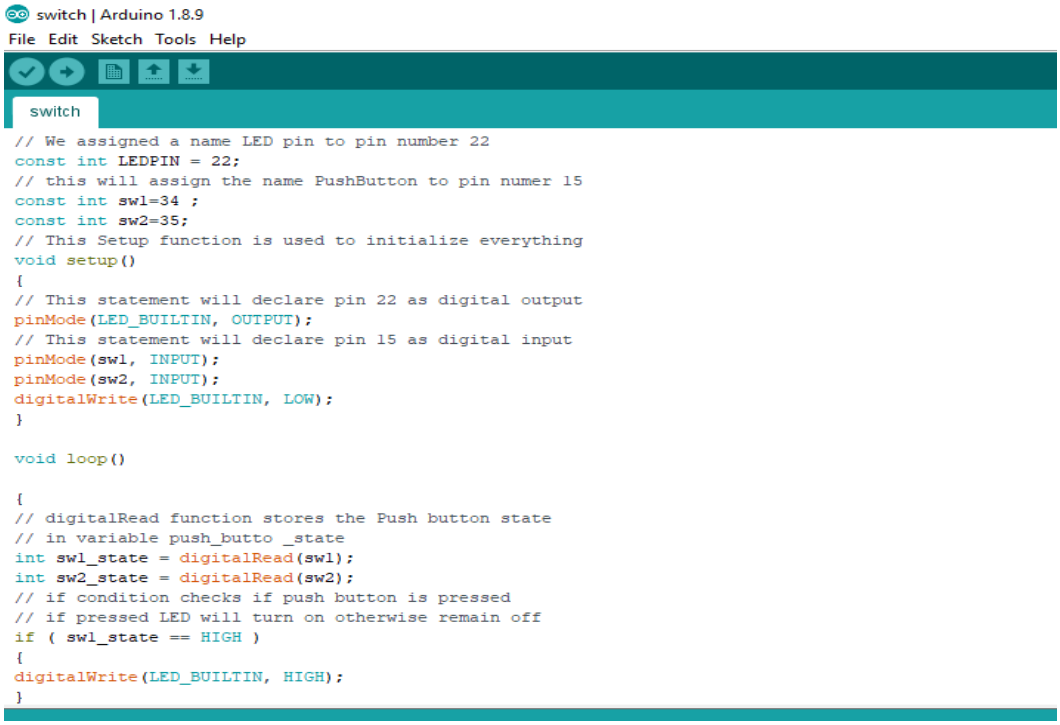
EQUIPMENTS:-

- WL-WIFI Trainer 01 no
- USB Cable 01 no
- Power supply 01 no
- PC or Laptop with installed Arduino software.

PROCEDURE:-

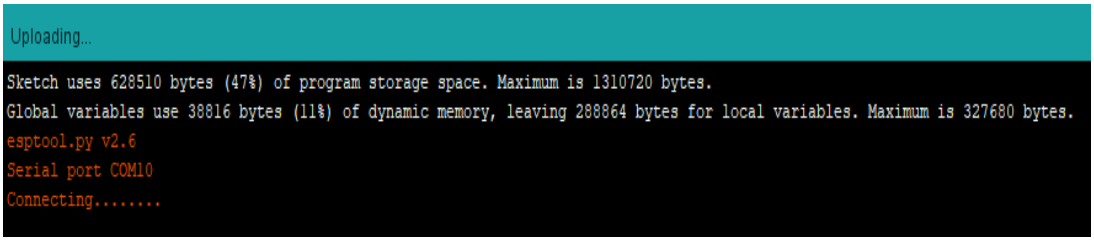
Below steps shows the method:

1. Connect USB cable between PC & WIFI triner kit.
2. Connect 5V adaptor to the WIFI trainer.
3. Open the given Arduino program file switch in Arduino software from the given folder path 'Peripheral & Sensors related Experiment Programs\5_**User_Switch_Read\switch.**' Arduino program will be open, as shown below.

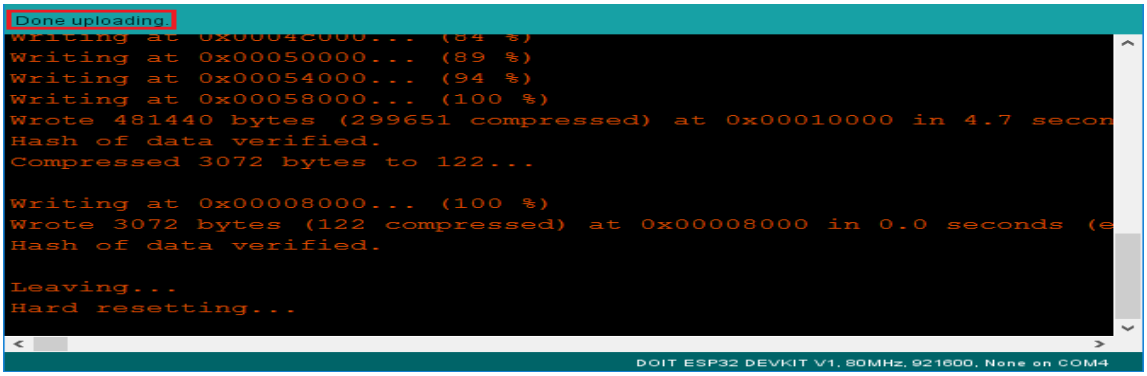



```
// We assigned a name LED pin to pin number 22
const int LEDPIN = 22;
// this will assign the name PushButton to pin numer 15
const int sw1=34 ;
const int sw2=35;
// This Setup function is used to initialize everything
void setup()
{
// This statement will declare pin 22 as digital output
pinMode(LED_BUILTIN, OUTPUT);
// This statement will declare pin 15 as digital input
pinMode(sw1, INPUT);
pinMode(sw2, INPUT);
digitalWrite(LED_BUILTIN, LOW);
}

void loop()

{
// digitalRead function stores the Push button state
// in variable push_butto _state
int sw1_state = digitalRead(sw1);
int sw2_state = digitalRead(sw2);
// if condition checks if push button is pressed
// if pressed LED will turn on otherwise remain off
if ( sw1_state == HIGH )
{
digitalWrite(LED_BUILTIN, HIGH);
}
```

4. Press the **Upload** button in the Arduino IDE. After getting following message.

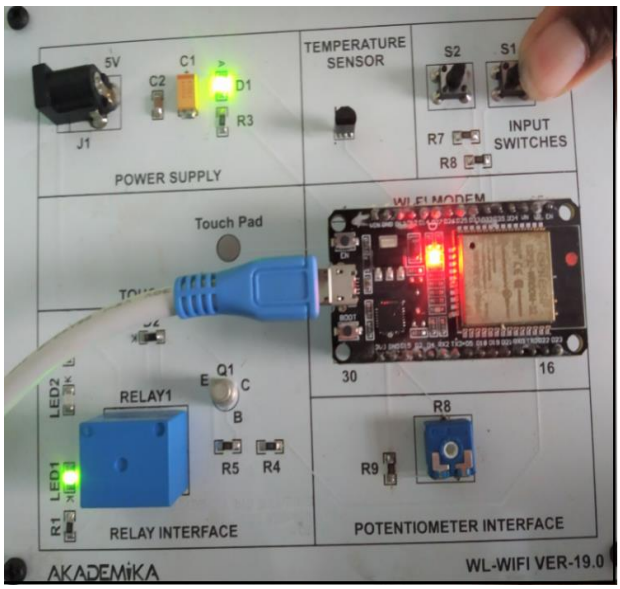


5. You need to Hold-down the "**BOOT**" button in your ESP32 board, Wait a few seconds while the code compiles and uploads to your board.
6. If everything went as expected, you should see a "**Done uploading.**" message.

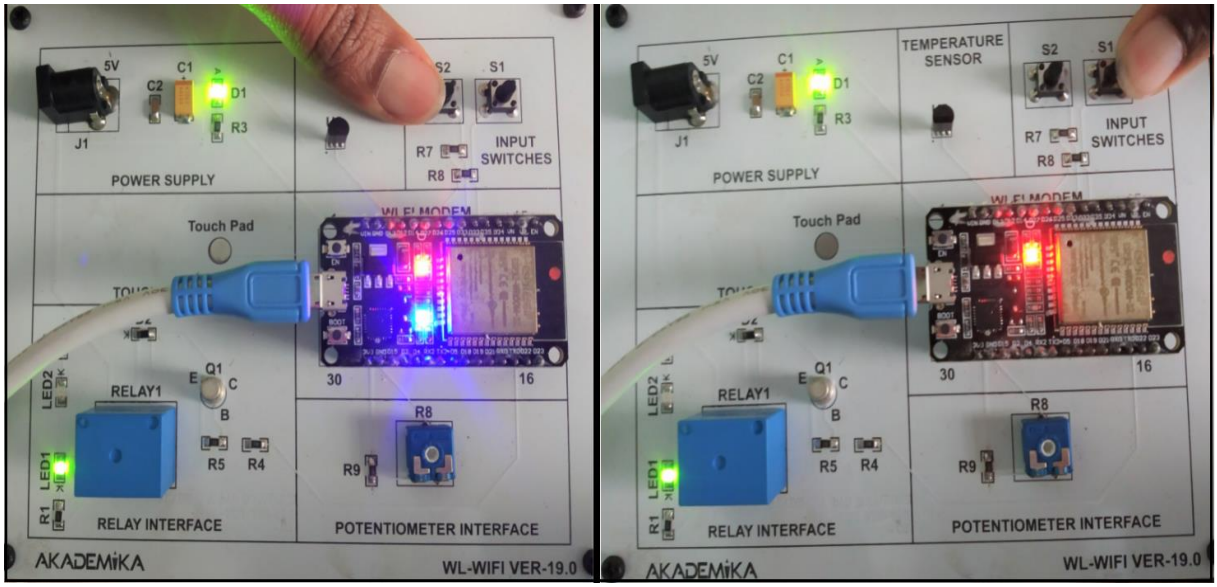


OBSERVATIONS:-

7. Press the ESP32 on-board **Enable** button and test user switches. Press S2 to glow on board led bright and press S1 to turn off the on board led as shown below.



If S1 is pressed on board LED will turn OFF



If S2 is pressed, on board LED brightness will increases

EXPERIMENT NO. 7

EXPERIMENT NO: 7

NAME:-

Scanning the available SSID's in the range of WI-FI Trainer.

OBJECTIVE:-

The goal of the experiment is to understand how the Wi-Fi device will discover nearby SSID to communicate with.

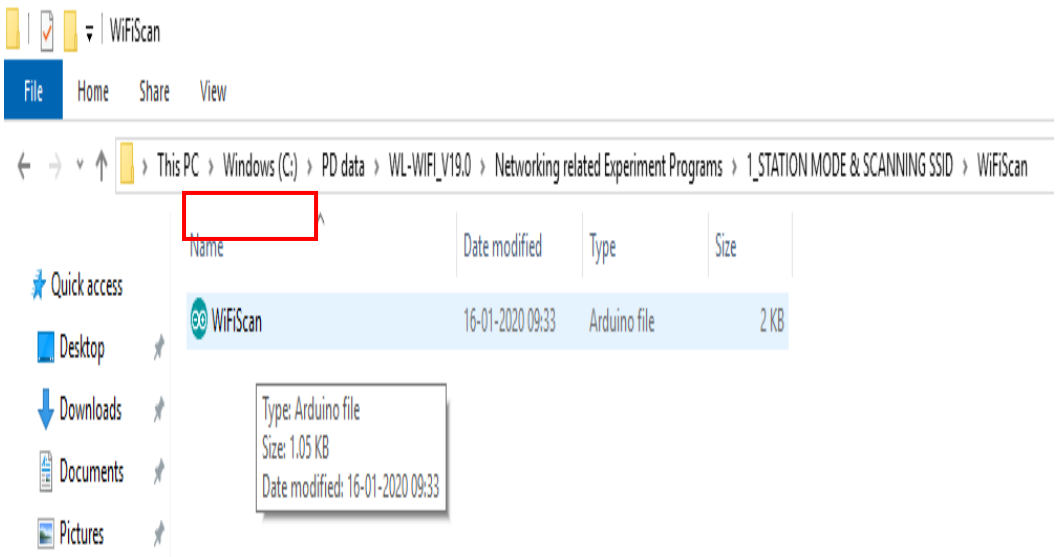
EQUIPMENTS:-

- WL-WIFI Trainer 01 no
- USB Cable 01 no
- Power supply 01 no
- PC or Laptop with installed Arduino software.

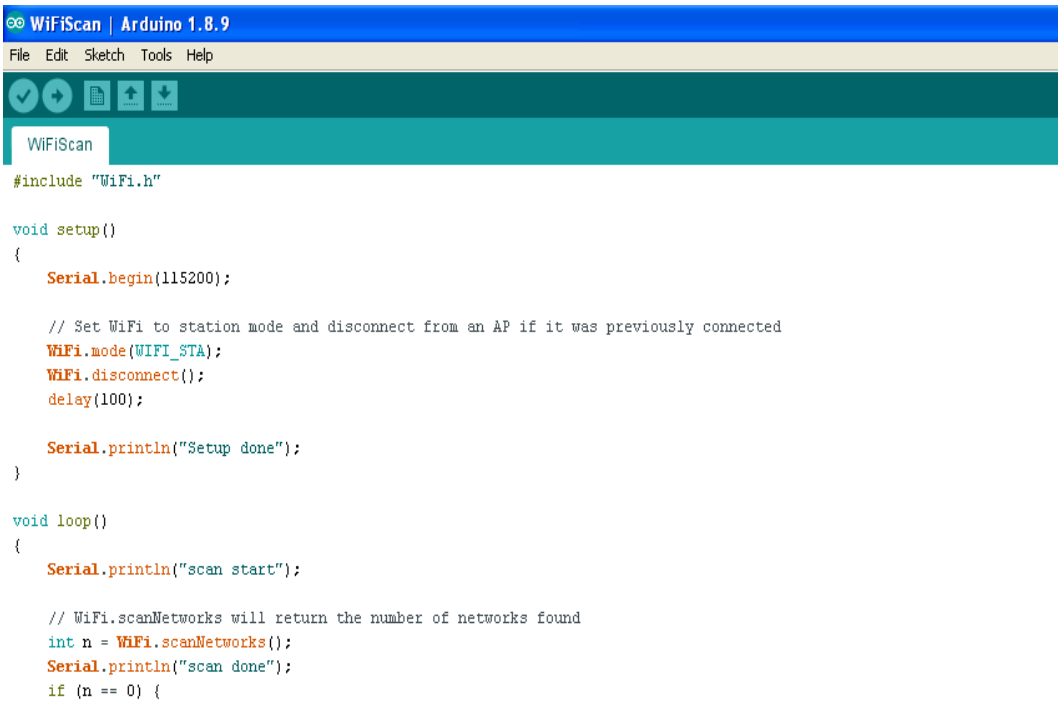
PROCEDURE:-

Below steps shows the method:

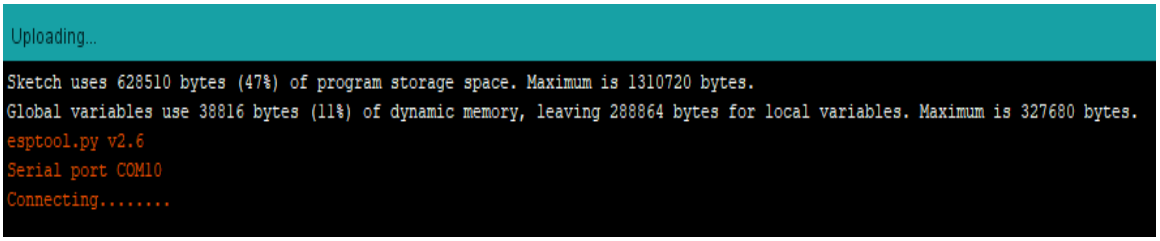
1. Connect USB Cable between PC & WIFI trainer kit.
2. Connect 5V adaptor to the WIFI trainer kit.
3. Open the given Arduino program file WIFI Scan in Arduino software from the given folder 'Networking related Experiment Programs\ 1_**STATION MODE & SCANNING SSID\WiFiScan'.**



Arduino webpage program will be open, as shown below.

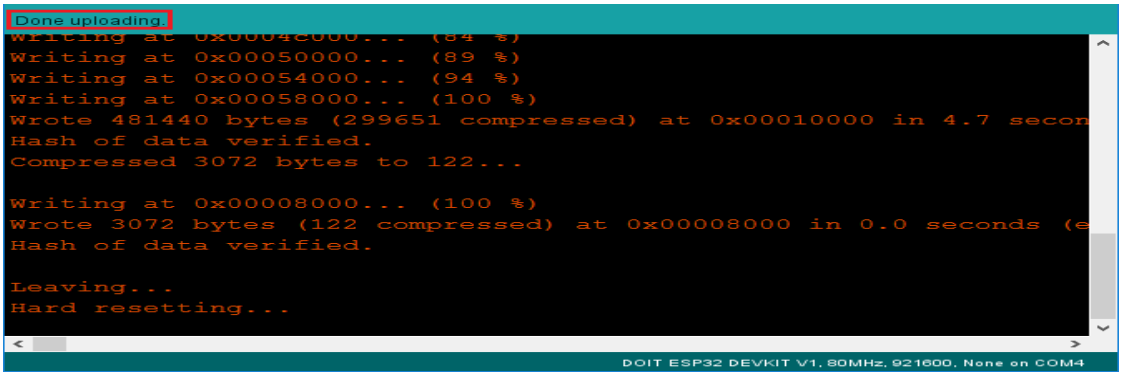


4. Press the **Upload** button in the Arduino IDE. After getting following message.



5. You need to Hold-down the "**BOOT**" button in your ESP32 board, Wait a few seconds while the code compiles and uploads to your board.

6. If everything went as expected, you should see a "**Done uploading.**" message.



7. Open the Arduino **IDE Serial Monitor** at a baud rate of 115200.

OBSERVATIONS:-

5. Press the ESP32 on-board **Enable** button and Serial Monitor shows the number available of SSID's and name of the same on serial monitor as shown below.

EXPERIMENT NO. 8

EXPERIMENT NO: 8

NAME:-

WI-FI Module as Access point mode, WIFI MODULE AS Station (STA) and Access Point (AP).

OBJECTIVE:-

WIFI module works in two modes: Station (STA) and Access Point (AP). In short, AP mode allows it to create its own network and have other devices connect to it and STA mode allows the WIFI module to connect to a Wi-Fi network. So, an important feature about the WIFI module is that it can function as a client or as an access point or even both.

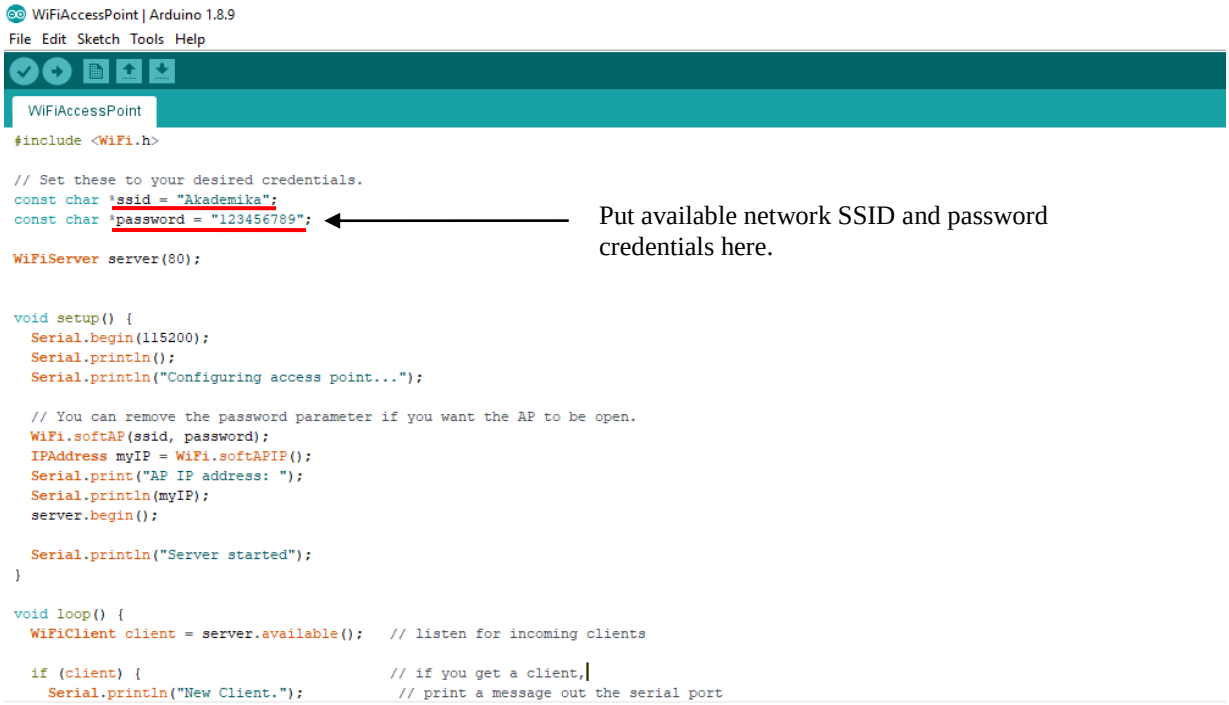
EQUIPMENTS:-

- WL-WIFI Trainer 02 no
- USB Cable 02 no
- Power supply 02 no
- PC or Laptop with installed Arduino software.

PROCEDURE:-

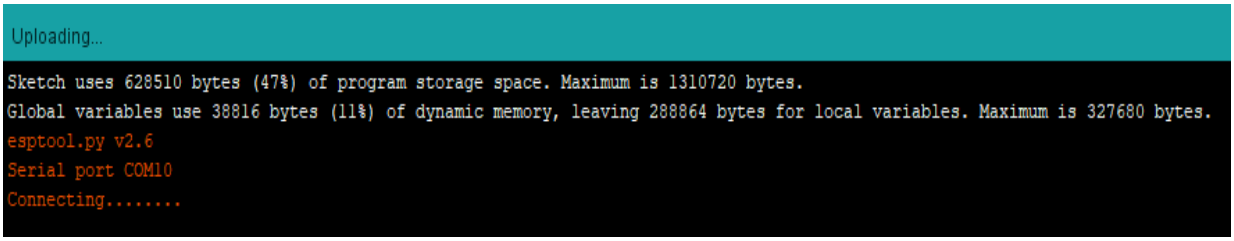
Below steps shows the method for configuring Access Point (AP):

1. Connect USB cable between PC & WIFI trainer kit.
2. Connect 5V adaptor to the WIFI trainer kit.
3. Open the given Arduino program file ‘access_point’ in Arduino software from the given folder ‘Networking related Experiment Programs\2_**ACCESS POINT MODE & CONNECTIVITY\WiFiAccessPoint**’.
4. To connect with available WI-FI network, put SSID and password credentials in that Arduino program.

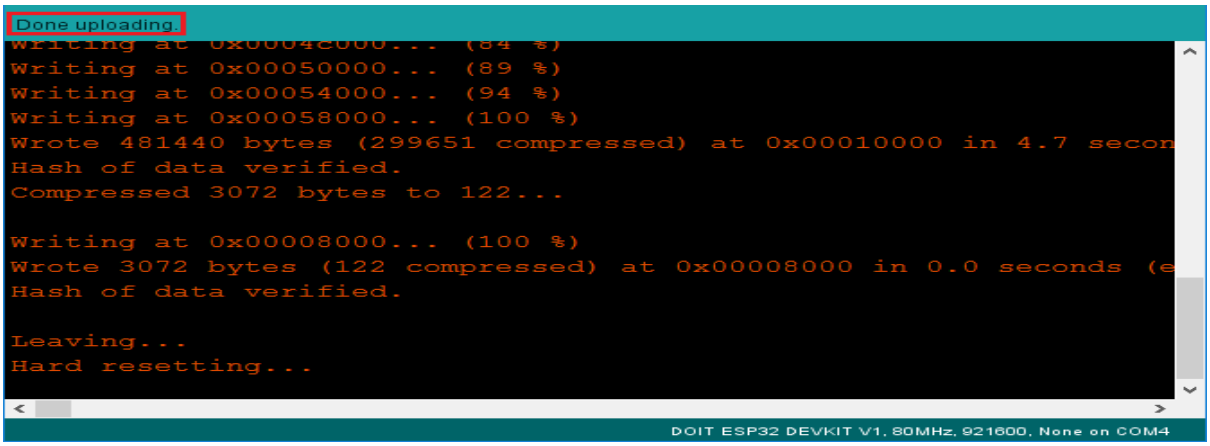


Put available network SSID and password credentials here.

3. Press the **Upload** button in the Arduino IDE. After getting following message,



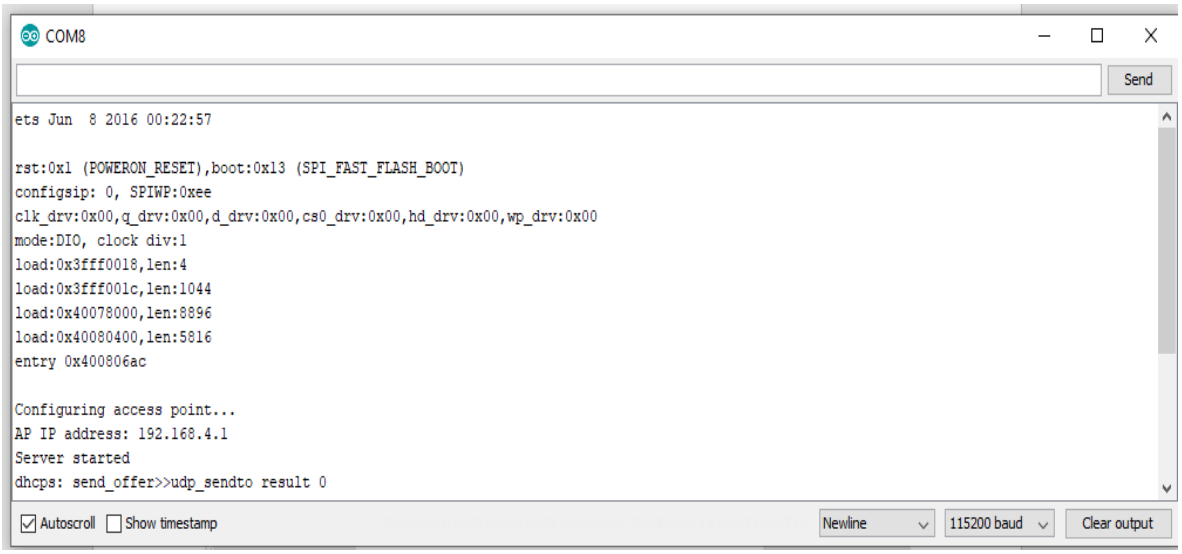
4. You need to Hold-down the "**BOOT**" button in your ESP32 board, Wait a few seconds while the code compiles and uploads to your board.
5. If everything went as expected, you should see a "**Done uploading.**" message.



6. Open the Arduino IDE Serial Monitor  at a baud rate of 115200.

7. Press the on-board **Enable** button and Serial Monitor shows the IP Address, after uploading program take your mobile or device to find access point “Akademika” and connect to it with password “12345789” as we have given in program.

OBSERVATIONS:-

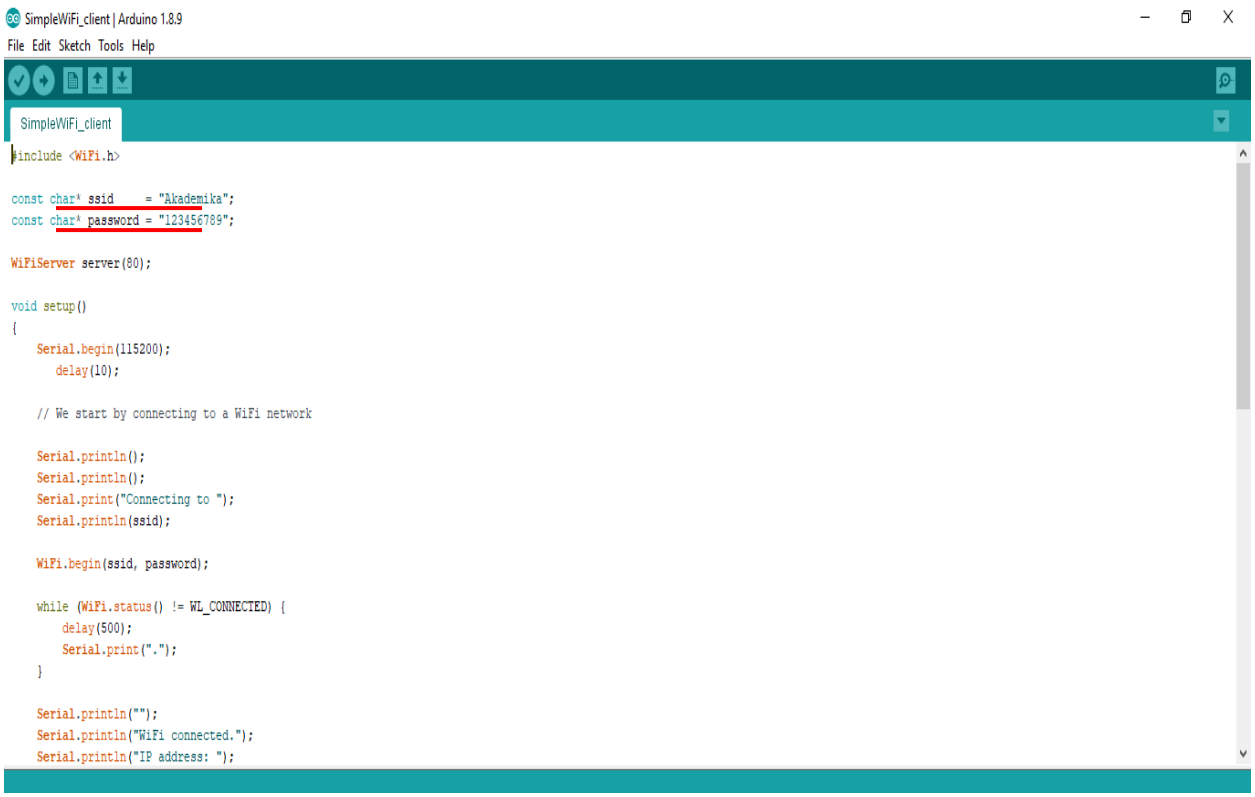


WI-FI Module as Station Mode.

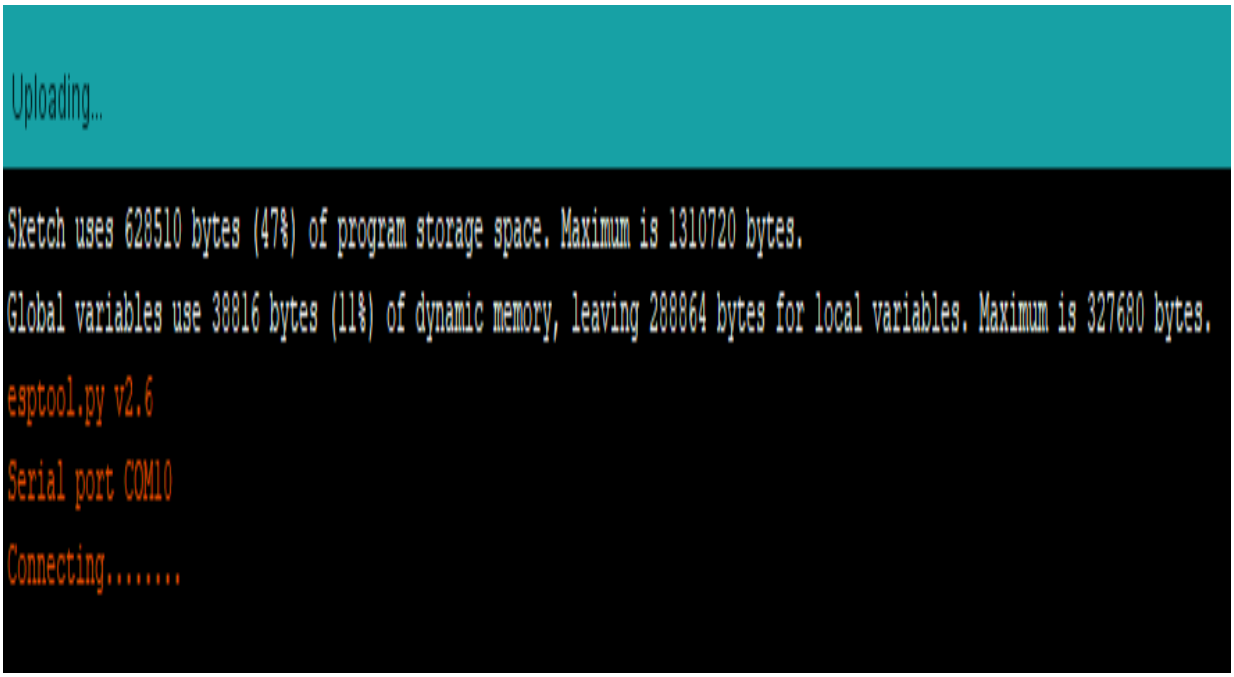
Below steps shows the method for configuring Station Mode (STA):

1. Open the given Arduino program file ‘SimpleWiFi_client’ in Arduino software from the given folder ‘Networking related Experiment Programs\2_**ACCESS POINT MODE & CONNECTIVITY\SimpleWiFi_client’**.

2. Check the COM port for client access point & change it for client (detected on Device Maneger).

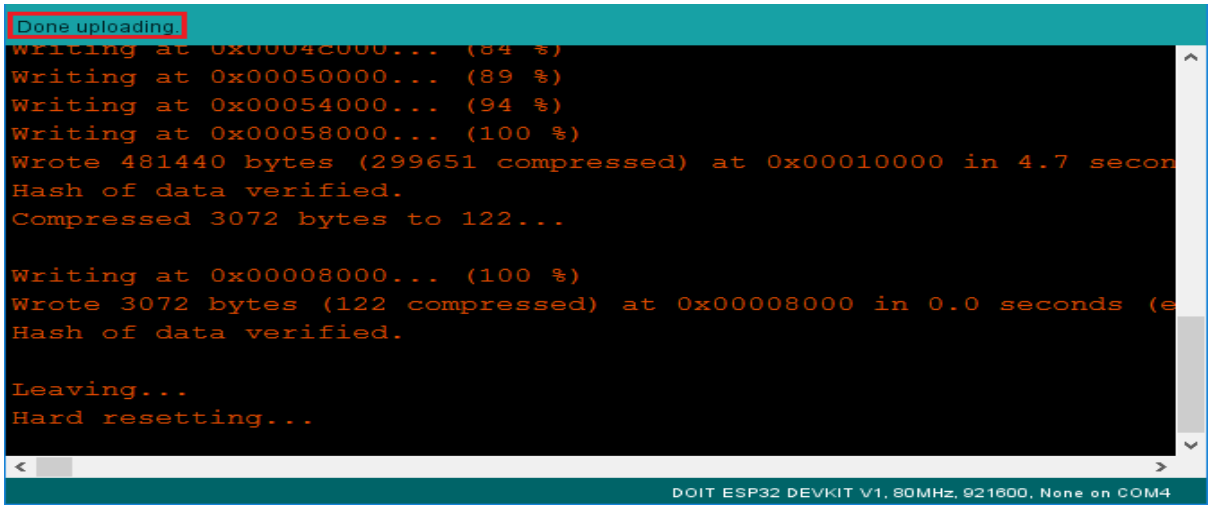


2. Press the **Upload** button in the Arduino IDE. After getting following message.



3. You need to Hold-down the "**BOOT**" button in your ESP32 board, Wait a few seconds while the code compiles and uploads to your board.

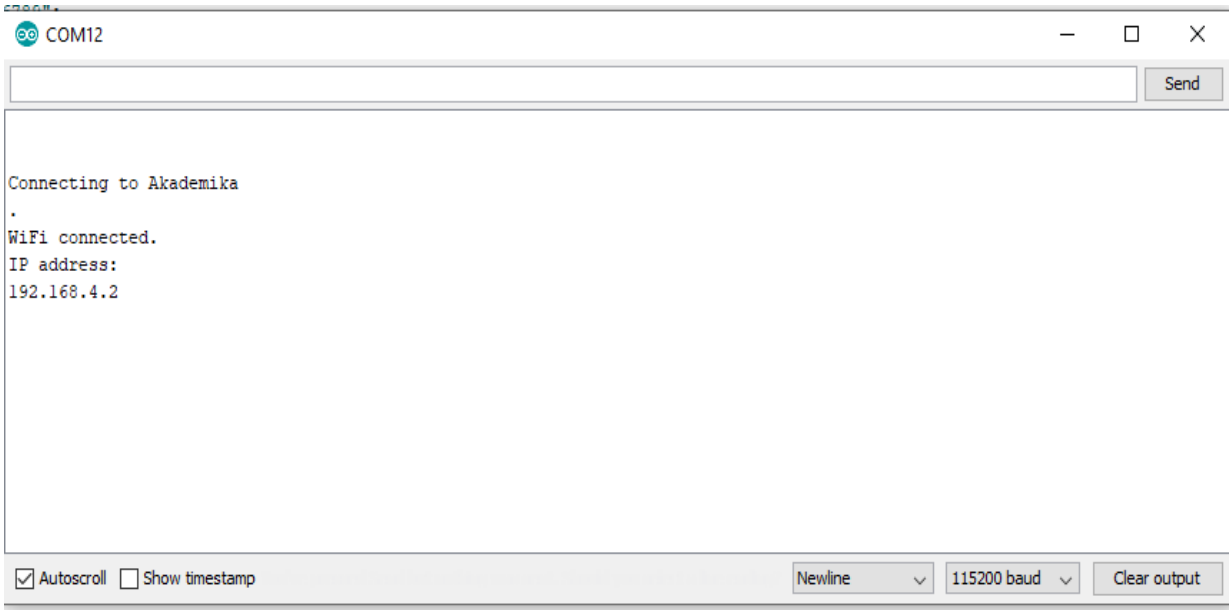
4. If everything went as expected, you should see a "**Done uploading.**" message.



5. Open the Arduino IDE Serial Monitor  at a baud rate of 115200.

6. Press the on-board **Enable** button to see the results first get the IP address from serial monitor, Open serial monitor. It shows will return the number of networks found.

OBSERVATIONS:-



EXPERIMENT NO. 9

EXPERIMENT NO: 9

NAME:-

WI-FI connectivity between two WI-FI modules in same local network.

OBJECTIVE:-

The goal of the experiment is to understand connectivity between two WI-FI modules in same local network.

This experiment involves three WI-FI modules. One WI-FI module in access point mode and make remaining two WI-FI devices in station mode which are we need to communicate with each other and successfully data transfer between them through the same local network.

EQUIPMENTS:-

- WL-WIFI Trainer 03 nos.
- USB Cable 03 nos.
- Power supply 03 nos.
- PC or Laptop with installed Arduino software.

PROCEDURE:-

1. Connect USB Cable between PC & WIFI trainer kit.
2. Connect 5V adaptor to the WIFI trainer kit.

Follow the following steps to check connectivity between two WI-FI modules in same local network:

Step 1: To configure one WI-FI Module as Access Point (AP):

3. Open the given arduino program file ‘access_point’ in Arduino software from the given folder Networking related Experiment Programs\3_**LOCAL NETWORK DATA TRANFER\WiFiAccessPoint**’.

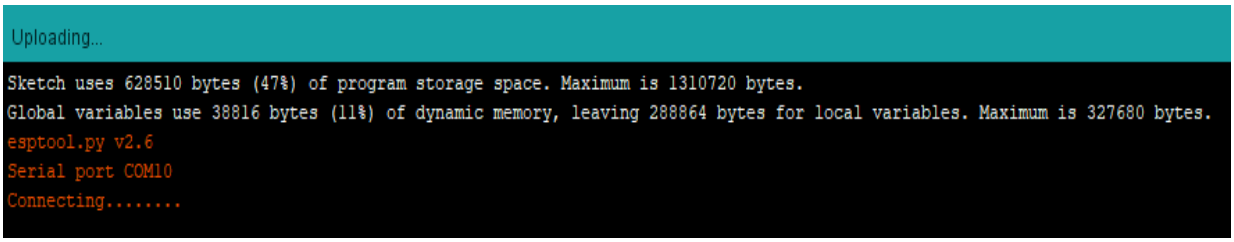
WiFiAccessPoint | Arduino 1.8.9

File Edit Sketch Tools Help

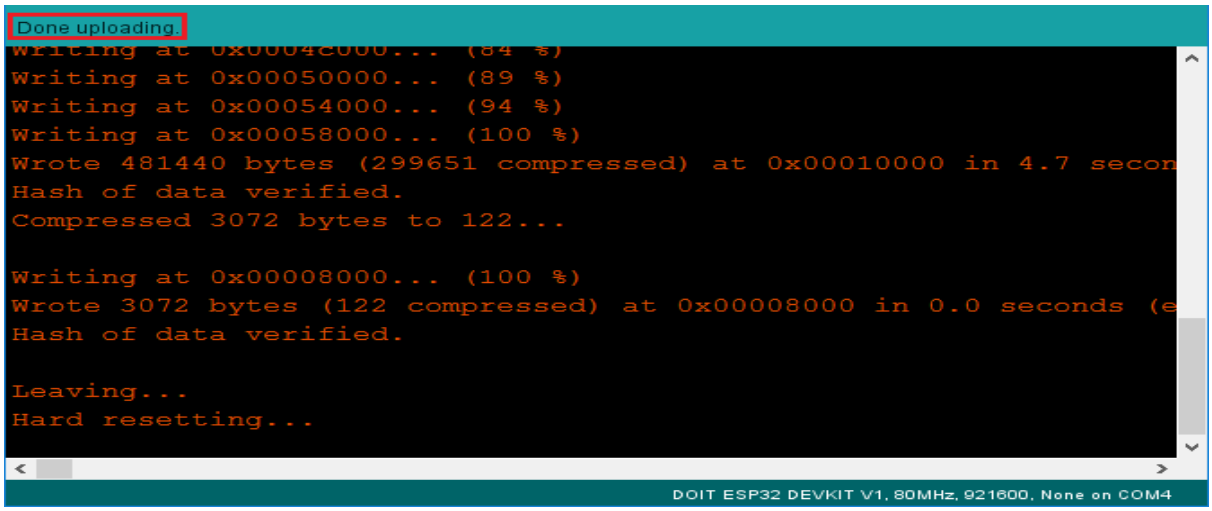
WiFiAccessPoint

```
#include <WiFi.h>

// Set these to your desired credentials.
const char *ssid = "Akademika";
const char *password = "123456789";

WiFiServer server(80);


void setup() {
  Serial.begin(115200);
  Serial.println();
  Serial.println("Configuring access point...");

  // You can remove the password parameter if you want the AP to be open.
  WiFi.softAP(ssid, password);
  IPAddress myIP = WiFi.softAPIP();
  Serial.print("AP IP address: ");
  Serial.println(myIP);
  server.begin();

  Serial.println("Server started");
}

void loop() {
  WiFiClient client = server.available();   // listen for incoming clients

  if (client) {                             // if you get a client,
    Serial.println("New Client.");          // print a message out the serial port
```

1. Press the **Upload**  button in the Arduino IDE. After getting following message.



2. You need to Hold-down the "**BOOT**" button in your ESP32 board, Wait a few seconds while the code compiles and uploads to your board.
3. If everything went as expected, you should see a "**Done uploading.**" message.



4. Open the Arduino IDE Serial Monitor  at a baud rate of 115200.

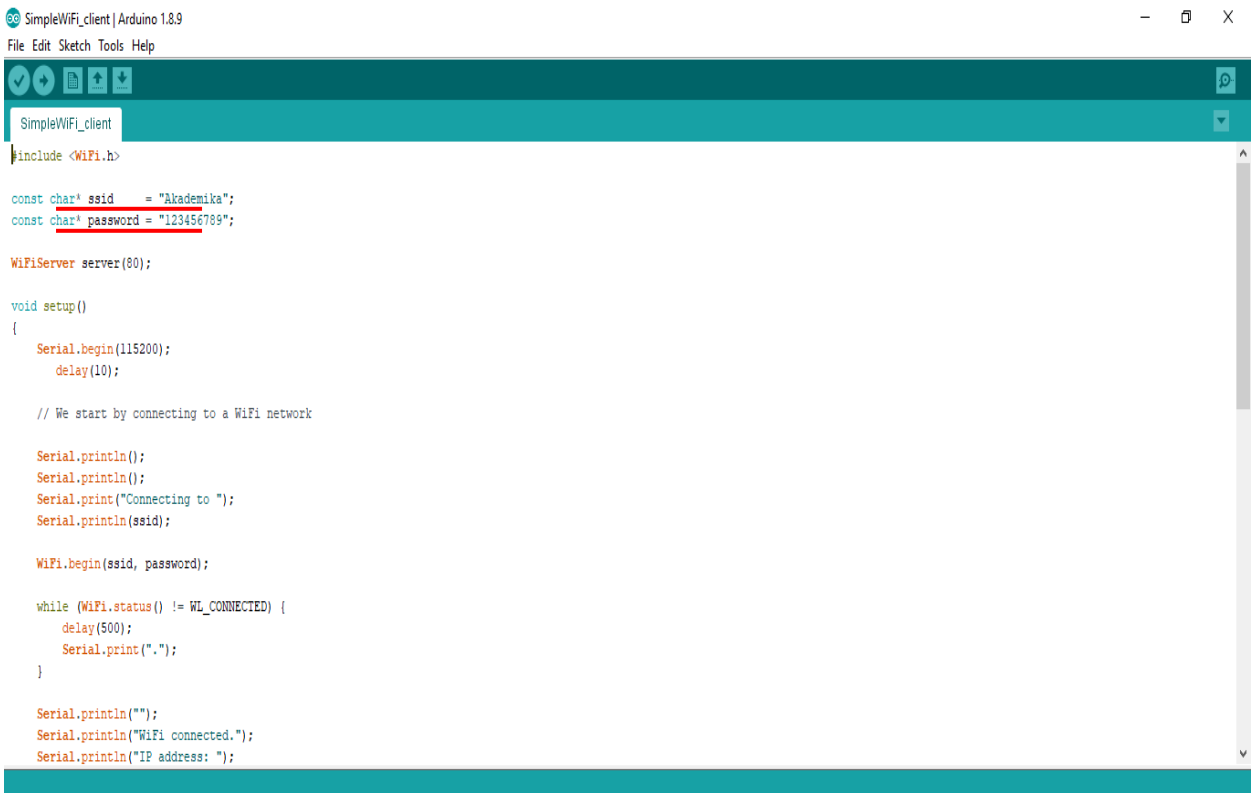
5. Press the on-board Enable button and Serial Monitor shows the IP Address, after uploading program take your mobile or device to find access point “Akademika” and connect to it with password “12345789” as we have given in program.

WI-FI Module as Station Mode.

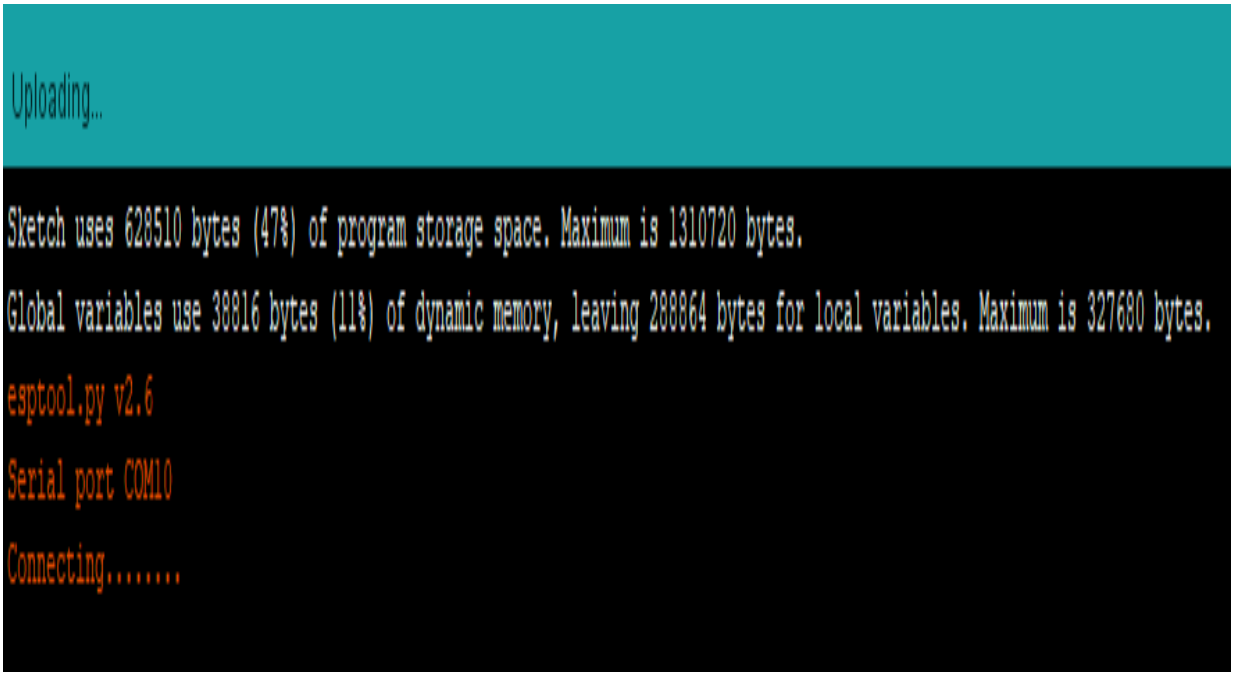
Below steps shows the method for configuring Station Mode (STA):

1. Open the given Arduino program file ‘SimpleWiFi_client’ in Arduino software from the given folder ‘Networking related Experiment Programs\2_**ACCESS POINT MODE & CONNECTIVITY\SimpleWiFi_client’**.

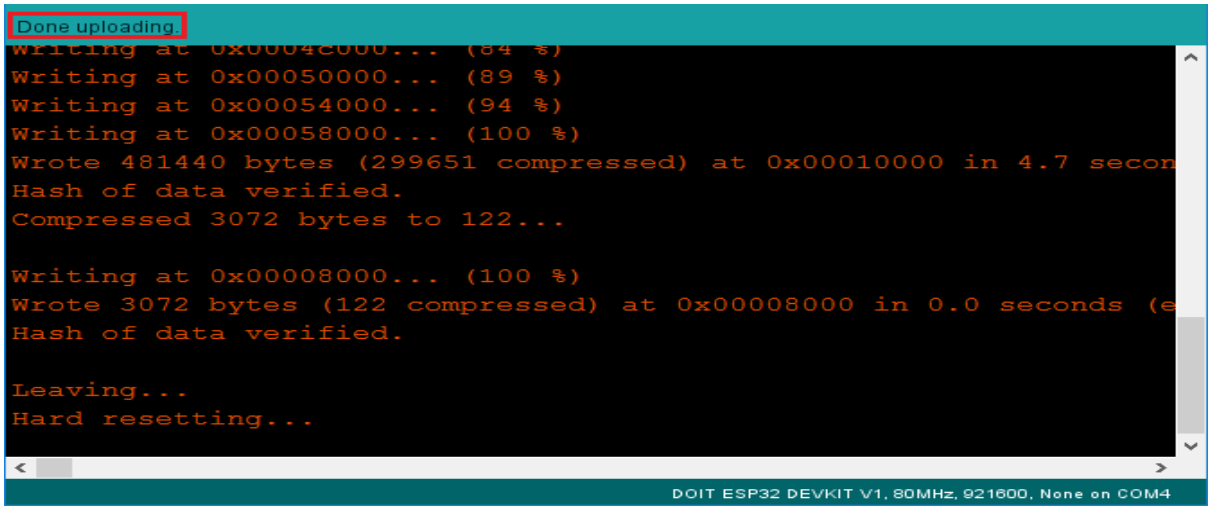
2. Check the COM port for client access point & change it for client (detected on Device Maneger).



7. Press the **Upload**  button in the Arduino IDE. After getting following message.



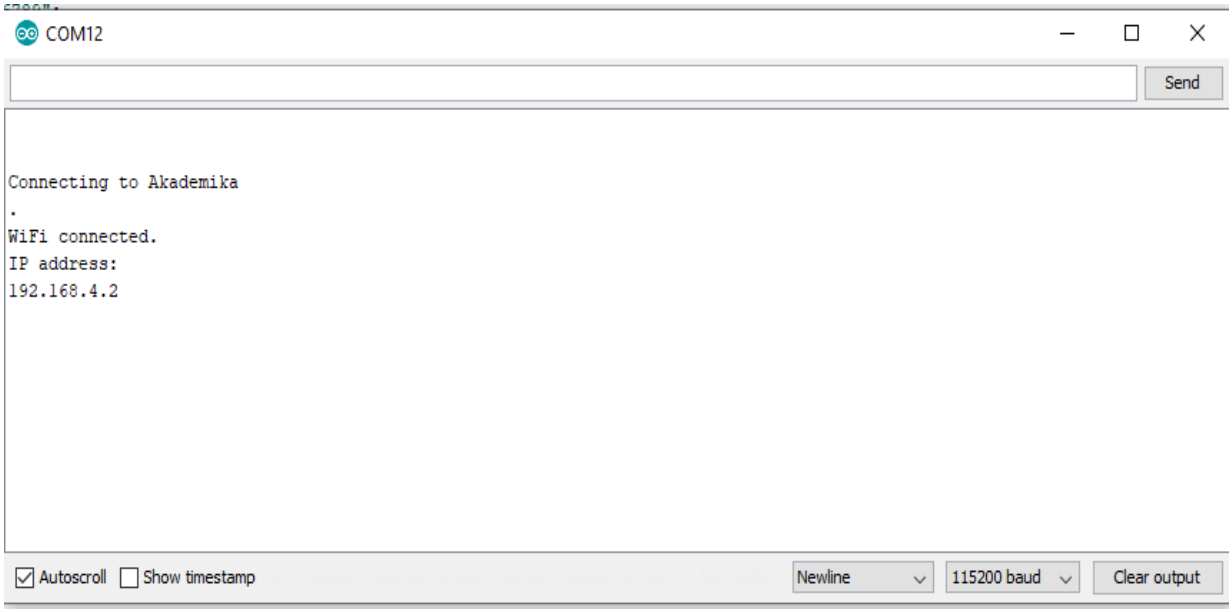
8. You need to Hold-down the "**BOOT**" button in your ESP32 board, Wait a few seconds while the code compiles and uploads to your board.
9. If everything went as expected, you should see a "**Done uploading.**" message.



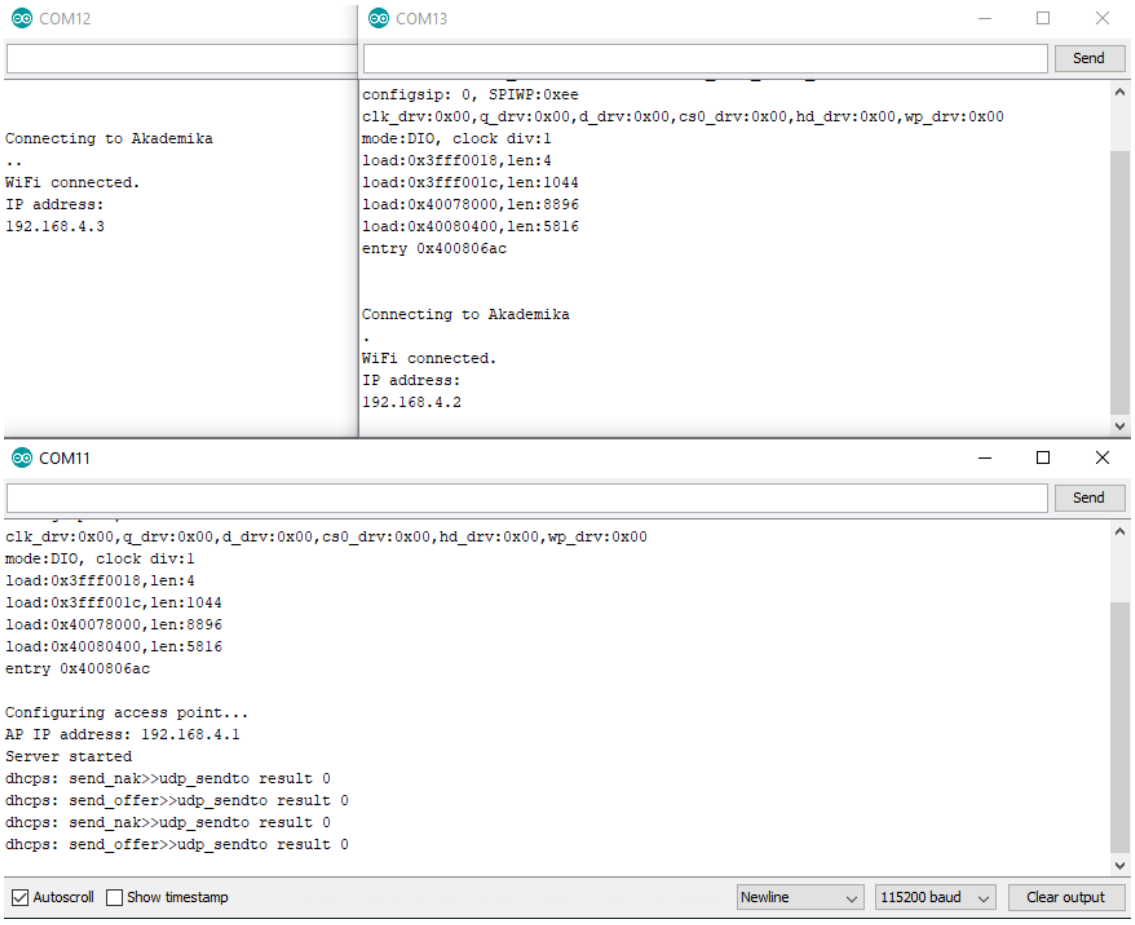
10. Open the Arduino IDE Serial Monitor  at a baud rate of 115200.

11. Press the on-board **Enable** button to see the results first get the IP address from serial monitor, Open serial monitor.It shows will return the number of networks found.

OBSERVATIONS:-



12. For second client access point reapete the steps from 1 to 11 for client access point.



EXPERIMENT NO. 10

EXPERIMENT NO: 10

NAME:-

SIMPLE DIRECT COMMUNICATION BETWEEN TWO WI-FI MODULES.

OBJECTIVE:-

In this experiment ESP-NOW Protocol is used which enables multiple devices to communicate with one another without using WI-FI.
The goal of the experiment is to understand simple direct connectivity between two WI-FI modules.
This experiment involves two WI-FI modules. Simple direct communications between two WI-FI devices. Make one WI-FI device as Transmitter and another one as Receiver.

EQUIPMENTS:-

- WL-WIFI Trainer 02 no
- USB Cable 02 no
- Power supply 02 no
- PC or Laptop with installed Arduino software.

PROCEDURE:-

Title: WI-FI Module as esp_sender & esp_receiver.

1. Connect USB cable betwwen PC & WIFI trainer.
2. Connect 5V adaptor to the WIFI trainer.

Connect two WI-FI boards to PC and follow the following steps:

Below steps shows the method for configuring 1st WI-FI board as ESP_SENDER:

3. Open the given arduino program file ‘esp_sender’ in Arduino software from the given folder path ‘Networking related Experiment Programs\ 4_**SIMPLE DIRECCT COMMUNICATION\esp_sender’**.

esp_sender | Arduino 1.8.9

File Edit Sketch Tools Help

esp_sender

```
#include <esp_now.h>
#include <WiFi.h>

uint8_t broadcastAddress[] = {0xFF, 0xFF,0xFF,0xFF,0xFF,0xFF};

char serial_data;

void setup() {

  Serial.begin(115200);

  WiFi.mode(WIFI_STA);

  Serial.println();
  Serial.println(WiFi.macAddress());

  if (esp_now_init() != ESP_OK) {
    Serial.println("Error initializing ESP-NOW");
    return;
  }

  // register peer
  esp_now_peer_info_t peerInfo;

  memcpy(peerInfo.peer_addr, broadcastAddress, 6);
  peerInfo.channel = 0;
  peerInfo.encrypt = false;

  if (esp_now_add_peer(&peerInfo) != ESP_OK){
```

4. Now, select appropriate **COM-PORT** of 1st WI-FI board, then Press the Upload  button in the **Arduino IDE**. After getting following message.

```
Uploading...

Sketch uses 628510 bytes (47%) of program storage space. Maximum is 1310720 bytes.
Global variables use 38816 bytes (11%) of dynamic memory, leaving 288864 bytes for local variables. Maximum is 327680 bytes.
esptool.py v2.6
Serial port COM10
Connecting.....
```

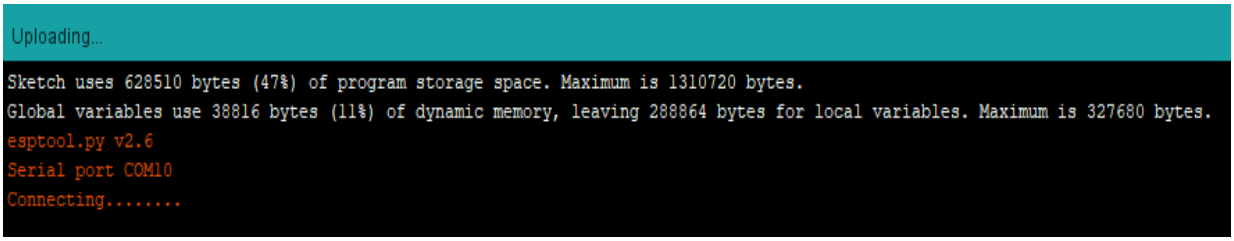
5. You need to Hold-down the "**BOOT**" button in your ESP32 board, Wait a few seconds while the code compiles and uploads to your board.
   If everything went as expected, you should see a "**Done uploading**." message.

```
Done uploading.

Writing at 0x00050000... (89 %)
Writing at 0x00054000... (94 %)
Writing at 0x00058000... (100 %)
Wrote 481440 bytes (299651 compressed) at 0x00010000 in 4.7 secon
Hash of data verified.
Compressed 3072 bytes to 122...

Writing at 0x00008000... (100 %)
Wrote 3072 bytes (122 compressed) at 0x00008000 in 0.0 seconds (e
Hash of data verified.

Leaving...
Hard resetting...

DOIT ESP32 DEVKIT V1, 80MHz, 921600, None on COM4
```

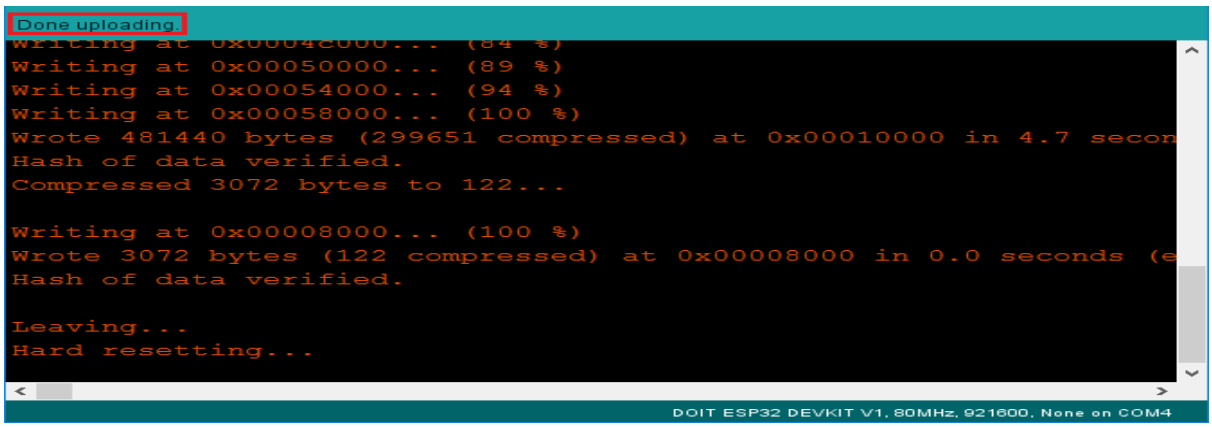
Below steps shows the method for configuring 2nd WI-FI board as **ESP_RECEIVER**:

6. Open the given Arduino program file ‘esp_receiver’ in Arduino software from the given folder path ‘Networking related Experiment Programs\ 4_**SIMPLE DIRECCT COMMUNICATION\ esp_receiver**’.

7. Now, select appropriate **COM-PORT** of 2nd WI-FI board, then Press the Upload  button in the **Arduino IDE**. After getting following message.

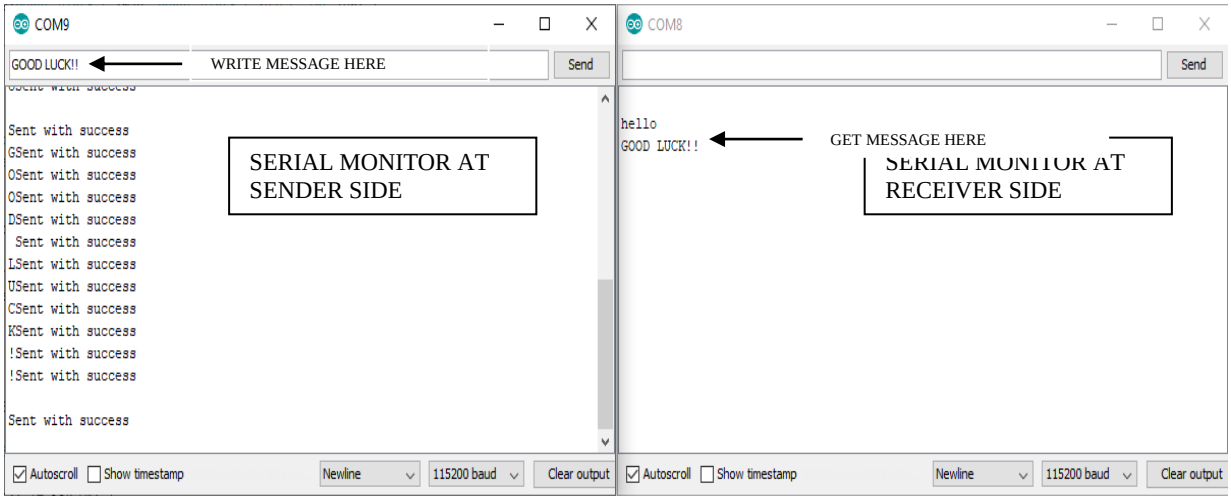


8. You need to Hold-down the “**BOOT**” button in your ESP32 board, Wait a few seconds while the code compiles and uploads to your board.
9. If everything went as expected, you should see a “**Done uploading**.” message.



OBSERVATION:-

10. Now open the both Serial Monitors  at a baud rate of 115200 and press the on-board **Enable** button for both boards. The Serial monitors of both programs shows results as shown below:



EXPERIMENT NO. 11

EXPERIMENT NO: 11

NAME:-

WI-FI TRAINER AS WEB SERVER TO CONTROL THE ON-BOARD PERIPHERALS IN THE LOCAL NETWORK.

OBJECTIVE:-

This experiment involves one Wi-fi Trainer board and one PC/MOBILE. Here Wi-Fi module is in access point mode and PC/MOBILE device is in station mode and PC/MOBILE should be connected to Wi-Fi module network. This is just a simple example to illustrate how to build a web server that controls outputs (such as LED, Relay).

THEORY:-

A web server can mean two things - a computer on which a web site is hosted and a program that runs on such a computer. So, the term web server refers to both hardware and software. We'll look at these two one by one.
A web site is a collection of web pages which are digital files generally written using Hyper Text Markup Language (HTML). For a web site to be available to everyone in the world at all times, it needs to be stored or "hosted" on a computer that is connected to the internet. Such a computer is known as a Web Server.

There are several requirements for a Server - it needs to be fast, have a large storage capacity hard disk and lots of RAM. But the most important is having a permanent internet address also known as an I.P. (Internet protocol) address. If the I.P. address changes, the web site would not be found and will appear offline - the browser will display cannot find web site error.

In WL-WIFI Trainer modules provide web service functionalities through the TCP/IP stack and a light Web Server. The Web Server is a collection of libraries concurring to enable the WIFI Module to serve web pages and dynamic content in a way similar to a real web server. Remember the WIFI Module is an embedded device so it is not meant to replace a full-fledged web server like apache.

The goal of the experiment is to understand a standalone web server with an Wi-Fi Module that controls on board peripherals.

EQUIPMENTS:-

- WL-WIFI Trainer 01 no
- USB Cable 01 no
- Power supply 01 no
- PC or Laptop with installed Arduino software.

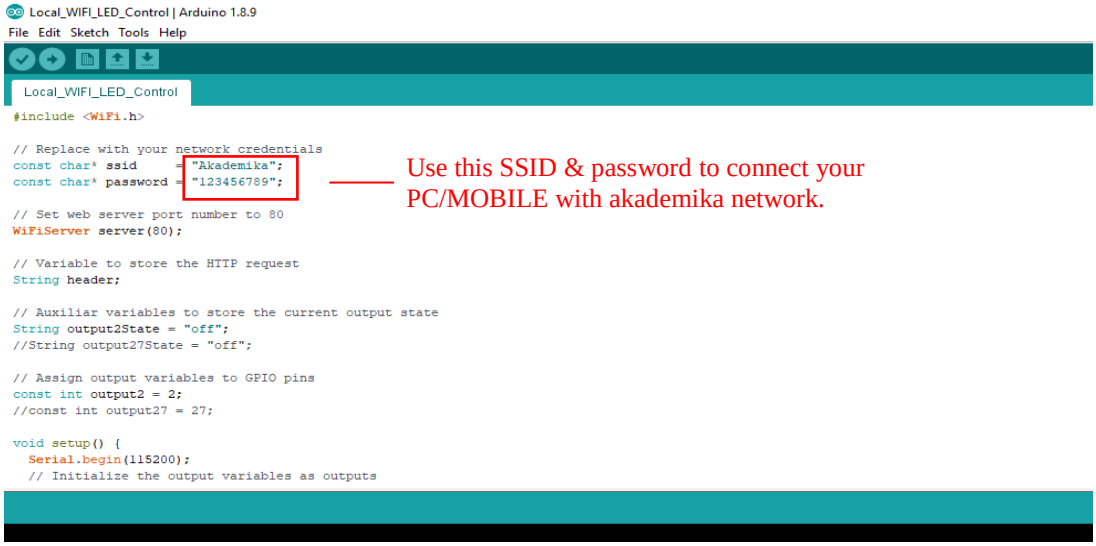
PROCEDURE:-

1. Connect USB Cable between PC & WIFI triner.
2. Connect 5V adaptor to the WIFI trainer.

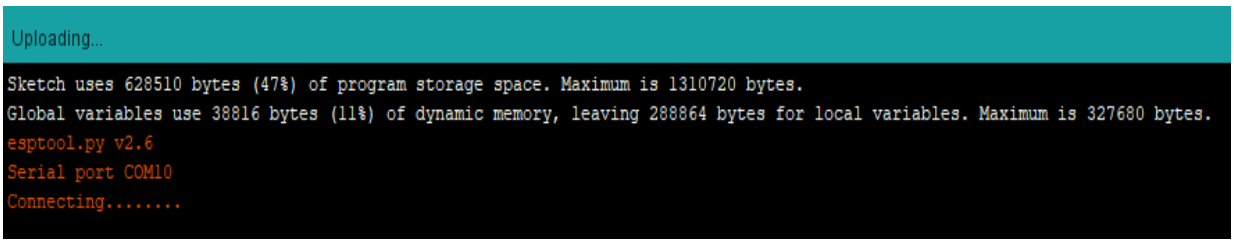
Title: Wi-Fi Trainer board as web server to controll on-board LED in local network.

Below steps shows the method:

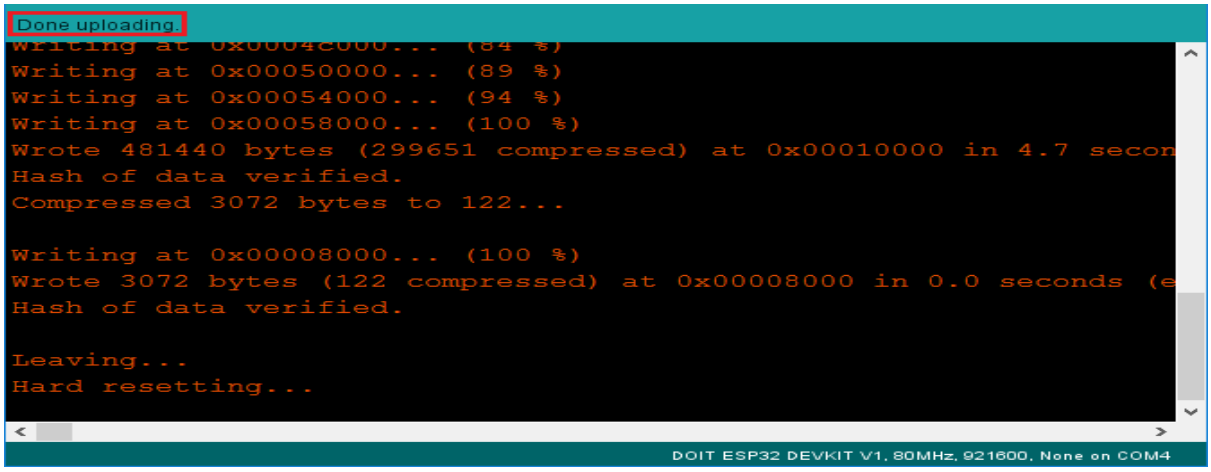
3. Open the given arduino program file ‘Local_WIFI_LED_Control’ in Arduino software from the given folder path ‘WL-WIFI\Networking related Experiment Programs\5_LOCAL NET PERIPHERAL WEB CONTROL**Local_WIFI_LED_Control’**.
4. Arduino webpage program will be open, as shown below. Here use given SSID & password with akademika network.



5. Press the **Upload** button in the Arduino IDE. After getting following message,

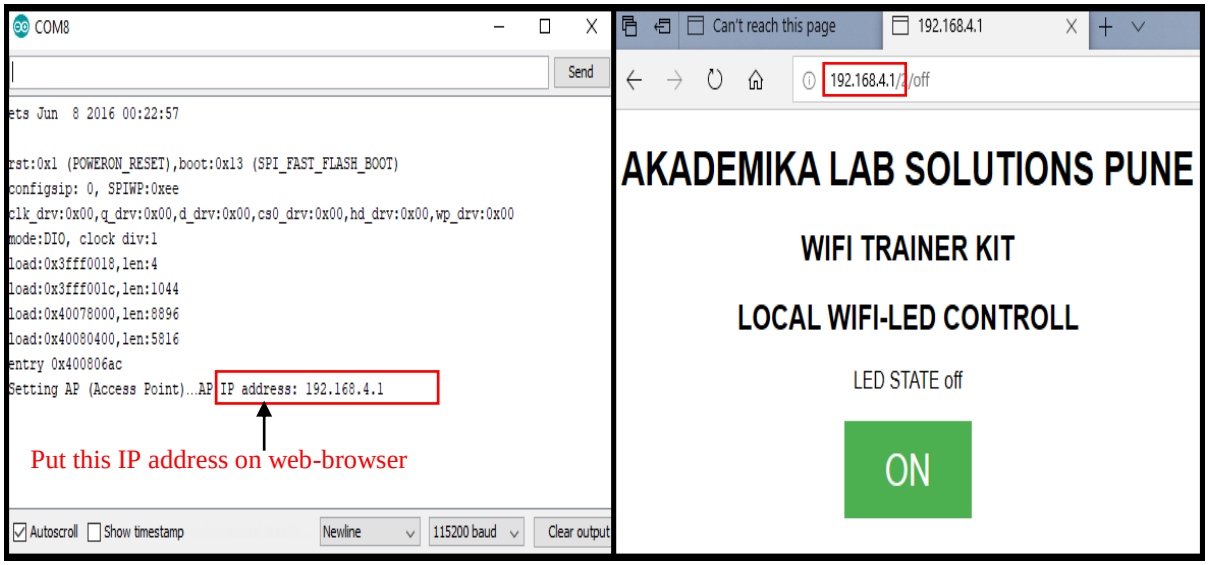


6. You need to Hold-down the “**BOOT**” button in your ESP32 board, Wait a few seconds while the code compiles and uploads to your board.
7. If everything went as expected, you should see a “**Done uploading.**” message.



8. Open the Arduino IDE Serial Monitor  at a baud rate of 115200.

9. Press the ESP32 on-board **Enable** button and Serial Monitor shows the IP Address, put this IP Address on a Web Browser, we can access the webpage and can control on-board LED ON/OFF operation in same Wi-Fi network.

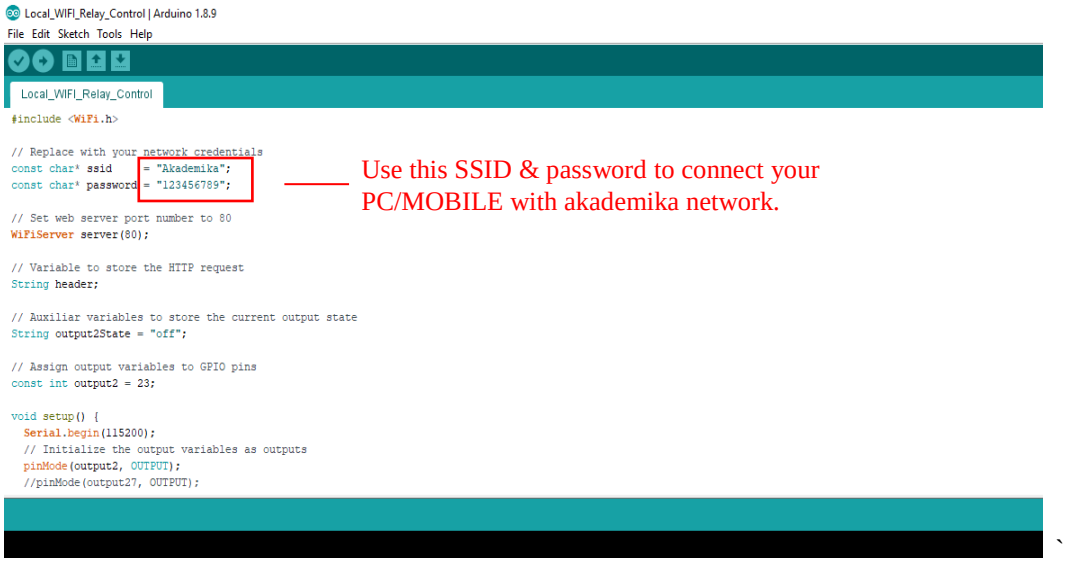


Serial Monitor Result Web-Browser Result

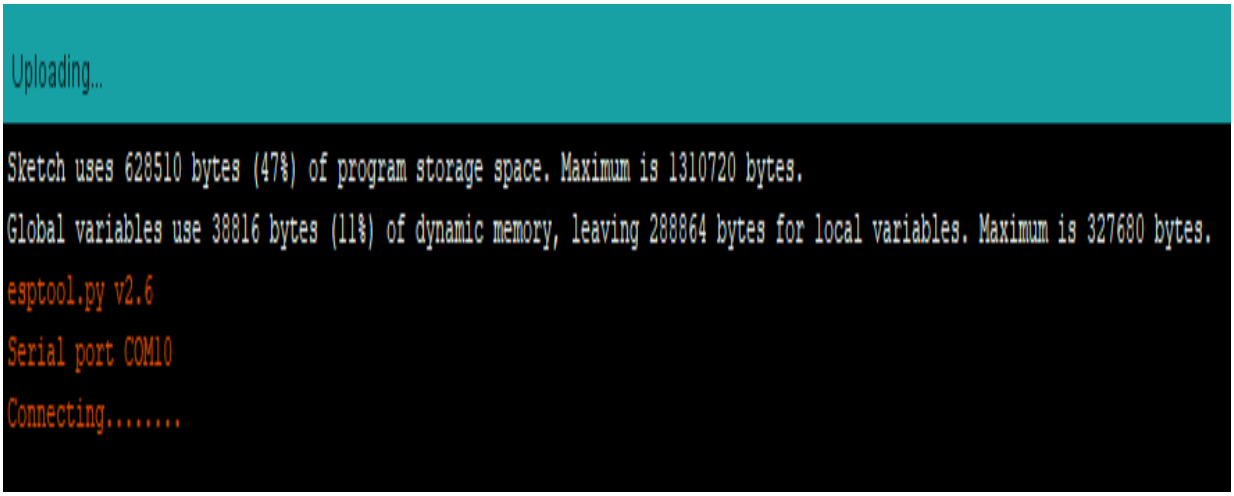
Title: Wi-Fi Trainer board as web server to controll Realy in local network.

Below steps shows the method:

1. Open the given arduino program file ‘Local_WIFI_Relay_Control’ in Arduino software from the given folder path ‘WL-WIFI_V19.0\Networking related Experiment Programs\5_LOCAL NET PERIPHERAL WEB CONTROL\ **Local_WIFI_Relay_Control’**.
2. Arduino webpage program will be open, as shown below. Here use given ssid & password with akademika network.

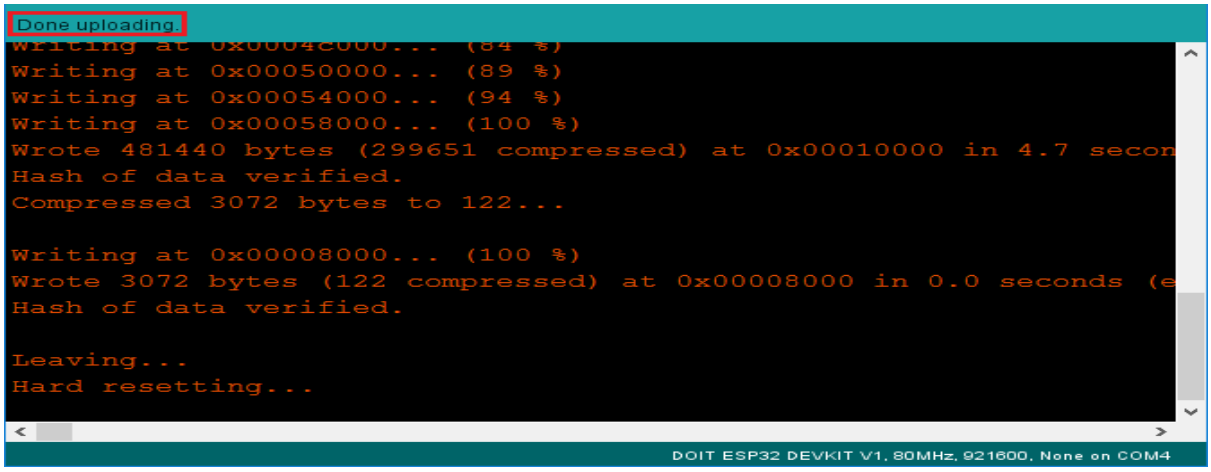


3. Press the **Upload** button in the Arduino IDE. After getting following message.



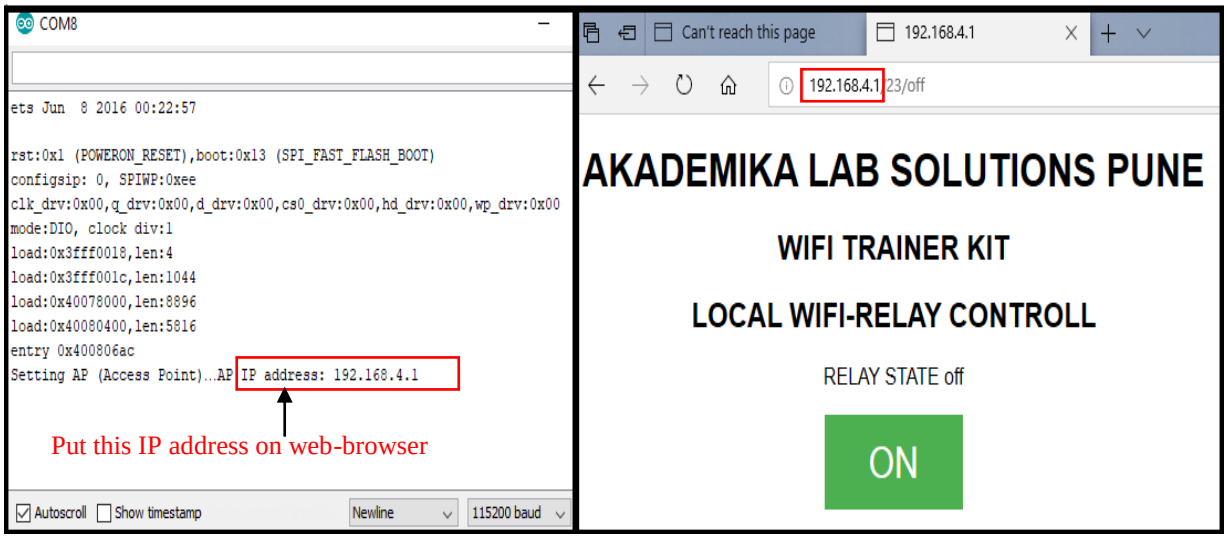
4. You need to Hold-down the “**BOOT**” button in your ESP32 board, Wait a few seconds while the code compiles and uploads to your board.

5. If everything went as expected, you should see a “**Done uploading.**” message.



6. Open the **Arduino IDE Serial Monitor**  at a baud rate of 115200.

7. Press the ESP32 on-board **Enable** button and Serial Monitor shows the IP Address, put this IP Address on a Web Browser, we can access the webpage and can control relay ON/OFF operation in same Wi-Fi network.



Serial Monitor Result Web-Browser Result

EXPERIMENT NO. 12

EXPERIMENT NO: 12

NAME:-

DYNAMIC WEBSERVER PAGE.

OBJECTIVE:-

As we seen in the previous example, getting data from the server to the client is easy enough: just place a few keywords and you can get control over what is sent to the client. Nothing prevents you from using the same method to create html code on the WIFI Module. In Dynamic Webpage information flow bi-directionally so we will see the parameters from sever and control some parameters or hardware located in server side.

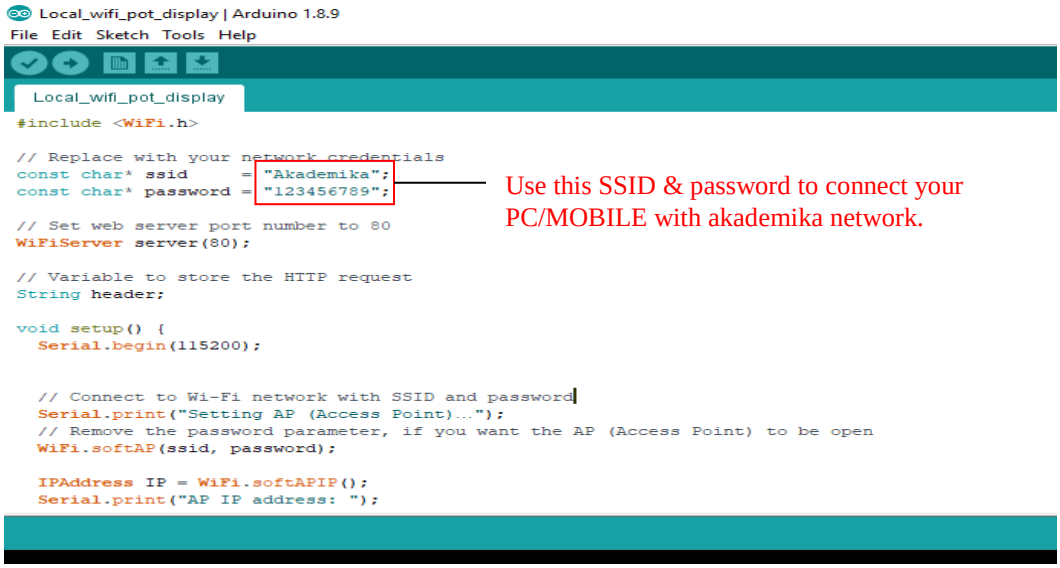
EQUIPMENTS:-

- WL-WIFI Trainer 01 no
- USB Cable 01 no
- Power supply 01 no
- PC or Laptop with installed Arduino software.

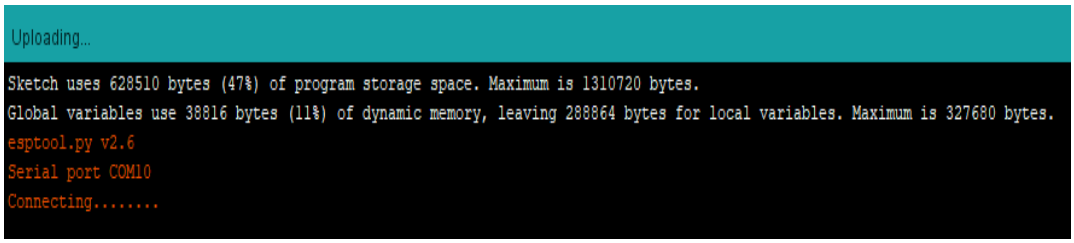
PROCEDURE:-

Below steps shows the method to get potentiometer variable voltage results on webpage:

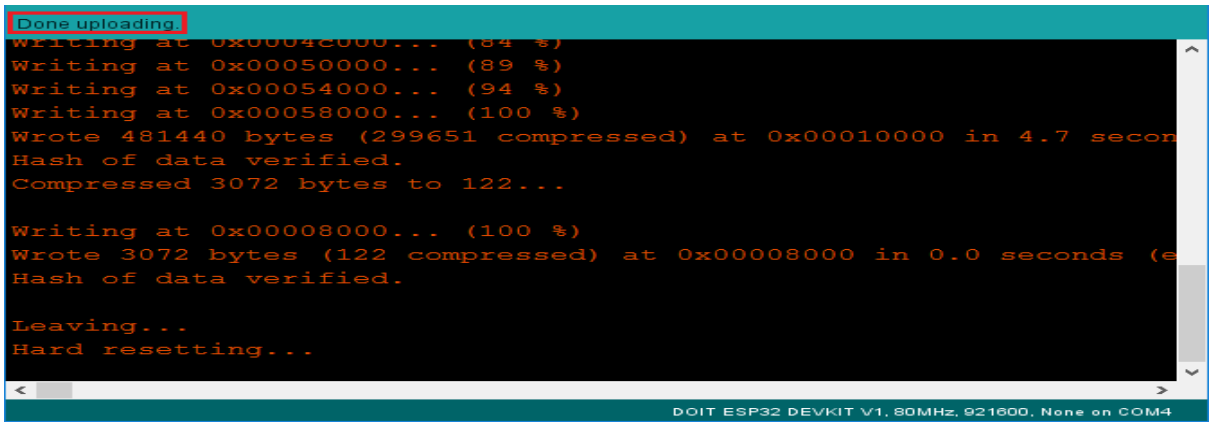
1. Connect USB cable between PC & WIFI trainer.
2. Connect 5V adaptor to the WIFI trainer.
3. Open the given arduino program file ‘Local_wifi_pot_display’ in Arduino software from the given folder path ‘WL-WIFI_V19.0\Networking related Experiment Programs\ 6_**LOCAL NET PERIPHERAL OUT ON WEB DISPLAY\Local_wifi_pot_display’**.
4. Arduino webpage program will be open, as shown below. Here use given SSID & password with akademika network.



5. Press the **Upload** button in the Arduino IDE. After getting following message.

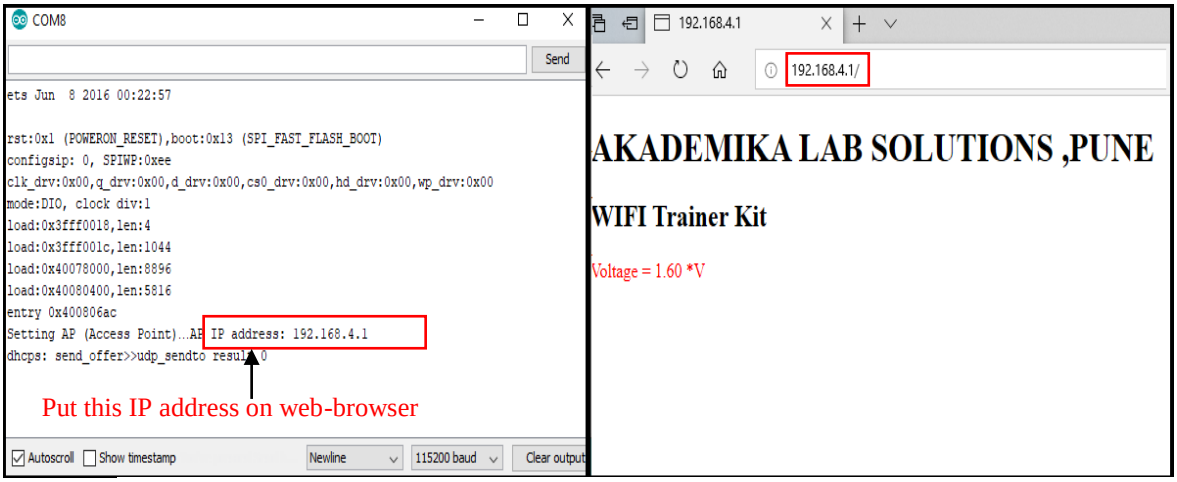


6. You need to Hold-down the "**BOOT**" button in your ESP32 board, Wait a few seconds while the code compiles and uploads to your board.
7. If everything went as expected, you should see a "**Done uploading**" message.



8. Open the Arduino IDE Serial Monitor at a baud rate of 115200.

9. Press the ESP32 on-board **Enable** button and Serial Monitor shows the IP Address, put this IP Address on a Web Browser, we can access the webpage and it shows variable voltage across potentiometer.



Serial Monitor Result Web-Browser Result

Below steps shows the method to get temperature sensor results on webpage:

1. Open the given Arduino program file ‘Local_wifi_temp_display’ in Arduino software from the given folder ‘WL-WIFI_V19.0\Networking related Experiment Programs\6_**LOCAL NET PERIPHERAL OUT ON WEB DISPLAY\Local_wifi_temp_display**’.
2. Arduino webpage program will be open, as shown below. Here use given SSID & password with akademika network.

3. Press the **Upload**  button in the Arduino IDE. After getting following message,

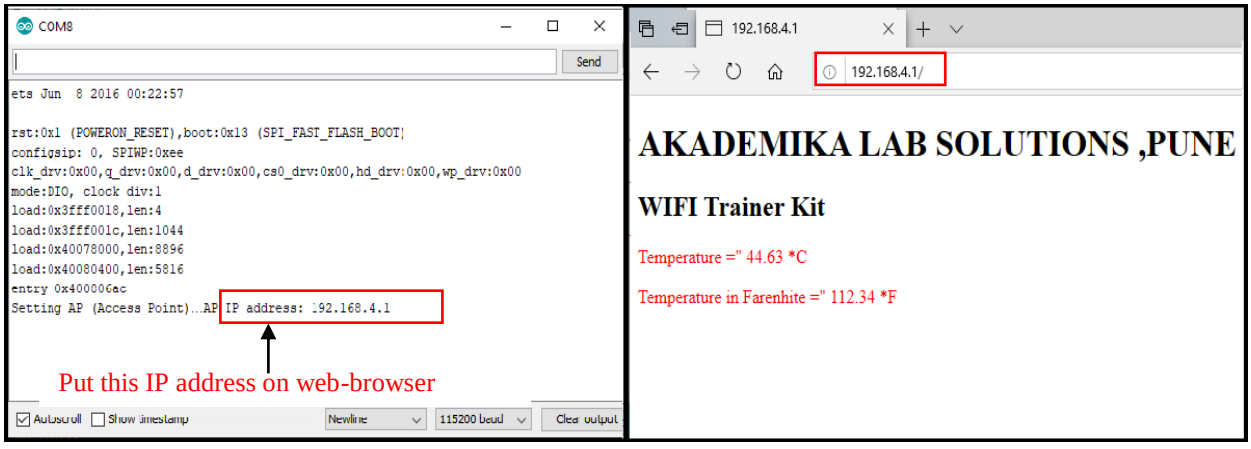
```
Uploading...
Sketch uses 628510 bytes (47%) of program storage space. Maximum is 1310720 bytes.
Global variables use 38816 bytes (11%) of dynamic memory, leaving 288864 bytes for local variables. Maximum is 327680 bytes.
esptool.py v2.6
Serial port COM10
Connecting.....
```

4. You need to Hold-down the “**BOOT**” button in your ESP32 board, Wait a few seconds while the code compiles and uploads to your board.
5. If everything went as expected, you should see a “**Done uploading.**” message.

```
Done uploading.
Writing at 0x00042000... (84 %)
Writing at 0x00050000... (89 %)
Writing at 0x00054000... (94 %)
Writing at 0x00058000... (100 %)
Wrote 481440 bytes (299651 compressed) at 0x00010000 in 4.7 secon
Hash of data verified.
Compressed 3072 bytes to 122...

Writing at 0x00008000... (100 %)
Wrote 3072 bytes (122 compressed) at 0x00008000 in 0.0 seconds (e
Hash of data verified.

Leaving...
Hard resetting...
DOIT ESP32 DEVKIT V1, 80MHz, 921600, None on COM4
```

6. Open the Arduino IDE Serial Monitor  at a baud rate of 115200.

7. Press the ESP32 on-board **Enable** button and Serial Monitor shows the IP Address, using this IP Address & a Web Browser, we can access the webpage and it shows temperature sensor results in Celsius and Fahrenheit scale.



Serial Monitor Result Web-Browser Result

EXPERIMENT NO. 13

EXPERIMENT NO: 13

NAME:-

MONITORING PERIPHERAL SENSOR'S OUTPUT DATA FROM ANYWHERE VIA THE INTERNET.

OBJECTIVE:-

Below steps shows the method to understand WI-FI Module as web server to monitoring sensors output such as temperature sensor and potentiometer from anywhere via the internet.

EQUIPMENTS:-

- WL-WIFI Trainer 01 no
- USB Cable 01 no
- Power supply 01 no
- PC or Laptop with installed Arduino software.

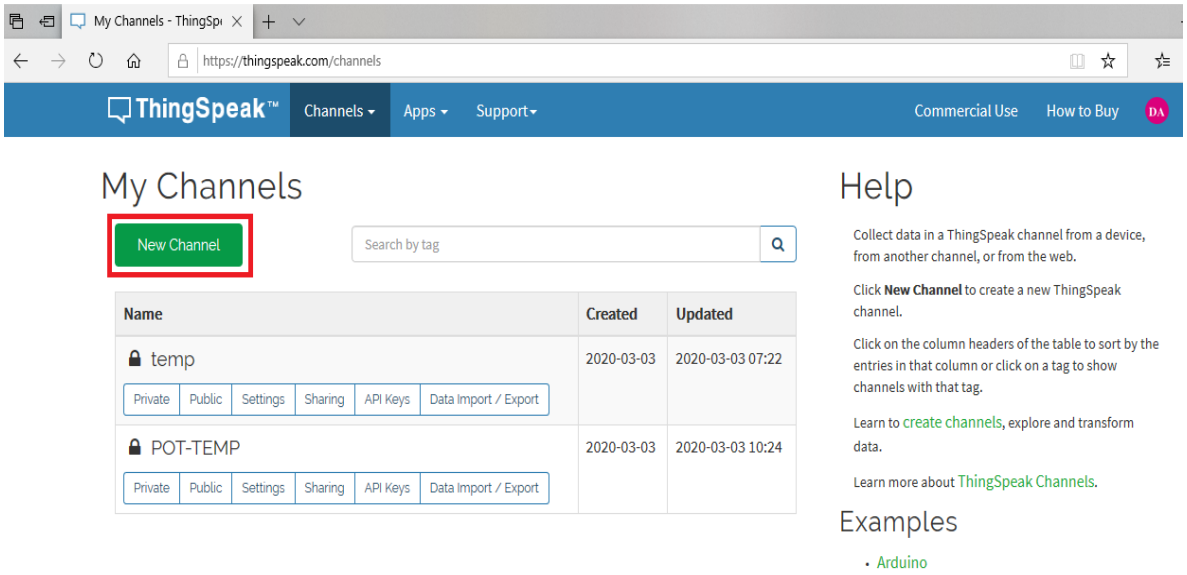
THEORY:-

Internet of Things (IoT) is a concept that aims to expand the benefits of connected internet connectivity continuously -the ability to share data, remote control, and etc, as well as on objects in the real world. For example, food, electronics, collectibles, any equipment, including living things that are all connected to local and global networks through embedded and active sensors.
Here is an example program of Arduino IoT using Wireless Module -how to send data in the form of random value, which can be monitored from anywhere by utilizing the free IoT service provider –Thing Speak.
Thing Speak is a free web service for displaying the data online and you can access and monitor the data from ThingSpeak from anywhere. By using Thing Speak server, we can monitor our data over the internet using the API and channels provided by Thing Speak.

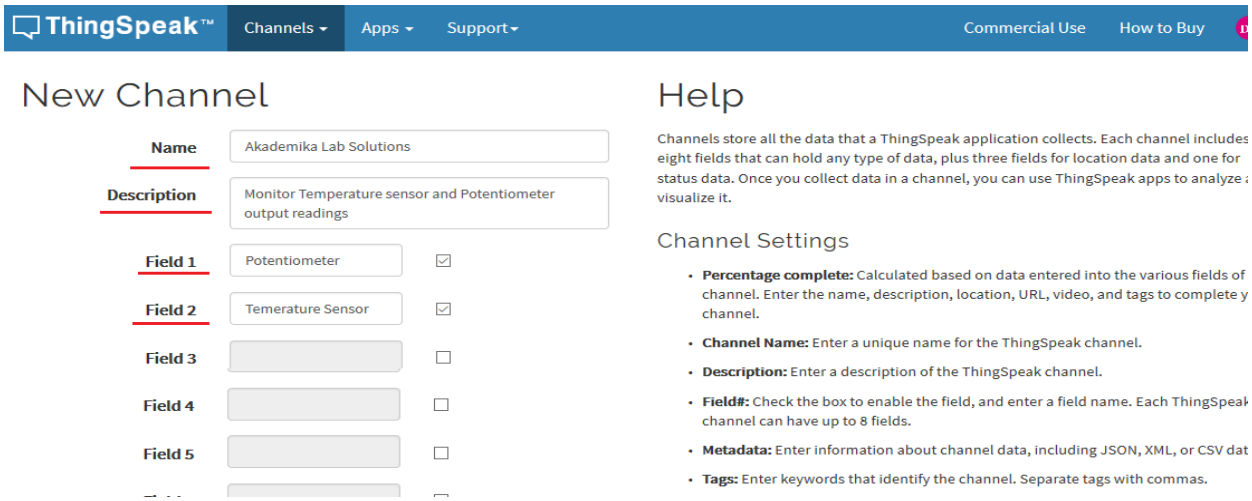
PROCEDURE:-

Go to 'https://thingspeak.com/' and create your Thing Speak Account if you don't have.
Login to Your Account.

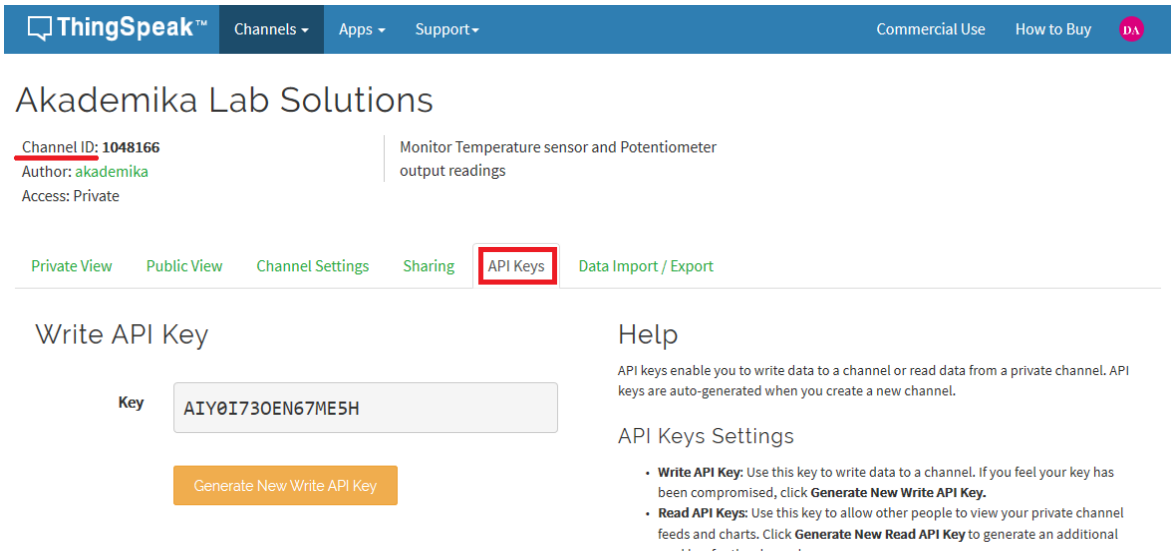
1. Create a Channel by clicking ’New Channel’.



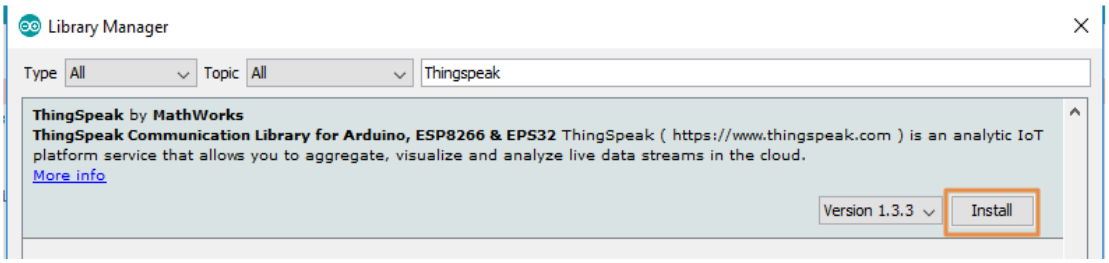
2. Enter the channel details.
   Name: Any Name
   Description: Optional
   Field 1: Potentiometer
   Field 2: Temerature Sensor
   This will be displayed on the analytics graph. If you need more than 1 Channels you can create for additional Sensor Data. Save this setting.



3. Now you can see the channels. Click on the 'API Keys' tab. Here you will get the Channel ID and API Keys. Note this down.



4. Open Arduino IDE and Install the ThingSpeak Library. To do this go to Sketch>Include Library>Manage Libraries. Search for ThingSpeak and install the library.



5. Now, open the given Arduino program file 'pot_temp_web_things' in Arduino software from the given folder path 'WL-WIFI_V19.0\Networking related Experiment Programs\ 7_**DIFFERENT NET PERIPHERAL OUT ON WEB DISPLAY\pot_temp_web_things**'

6. We need to modify the program with your available network credentials. In the below code you need to change your available Network SSID, Password and your ThingSpeak Channel and API Keys.

7. Replace the following content in the code,

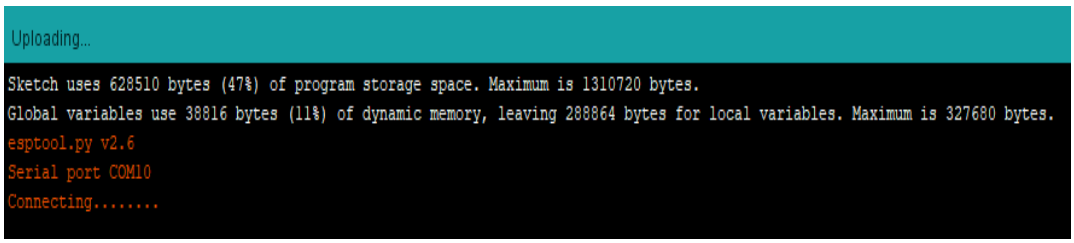
pot_temp_web_things | Arduino 1.8.9

File Edit Sketch Tools Help

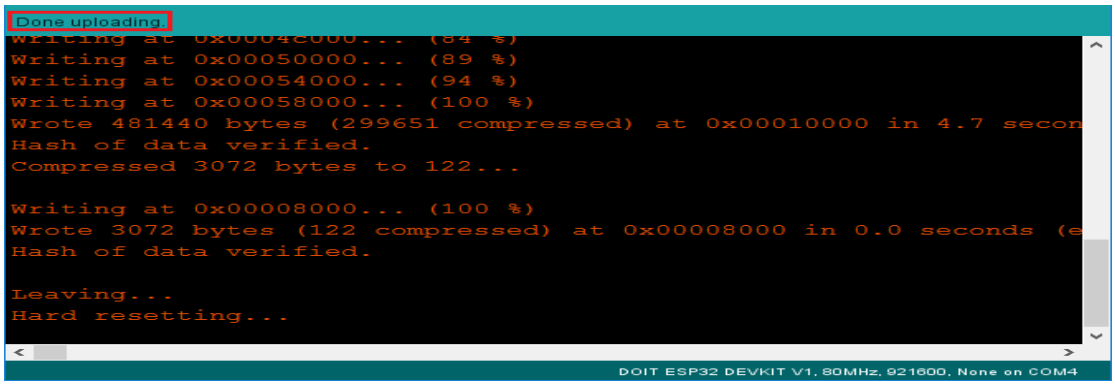
pot_temp_web_things

```
#include <WiFi.h>
const char ssid[] = "DND";
const char password[] = "Akademikadnd";
WiFiClient client;

#include <ThingSpeak.h>
const long CHANNEL = 1008586;
const char *WRITE_API = "Y71JH0JEI0YCEV6K";

#include <Wire.h>
long prevMillisThingSpeak = 0;
int intervalThingSpeak = 20000; // Minimum ThingSpeak write interval is 15 seconds

void setup()
{
  Serial.begin(115200);
  Serial.println();
  Serial.println("Send Sensor's Data to ThingSpeak Using ESP32");
  Serial.println();

  WiFi.mode(WIFI_STA);
  ThingSpeak.begin(client);  // Initialize ThingSpeak
}
```

8. Press the **Upload**  button in the Arduino IDE. After getting following message,

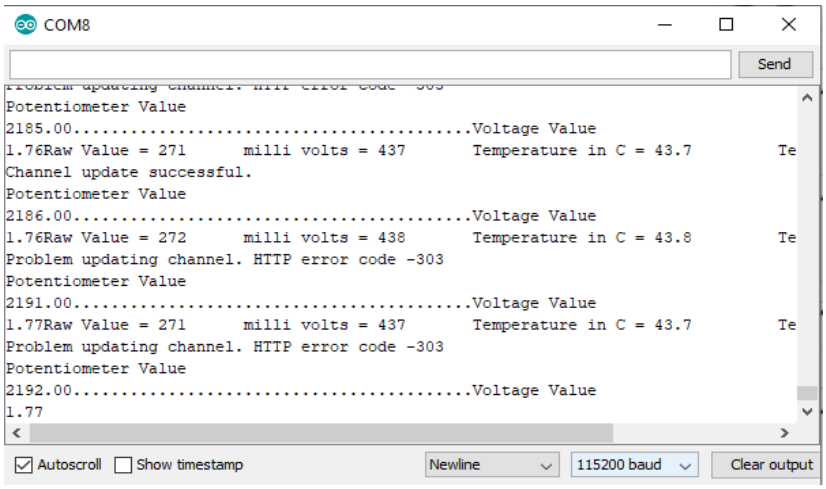


9. You need to Hold-down the "**BOOT**" button in your ESP32 board, Wait a few seconds while the code compiles and uploads to your board.
10. If everything went as expected, you should see a "**Done uploading.**" message.

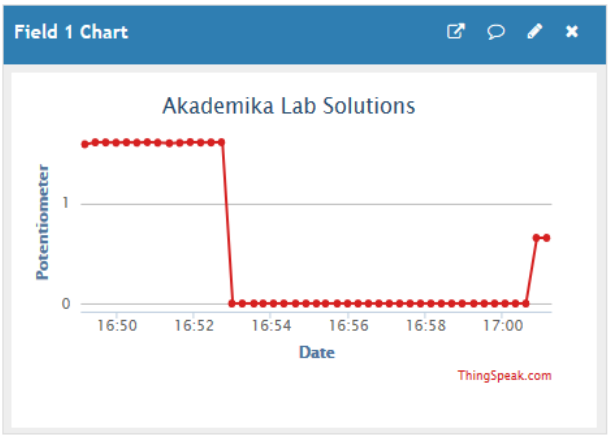


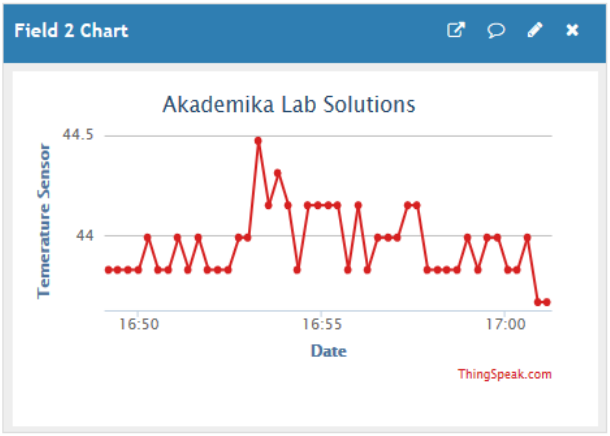
11. Open the **Arduino IDE Serial Monitor**  at a baud rate of 115200.

12. Press the ESP32 on-board **Enable** button and open the Serial Monitor to verify the connectivity of the device and the data sent.



13. Once it is connected to Wi-Fi the data will start uploading to the ThingSpeak Channel. You can now open your Channel and see the data changes plotted on the ThingSpeak.





EXPERIMENT NO. 14

EXPERIMENT NO: 14

NAME:-

CONTROLLING PERIPHERAL'S ON/OFF OPERATION FROM ANYWHERE VIA THE INTERNET.

OBJECTIVE:-

In this experiment, we will figure out how to make an WI-FI module-based web server to control LED and relay state, which is accessible from anywhere in the world.

EQUIPMENTS:-

- WL-WIFI Trainer 01 no
- USB Cable 01 no
- Power supply 01 no
- PC or Laptop with installed Arduino software.

THEORY:-

Access to a local web server from anywhere in the world using Ngrok. It is a platform that allows you to organize remote access to a web server or some other service running on your PC from the external internet. Access is organized through the secure tunnel created at the start of ngrok.
First of all, we will need to install the following libraries on Arduino IDE:
ESPAsyncWebServer and AsyncTCP libraries allow you to create a web server using files from the file system of Wi-Fi module.

PROCEDURE:-

1. Connect USB cable between PC & WIFI trainer.
2. Connect 5V adaptorto the PC.

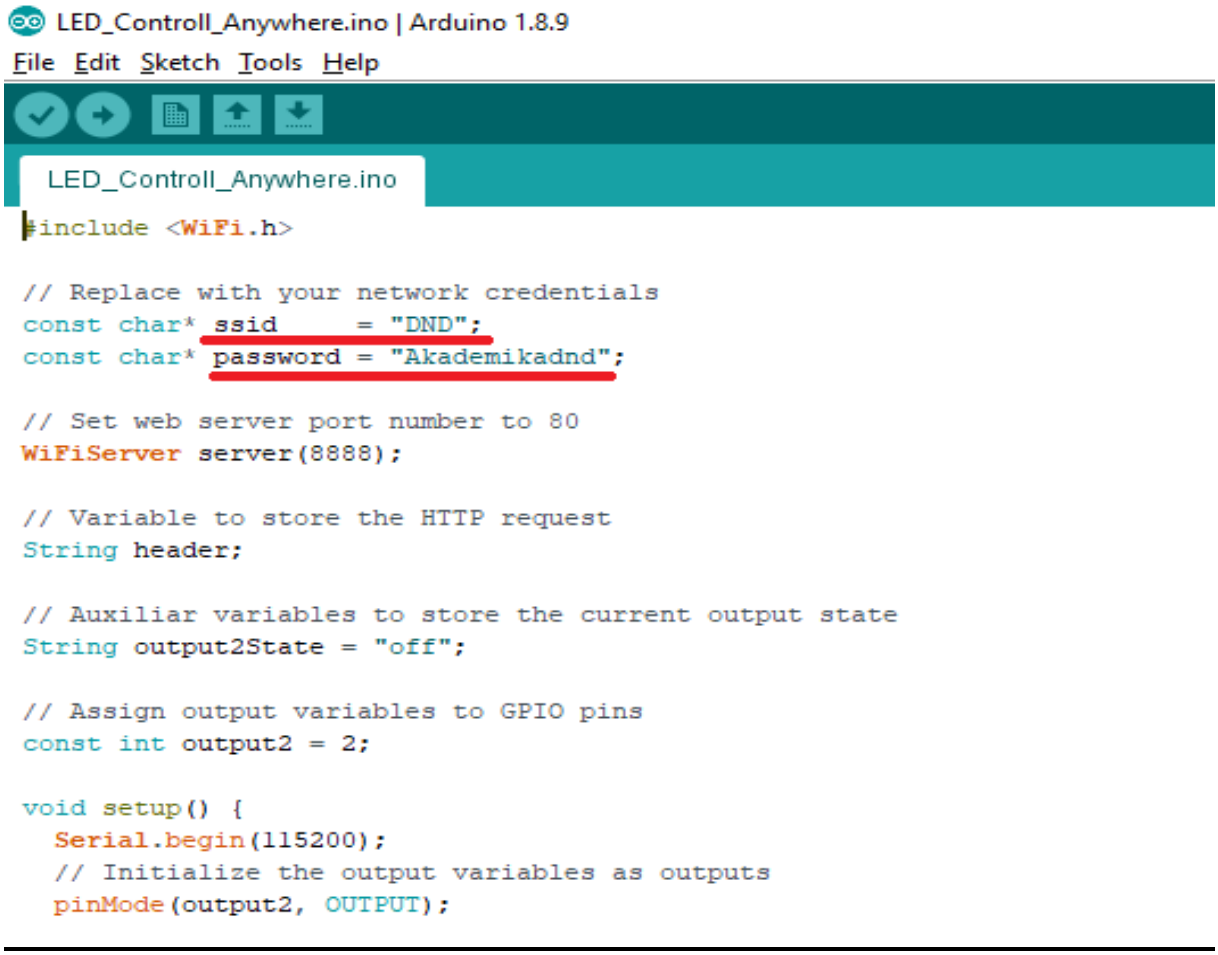
Install the ESPAsyncWebServer library

3. Click on this link https://github.com/me-no-dev/ESPAsyncWebServer/archive/master.zip to download the ZIP archive of the library.

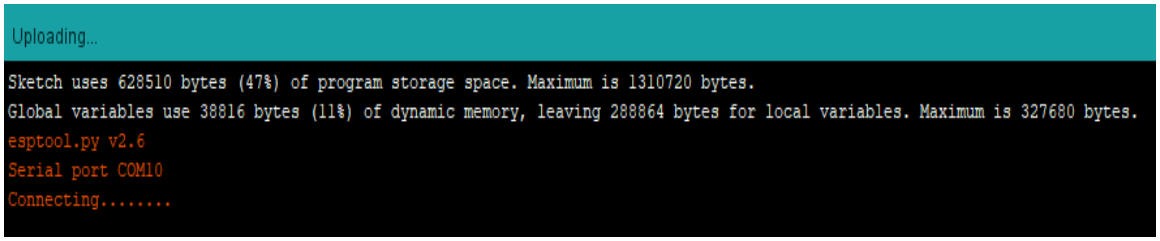
   Unzip this archive. You should get the ESPAsyncWebServer-master folder.
   Rename it to "ESPAsyncWebServer".

Install the AsyncTCP library

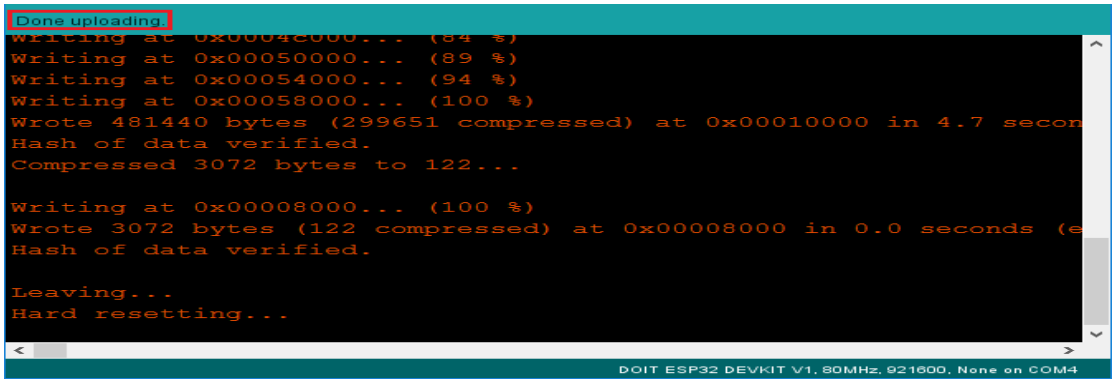
4. Click on this link https://github.com/me-no-dev/AsyncTCP/archive/master.zip to download the ZIP archive of the library.
5. Unzip this archive. You should get the AsyncTCP-master folder.
6. Rename it to “AsyncTCP”.
7. Move the ESPAsyncWebServer and AsyncTCP folders to the libraries folder, which is located inside the Documents directory.
8. Now, Restart the Arduino IDE and open the given Arduino program file ‘LED_Controll_Anywhere’ in Arduino software from the given folder path ‘WL-WIFI_V19.0\Networking related Experiment Programs\ 8_**DIFFERENT NET PERIPHERAL WEB CONTROL\LED_Controll_Anywhere\LED_Controll_Anywhere**’
9. We need to modify the program with your available network credentials. In the below code you need to change your available Network SSID, Password.



10.Press the **Upload**  button in the Arduino IDE. After getting following message

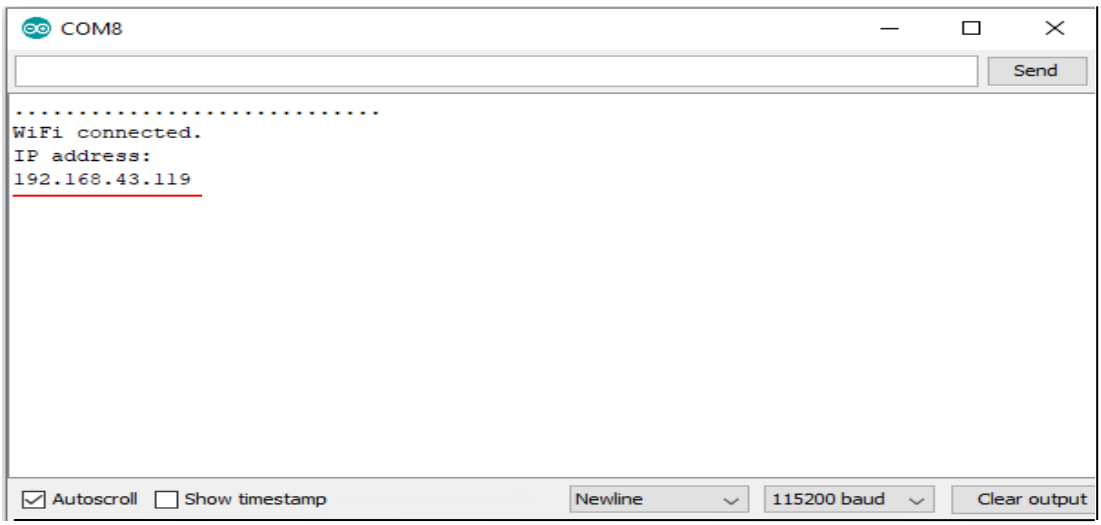


11. You need to Hold-down the "**BOOT**" button in your ESP32 board, Wait a few seconds while the code compiles and uploads to your board.
12. If everything went as expected, you should see a "**Done uploading.**" message.



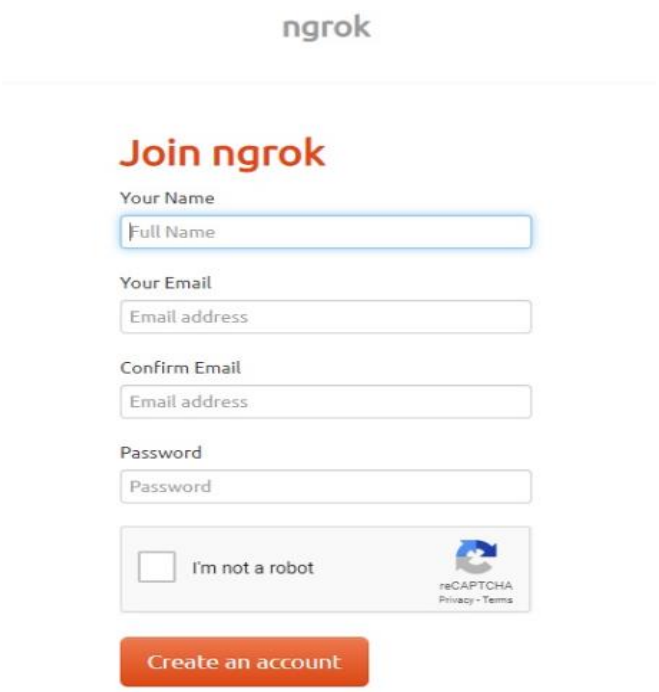
13. Open the **Arduino IDE Serial Monitor**  at a baud rate of 115200.

14. After uploading this code, press the ESP32 on-board Enable button and get the IP address from the serial monitor.



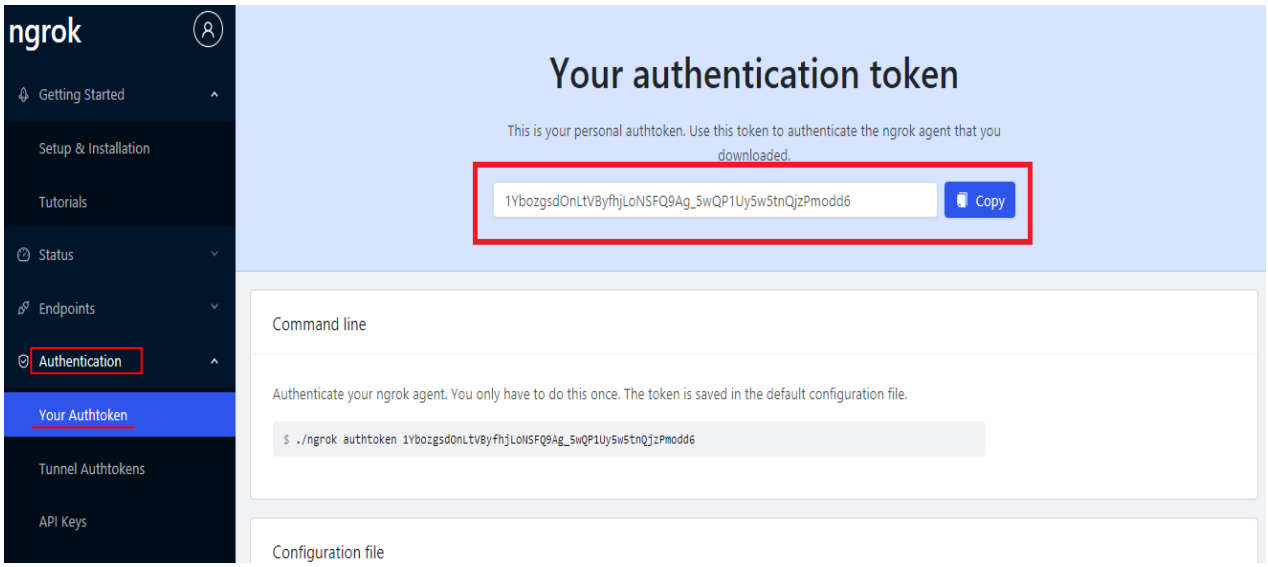
NGROK TUNNEL SERVICE

15. Use a free service known as ngrok which is used as a tunnel to create this feature.
16. Go to this link https://ngrok.com and make your account using your email address. Fill the required details and click on the signup button.

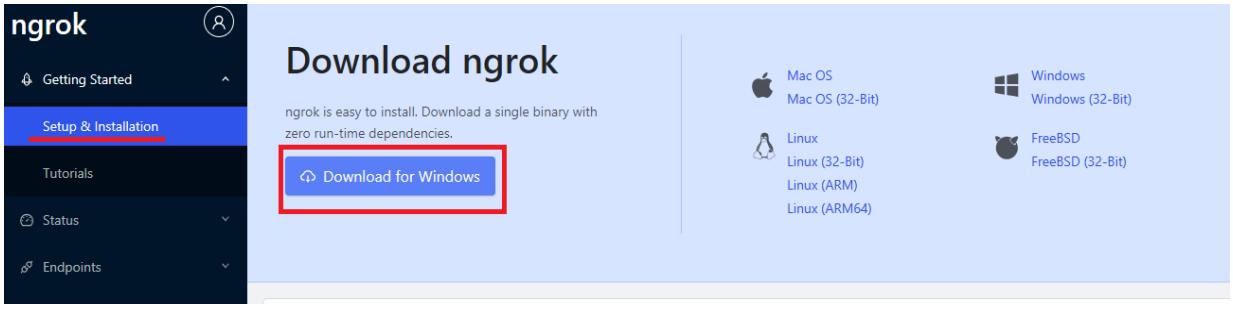


17. After signing up for the account, go to your email address and activate your account by clicking on the activation link.

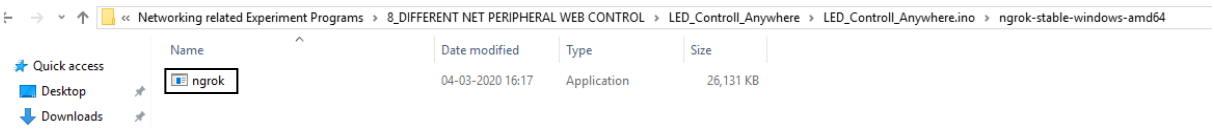
18. Now login to your account and click on the 'Authentication' button' to get your Tunnel Authtoken as shown in the figure below. Copy this Tunnel Authtoken, we will need it later on.



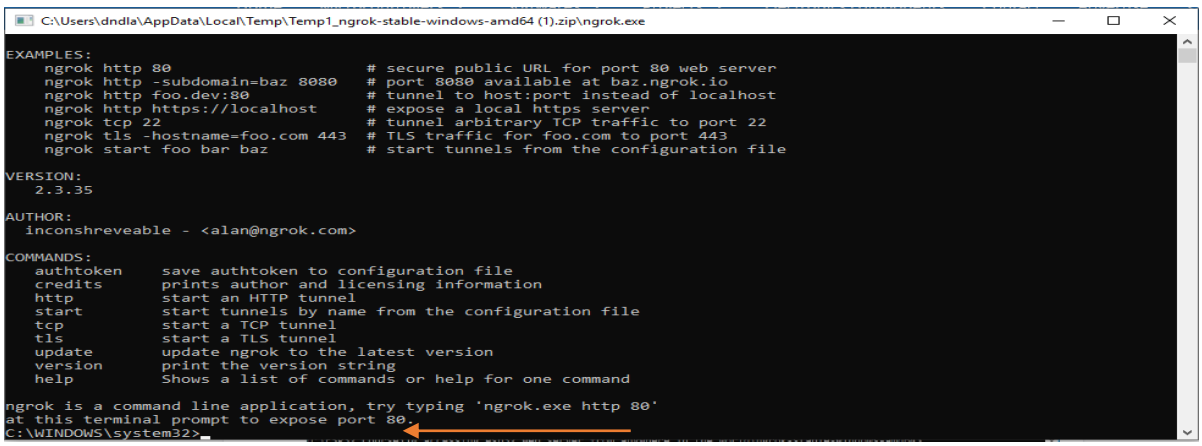
19. Now click on download button and download this software according to the operating system you are using.



20. After you have done with downloading, you will get a compressed folder. UnZip it using any software.
21. Now double click on a ngrok icon to open it.



22. After opening this application, you will see this windows command line. Now by using ‘cd’ command, change the current directory to folder where the ‘ngrok’ is installed.



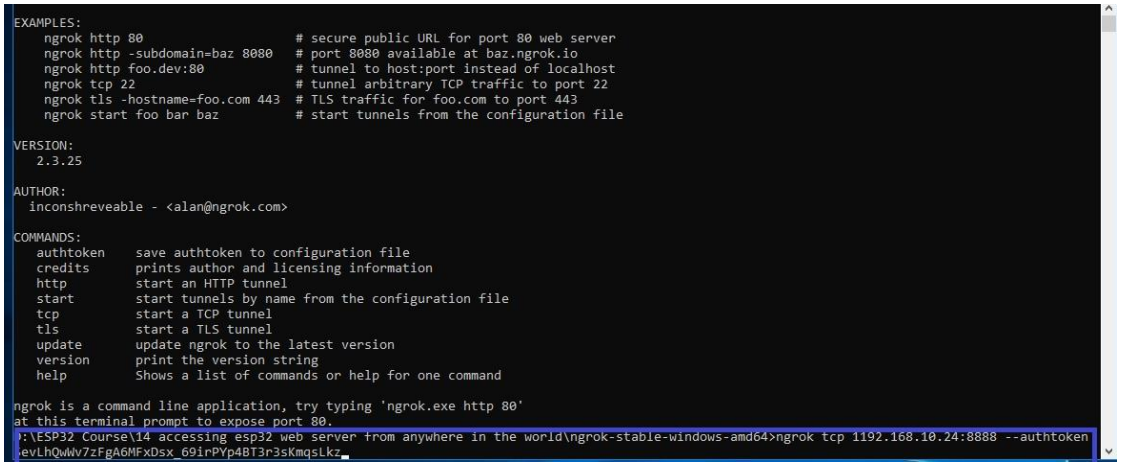
23. Now enter following line in windows terminal and replace IP_ADDRESS with your IP address from serial monitor and TUNNEL_AUTHTOKEN with your ngrok authentication code which we noted in last steps.

```
ngrok tcp IP_ADDRESS:8888 --authtoken TUNNEL_AUTHTOKEN
```

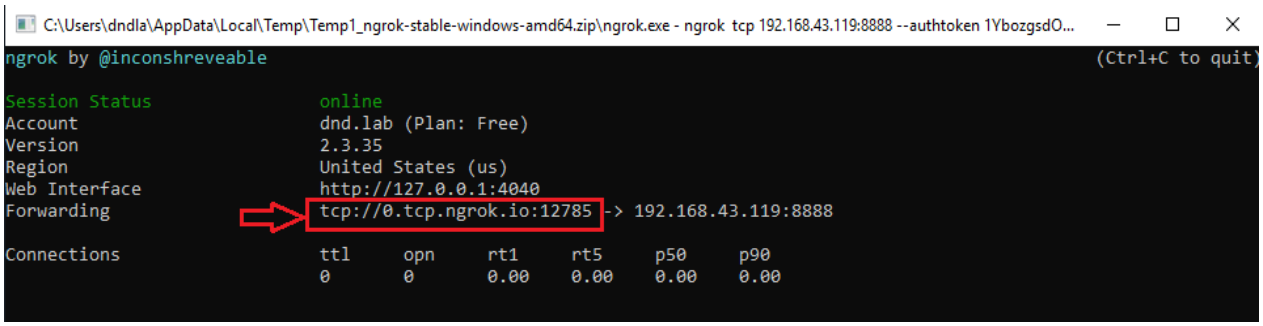
After adding IP address and Tunnel token it is how it looks.

```
ngrok tcp 1192.168.10.24:8888 –authtoken
5evLhQwWv7zFgA6MFxDsx_69irPYp4BT3r3sKmqsLkz
```

This is where you need to paste this command and hit enter.



24. After you hit the enter button, you will see this window. This window shows the tunnel URL as highlighted in this picture. We will use this URL to access the web server instead of an IP address.



25. Now you can copy this URL and note it down. You can access the web server from anywhere in the world using this URL.
26. If you see the URL it is of TCP type tcp://0.tcp.ngrok.io:12785 but you can replace TCP with HTTP and enter the URL in any web browser, you will be able to access the web server.